

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



THÁI ĐỖ THỊNH

**XÂY DỰNG HỆ THỐNG WEB TÌM VIỆC LÀM
TÍCH HỢP AI - FLEXICONNECT**

**ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH**

TP. HỒ CHÍ MINH, 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



THÁI ĐỖ THỊNH

XÂY DỰNG HỆ THỐNG WEB TÌM VIỆC LÀM
TÍCH HỢP AI - FLEXICONNECT

Mã số sinh viên: 2251010086

ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH

Giảng viên hướng dẫn: TS. PHAN TRẦN MINH KHUÊ

TP. HỒ CHÍ MINH, 2025

TRƯỜNG ĐẠI HỌC MỞ CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
THÀNH PHỐ HỒ CHÍ MINH Độc lập – Tự do – Hạnh phúc
KHOA CÔNG NGHỆ THÔNG TIN

GIẤY XÁC NHẬN

Tôi tên là: Thái Đỗ Thịnh

Ngày sinh: 21/02/2004

Nơi sinh: An Giang

Chuyên ngành: Khoa học máy tính Mã sinh viên: 2251010086

Tôi đồng ý cung cấp toàn văn thông tin đồ án/ khóa luận tốt nghiệp hợp lệ về bản quyền cho Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh. Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh sẽ kết nối toàn văn thông tin đồ án/ khóa luận tốt nghiệp vào hệ thống thông tin khoa học của Sở Khoa học và Công nghệ Thành phố Hồ Chí Minh.

Ký tên
(Ghi rõ họ và tên)

.....

LỜI CẢM ƠN

Trong suốt hành trình hoàn thành đồ án tốt nghiệp cũng là những khoảng thời gian cuối cùng trên chặng đường đại học tại Trường Đại học Mở Thành phố Hồ Chí Minh. Để có được thành quả như ngày hôm nay là nhờ có sự quan tâm, động viên đến từ gia đình, sự tận tụy của thầy cô trong trường và sự sẻ chia đến từ bạn bè.

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc và chân thành đến thầy Phan Trần Minh Khuê. Từ những lời hỏi thăm, góp ý vô cùng chân thành về những kiến thức vô cùng quý giá của thầy đã có thể dẫn lối em qua từng giai đoạn của đồ án. Em rất biết ơn với những đóng góp kịp thời và sự kiên nhẫn của thầy trong quá trình em thực hiện đề tài này. Bên cạnh đó em cũng gửi lời cảm ơn đến các thầy cô trong khoa Công nghệ thông tin đã tâm huyết truyền dạy những kiến thức quý giá và bổ ích đến em và các bạn sinh viên nói chung. Đây là một nền tảng vững chắc để em gói lại và đi tiếp trên hành trình sự nghiệp của mình.

Qua đề tài này, em cũng nhận thấy bản thân còn hạn chế khi phát triển đồ án nên sẽ không tránh khỏi những thiếu sót. Em rất mong nhận được những đóng góp chân thành từ quý thầy cô để có thể hoàn thiện hơn trong tương lai.

Em xin chân thành cảm ơn.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TÓM TẮT ĐỒ ÁN NGÀNH

Đề tài “Hệ thống tìm việc làm” được phát triển nhằm xây dựng một hệ thống đơn giản nhưng vẫn hiện đại với mục tiêu cốt lõi là xây dựng cầu nối hiệu quả giữa nhà tuyển dụng và các ứng viên tiềm năng.

Về mặt kỹ thuật, hệ thống phát triển với kiến trúc đa dạng gồm: phần máy chủ (**Spring Boot**), giao diện người dùng (**ReactJS**). Đây là một sự kết hợp mạnh mẽ để đảm bảo hiệu suất cao và giao diện thân thiện với người dùng. Cơ sở dữ liệu (**MySQL**) đảm bảo tính toàn vẹn, ổn định và truy vấn dữ liệu một cách hiệu quả. Thay vì chạy theo số lượng tính năng, tập trung vào chất lượng xây dựng các chức năng cốt lõi từ đăng ký/ đăng nhập tài khoản, tạo hồ sơ cá nhân, tìm kiếm đến ứng tuyển một cách trực quan và hiệu quả nhất. Để bảo mật và phân quyền, hệ thống sử dụng **JSON Web Token (JWT)** xác thực các **API**, qua đó đảm bảo rằng mọi yêu cầu gửi đến máy chủ đều được xác thực an toàn và chỉ những người dùng có quyền mới có thể truy cập vào các tài nguyên tương ứng.

Đặc biệt, có thêm tích hợp **trí tuệ nhân tạo (AI)** vào hệ thống, **AI** sẽ phân tích và đưa ra những gợi ý chuyên sâu, giúp người dùng tối ưu hóa hồ sơ tạo điểm sáng trong mắt các nhà tuyển dụng. Ngoài ra, chức năng luyện tập phỏng vấn giúp người dùng nâng cao sự tự tin và kỹ năng phản xạ trước các câu hỏi phỏng vấn thực tế. Sẽ là một nền tảng vững chắc để bước vào con đường tìm việc hiệu quả, thành công nhất.

Kết quả đạt được là một nền tảng tìm việc làm tối giản và hiện đại, giúp cải thiện sự rối rắm thường thấy ở các trang web tuyển dụng hiện nay. Đây là một hệ thống mở có thể ứng dụng cho tất cả mọi người, trọng điểm là các bạn sinh viên mới ra trường đang có nhu cầu muốn tìm kiếm một bước tiến mới trong sự nghiệp.

ABSTRACT

The topic “Job Search System” is developed to build a simple but modern system with the core goal of building an effective bridge between employers and potential candidates.

Technically, the system is developed with a diverse architecture including: server (**Spring Boot**), user interface (**ReactJS**). This is a powerful combination to ensure high performance and user-friendly interface. The database (**MySQL**) ensures integrity, stability and efficient data querying. Instead of chasing the number of features, I focus on the quality of building core functions from account registration/login, creating personal profile, searching to applying in the most intuitive and effective way. For security and authorization, the system uses **JSON Web Token (JWT)** to authenticate **APIs**, thereby ensuring that all requests sent to the server are securely authenticated and only authorized users can access the corresponding resources.

In particular, I have integrated **artificial intelligence (AI)** into the system, **AI** will analyze and give in-depth suggestions, helping users optimize their profiles to create highlights in the eyes of recruiters. In addition, the interview practice function helps users improve their confidence and reflex skills in response to real interview questions. It will be a solid foundation for entering the most effective and successful job search path.

The result is a minimalist and modern job search platform, helping to improve the confusion often found on current recruitment websites. This is an open system that can be applied to everyone, focusing on new graduates who are looking for a new step in their career.

MỤC LỤC

LỜI CẢM ƠN	2
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN.....	3
TÓM TẮT ĐỒ ÁN NGÀNH.....	4
ABSTRACT	5
DANH MỤC TỪ VIẾT TẮT	9
DANH MỤC HÌNH VẼ	10
DANH MỤC BẢNG	12
MỞ ĐẦU.....	13
Chương 1. TỔNG QUAN ĐỀ TÀI.....	14
1.1. Giới thiệu đề tài.....	14
1.2. Lý do chọn đề tài.....	14
1.3. Mục tiêu đề tài.....	15
1.4. Phương pháp thực hiện	15
1.5. Bố cục đề tài.....	16
Chương 2. CƠ SỞ LÝ THUYẾT	18
2.1. Backend Framework - Spring Boot	18
2.1.1. Giới thiệu Spring Boot.....	18
2.1.2. Tại sao lại chọn Spring Boot	18
2.1.3. Kiến trúc Spring Boot.....	21
2.1.4. Spring Security	25
2.2. Frontend ReactJS	29
2.2.1. Giới thiệu về ReactJS	29
2.2.2. Lý do lựa chọn ReactJS thay thế Thymeleaf.....	30
2.2.3. Các công nghệ được sử dụng trong ReactJS	31
2.2.4. Kiến trúc kết nối ReactJS với Backend	32

2.3.	n8n.....	32
2.3.1.	Giới thiệu về n8n	32
2.3.2.	Tại sao lại chọn n8n.....	33
2.3.3.	Kiến trúc cốt lõi: Node, Workflow, Mô hình Hướng sự kiện	34
2.3.4.	Triển khai Kiến trúc và Tùy chỉnh.....	35
2.3.5.	Triển khai và Phân tích Luồng công việc Cốt lõi.....	36
2.4.	Các công cụ và thư viện hỗ trợ	46
2.4.1.	Docker.....	46
2.4.2.	Nền tảng Triển khai: Render và Clever Cloud	49
Chương 3.	HỆ THỐNG WEB TÌM VIỆC LÀM	51
3.1.	Giới thiệu hệ thống web tìm việc làm.....	51
3.2.	Kiến trúc hệ thống.....	52
3.3.	Phân tích hệ thống.....	53
3.3.1.	Use case Tổng quan hệ thống	53
3.3.2.	Đặc tả use case Tổng quan hệ thống	54
3.3.3.	Use case Ứng viên	56
3.3.4.	Đặc tả use case Ứng viên.....	56
3.3.5.	Use case Tổng quan quy trình tuyển dụng	58
3.3.6.	Đặc tả Use case Tổng quan quy trình tuyển dụng	59
3.4.	Thiết kế hệ thống.....	61
3.4.1.	Sơ đồ tuần tự (Sequence diagram) và Sơ đồ trạng thái (State diagram)	61
3.4.2.	Sơ đồ hoạt động (Activity diagram)	62
3.4.3.	Thiết kế cơ sở dữ liệu	63
3.4.4.	Thiết kế giao diện	68
Chương 4.	KẾT QUẢ ĐẠT ĐƯỢC.....	73

4.1.1.	Đăng ký, đăng nhập vào hệ thống	73
4.1.2.	Ứng tuyển việc làm.....	74
4.1.3.	Chức năng đăng tin và quản lý tin.....	77
4.1.4.	Các chức năng AI	79
4.1.5.	Chức năng của quản trị viên	83
4.1.6.	Các chức năng phụ.....	86
Chương 5.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	88
5.1.	Kết luận	88
5.2.	Hướng phát triển	88
5.3.	Thử thách và khó khăn.....	90
TÀI LIỆU THAM KHẢO		93
PHỤ LỤC		94

DANH MỤC TỪ VIẾT TẮT

JWT	JSON Web Token
UI	User Interface
API	Application Programming Interface
DB	Database
CI/CD	Continuous Integration/Continuous Delivery
REST	REpresentational State Transfer
MVC	Model-View-Controller
CRUD	Create, Read, Update, Delete.
JSX	JavaScript XML
PaaS	Platform-as-a-Service
DBaaS	Database-as-a-Service

DANH MỤC HÌNH VẼ

Hình 1.1: Mô phỏng hệ thống tìm việc làm	14
Hình 2.1: Spring Boot.....	18
Hình 2.2: So sánh Spring Boot và Spring truyền thống	19
Hình 2.3: Kiến trúc Spring Boot.....	22
Hình 2.4: Sơ đồ chứng thực tài khoản.....	26
Hình 2.5: Luồng xác thực JWT trong Spring Security	27
Hình 2.6: Biên dịch JSX trong React.....	30
Hình 2.7: ReactJS so sánh với Thymeleaf.....	31
Hình 2.8: Các công nghệ được sử dụng trong ReactJS	32
Hình 2.9: n8n.....	33
Hình 2.10: Workflow gửi mail chào mừng của n8n	35
Hình 2.11: Workflow Cải thiện hồ sơ người dùng	36
Hình 2.12: Workflow Mô phỏng Phỏng vấn thử	40
Hình 2.13: Render	49
Hình 2.14: Clever Cloud	50
Hình 3.1: Luồng xử lý dữ liệu hệ thống web tìm việc làm.....	52
Hình 3.2: Sơ đồ Use case Hệ thống tìm việc	54
Hình 3.3: Sơ đồ Use case Tổng quan quy trình tuyển dụng.....	58
Hình 3.4: Sơ đồ quy trình ứng tuyển	61
Hình 3.5: Sơ đồ trạng thái ứng tuyển	62
Hình 3.6: Sơ đồ hoạt động quy trình ứng tuyển	62
Hình 3.7: Lược đồ cơ sở dữ liệu toàn bộ hệ thống.....	63
Hình 3.8: Lược đồ người dùng và hồ sơ người dùng	63
Hình 3.9: Lược đồ hồ sơ ứng viên.....	64
Hình 3.10: Lược đồ quy trình ứng tuyển và giao tiếp	65
Hình 3.11: Lược đồ tính năng AI và tương tác	65
Hình 3.12: Lược đồ dịch vụ thanh toán và thông báo.....	66
Hình 3.13: Giao diện trang tìm việc.....	68
Hình 3.14: Giao diện chức năng ứng tuyển	70
Hình 3.15: Giao diện trang xét duyệt ứng viên	71
Hình 4.1: Trang đăng ký dành cho nhà tuyển dụng (PUBLIC)	73

Hình 4.2: Trang đăng ký dành cho ứng viên (PUBLIC)	73
Hình 4.3: Trang đăng nhập (PUBLIC)	74
Hình 4.4: Trang chủ của ứng viên (CANDIDATE)	74
Hình 4.5: Trang chi tiết việc làm (PUBLIC)	75
Hình 4.6: Dialog ứng tuyển (CANDIDATE)	75
Hình 4.7: Trang quản lý hồ sơ ứng tuyển (EMPLOYER)	76
Hình 4.8: Trang theo dõi tin đã ứng tuyển (CANDIDATE)	76
Hình 4.9: Trạng thái rút đơn được cập nhật ở trang quản lý ứng tuyển (EMPLOYER)	77
Hình 4.10: Trang chủ đăng tin của nhà tuyển dụng (EMPLOYER)	77
Hình 4.11: Dialog nhập thông tin bài đăng việc làm (EMPLOYER)	78
Hình 4.12: Thông tin chi tiết của bài đăng việc làm (1) (PUBLIC)	78
Hình 4.13: Thông tin chi tiết của bài đăng việc làm (2) (PUBLIC)	79
Hình 4.14: Chức năng cải thiện hồ sơ với AI (chưa nâng cấp tài khoản) (CANDIDATE).....	79
Hình 4.15: Trang luyện tập phỏng vấn AI (chưa nâng cấp tài khoản) (CANDIDATE)	80
Hình 4.16: Các gói nâng cấp (CANDIDATE)	80
Hình 4.17: Gợi ý cải thiện hồ sơ từ AI (CANDIDATE)	81
Hình 4.18: Trang luyện tập phỏng vấn AI (CANDIDATE)	81
Hình 4.19: Chức năng luyện tập phỏng vấn với AI (CANDIDATE)	82
Hình 4.20: Trang kết quả phỏng vấn luyện tập (CANDIDATE)	83
Hình 4.21: Trang chủ của quản trị viên (ADMIN) (1)	83
Hình 4.22: Trang chủ của quản trị viên (ADMIN) (2)	84
Hình 4.23: Trang duyệt nhà tuyển dụng (ADMIN)	84
Hình 4.24: Trang quản lý người dùng (ADMIN)	85
Hình 4.25: Trang quản lý tin tuyển dụng (ADMIN)	85
Hình 4.26: Nhận thông báo Gmail từ hệ thống	86
Hình 4.27: Chức năng thông báo	86
Hình 4.28: Đổi chủ đề cho toàn hệ thống (LightMode/DarkMode).....	87
Hình 4.29: Bản đồ địa chỉ công ty bài đăng việc làm	87

DANH MỤC BẢNG

<i>Bảng 2.1: Lớp chạy ứng dụng</i>	<i>20</i>
<i>Bảng 2.2: REST Controller xử lý yêu cầu</i>	<i>23</i>
<i>Bảng 2.3: Service xử lý logic nghiệp vụ</i>	<i>23</i>
<i>Bảng 2.4: Repository truy xuất dữ liệu</i>	<i>24</i>
<i>Bảng 2.5: Đoạn mã cấu hình SecurityFilterChain chính.....</i>	<i>29</i>
<i>Bảng 2.6: Phân tích Webhook (Trigger Node).....</i>	<i>37</i>
<i>Bảng 2.7: Chuẩn hóa dữ liệu đầu vào.....</i>	<i>37</i>
<i>Bảng 2.8: Phân tích Prepare Initial Data (Code Node)</i>	<i>38</i>
<i>Bảng 2.9: Cấu hình HTTP Request</i>	<i>38</i>
<i>Bảng 2.10: Phân tích HTTP Request (API Communication Node)</i>	<i>39</i>
<i>Bảng 2.11: Parse kết quả trả về từ AI thành JSON chuẩn.....</i>	<i>39</i>
<i>Bảng 2.12: Phân tích Respond to Webhook (Response Node).....</i>	<i>39</i>
<i>Bảng 2.13: Phân tích Webhook (interview-analysis) – Trigger Node</i>	<i>41</i>
<i>Bảng 2.14: Cấu hình Interview Analyst (HTTP Request Node).....</i>	<i>41</i>
<i>Bảng 2.15: Phân tích Interview Analyst (HTTP Request Node)</i>	<i>42</i>
<i>Bảng 2.16: Code xử lý phản hồi AI</i>	<i>42</i>
<i>Bảng 2.17: Phân tích Process AI Response (Code Node).....</i>	<i>43</i>
<i>Bảng 2.18: Cấu hình Respond to Webhook (Response Node)</i>	<i>43</i>
<i>Bảng 2.19: Phân tích Question Webhook (generate-question) – Trigger Node</i>	<i>43</i>
<i>Bảng 2.20: Cấu hình Question Webhook (generate-question) – Trigger Node.....</i>	<i>43</i>
<i>Bảng 2.21: Phân tích Question Generate (HTTP Request Node)</i>	<i>44</i>
<i>Bảng 2.22: Question Generate (HTTP Request Node)</i>	<i>44</i>
<i>Bảng 2.23: Phân tích Question Generate (HTTP Request Node)</i>	<i>45</i>
<i>Bảng 2.24: Code xử lý câu hỏi từ AI sinh ra</i>	<i>45</i>
<i>Bảng 2.25: Phân tích Process Question Response (Code Node).....</i>	<i>45</i>
<i>Bảng 2.26: Cấu hình Respond Question (Response Node)</i>	<i>46</i>
<i>Bảng 2.27: Phân tích Respond Question (Response Node)</i>	<i>46</i>
<i>Bảng 3.1: Đặc tả use case Tổng quan hệ thống.....</i>	<i>55</i>
<i>Bảng 3.2: Đặc tả use case Ứng viên</i>	<i>58</i>
<i>Bảng 3.3: Đặc tả Use case Tổng quan quy trình tuyển dụng.....</i>	<i>60</i>
<i>Bảng 5.1: Mô tả chi tiết các bảng cơ sở dữ liệu</i>	<i>97</i>

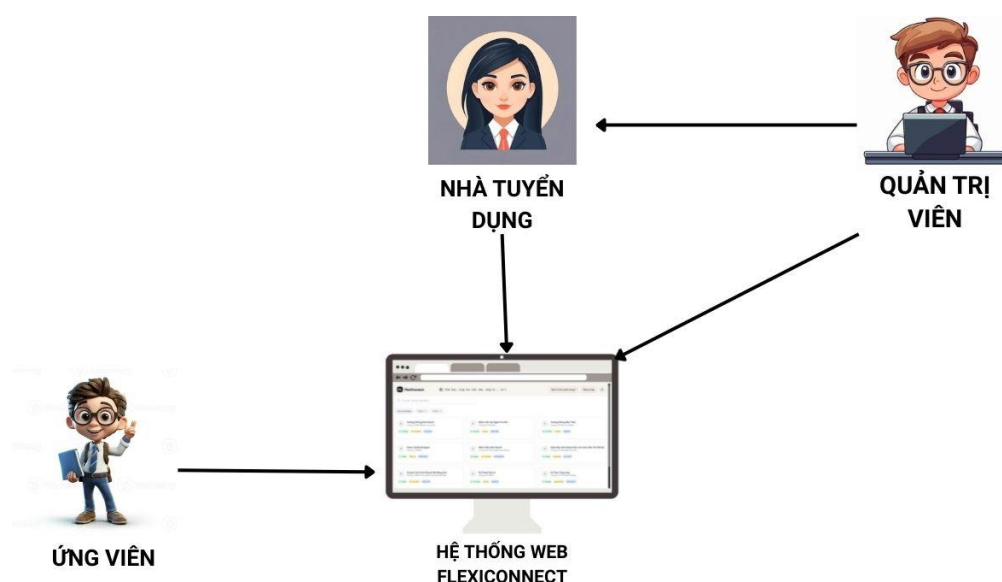
MỞ ĐẦU

Ngày nay, công nghệ thông tin phát triển nhanh chóng, kéo theo xu hướng tìm việc làm trên nền tảng trực tuyến ngày càng phổ biến. Nhiều bạn trẻ và người đi làm đã quen với việc tạo hồ sơ trực tuyến, tìm kiếm và ứng tuyển qua web thay vì ứng tuyển trực tiếp. Tuy nhiên, trên thực tế, quá trình tìm kiếm việc làm vẫn còn nhiều bất tiện và ví dụ như công cụ chọn lọc chưa hiệu quả hoặc giao diện chưa rõ ràng dẫn đến khó sử dụng. Điều này cho thấy các ứng dụng web trong lĩnh vực tuyển dụng tại Việt Nam vẫn còn rất nhiều tiềm năng phát triển. Từ nhu cầu thực tiễn đó, tôi lựa chọn đề tài **“Hệ thống web tìm kiếm việc làm”** với mục tiêu xây dựng một nền tảng cơ bản nhưng đầy đủ các chức năng cốt lõi, đồng thời tích hợp trí tuệ nhân tạo (AI) để hỗ trợ cải thiện hồ sơ ứng viên, giúp hồ sơ trở nên hiệu quả và gây ấn tượng hơn với nhà tuyển dụng. Ngoài ra, đề tài còn hướng đến việc phát triển thêm chức năng phù hợp với xu hướng thị trường hiện nay là phỏng vấn luyện tập (Mock Interview) để người dùng có thể nâng cao trình độ phỏng vấn của mình và tự tin ứng tuyển vào những vị trí mong muốn.

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Đề tài “Hệ thống web tìm việc làm” là một trang web kết nối giữ người tìm việc và nhà tuyển dụng. Hệ thống cho phép người dùng đăng ký và đăng nhập tài khoản với hai vai trò chính là: ứng viên và nhà tuyển dụng. Ứng viên ở đây bao gồm những người lao động hay các bạn sinh viên cần kiếm thêm thu nhập, có thể cập nhật hồ sơ cá nhân, tìm kiếm việc làm theo từ khóa, theo kỹ năng mình mong muốn, xem chi tiết công việc để có thể ứng tuyển và bên cạnh đó nhà tuyển dụng cũng có thể tiếp cận ứng viên một cách nhanh chóng.



Hình 1.1: Mô phỏng hệ thống tìm việc làm

1.2. Lý do chọn đề tài

Bản thân là một sinh viên, tôi cũng trăn trở về việc tìm kiếm một công việc phù hợp để có thêm thu nhập và tích lũy kinh nghiệm và tìm kiếm nguồn thu nhập với các việc làm phù hợp với bản thân. Từ quá trình đó, tôi phần nào nhận ra các không phải trang web nào cũng dễ dàng sử dụng và bởi giao diện khá rối rắm dẫn đến trải nghiệm cần mất nhiều thời gian hơn. Trong khi đó, vài năm gần đây thì thị trường lao động cũng đang có xu hướng giảm do sự xuất hiện đột ngột và phát triển vượt bậc của trí tuệ nhân tạo khiến AI dần thay thế vị trí việc làm của con người.

Từ những kiến thức đã học ở nhà trường, tôi chọn xây dựng một hệ thống tìm việc làm để vừa áp dụng thực tế, vừa mang lại trải nghiệm người dùng dễ sử dụng và phù hợp

với nhu cầu hiện nay. Đề tài cũng giúp tôi được trau dồi thêm nhiều kỹ năng lập trình full-stack và làm quen với quy trình phát triển phần mềm chuyên nghiệp, thực tế.

1.3. Mục tiêu đề tài

Hệ thống trang web tìm việc làm sẽ cung cấp cho người dùng một trải nghiệm liền mạch thông qua các chức năng tuy đơn giản nhưng mang lại hiệu quả cao, nhắm đến mong muốn trọng tâm của người dùng. Với các chức năng cơ bản của người dùng như đăng ký tài khoản, đăng nhập, cập nhật hồ sơ cá nhân. Đối với các Ứng viên (**Candidate**) hệ thống trang bị các công cụ tìm kiếm việc làm dễ dàng với các bộ lọc chi tiết phù hợp với bản thân, cho phép theo dõi các nhà tuyển dụng, lưu lại các tin tuyển dụng mình quan tâm. Tích hợp thanh toán trực tiếp với ví điện tử Momo để trải nghiệm gói tài khoản cao cấp mở khóa trí tuệ nhân tạo nhận gợi ý hồ sơ từ AI, phỏng vấn luyện tập với AI. Đối với Nhà tuyển dụng (**Employer**), mục tiêu là xây dựng hệ thống tối ưu hóa và đơn giản quá trình tuyển dụng, giúp học nhanh chóng tìm được ứng viên phù hợp với giao diện đăng tin và quản lý tin trực quan, dễ sử dụng và hiển thị đầy đủ thông tin của ứng viên để dễ dàng sàng lọc, liên hệ. Cuối cùng là bảng điều khiển tổng quát cho Quản trị viên (**Admin**), cung cấp các báo cáo, thống kê trực quan của hệ thống(số lượng người dùng, bài đăng tuyển, ...), các chức năng quản lý tài khoản người dùng và các bài đăng tuyển dụng (khóa, ẩn, hiện, ...), quy trình kiểm duyệt nhà tuyển dụng để đảm bảo chất lượng của hệ thống.

1.4. Phương pháp thực hiện

Để đạt được các mục tiêu đã đề ra, tôi đã áp dụng một quy trình phát triển có hệ thống, từ khâu tìm hiểu thực trạng đến triển khai sản phẩm, dựa trên các phương pháp và công nghệ hiện đại.

Đầu tiên, bắt đầu bằng việc nghiên cứu và phân tích các nền tảng tìm việc làm phổ biến hiện nay như TopCV, VietnamWorks, Indeed để hiểu rõ thực trạng của thị trường. Qua quá trình khảo sát này, mặc dù các website có nhiều ưu điểm như cung cấp nguồn thông tin việc làm rất đa dạng và tích hợp được rất nhiều chức năng hữu ích, nhưng cũng tồn tại một vài vấn đề có thể ảnh hưởng đến trải nghiệm của người dùng. Điều mà các người dùng hiện nay đang cảm thấy khó chịu nhất là trải nghiệm sử dụng gặp phiền phức bởi quảng cáo xung quanh trang web, giao diện nhìn rối mắt vì

quá nhiều thông tin được nhồi nhét vào một màn hình. Và với tư cách là sinh viên sắp ra trường, nhận thấy việc tạo CV chuyên nghiệp và tìm được công việc phù hợp với bản thân là một thử thách không nhỏ. Từ những trải nghiệm và quan sát, kết hợp cơ hội thực hiện đồ án, dự án được xây dựng với giao diện thân thiện hơn và tích hợp AI nhằm cải thiện trải nghiệm người dùng.

Trong quá trình thực hiện, dự án được triển khai theo hướng **Agile**, cụ thể là **Scrum/Kanban**. Vì đây là đồ án cá nhân, mô hình này có thể tự chia nhỏ công việc thành từng phần, làm dần dần, dễ kiểm soát tiến độ. Khi gặp vấn đề thì cách tiếp cận có thể linh hoạt điều chỉnh. Ngoài ra, sản phẩm thường xuyên nhờ bạn bè sử dụng thử và thầy hướng dẫn góp ý để chỉnh sửa và cải thiện, đảm bảo hữu ích và phù hợp với nhu cầu thực tế.

Đồ án được xây dựng hệ thống theo kiến trúc **monolithic**, toàn bộ các chức năng chính như quản lý người dùng, đăng tin tuyển dụng, thanh toán qua MoMo, và tích hợp **AI** đều được phát triển và gọi **API** trong một ứng dụng Spring Boot duy nhất, sử dụng chung cơ sở dữ liệu **MySQL**. Cách làm này giúp dễ triển khai, quản lý và phù hợp với quy mô đồ án cá nhân. Tuy nhiên, logic cũng được chia nhỏ thành các module riêng để dễ bảo trì và có thể mở rộng thành kiến trúc microservices trong tương lai sắp tới.

Về công nghệ, **ReactJS** kết hợp với Vite và Tailwind CSS cho phần frontend để tạo giao diện nhanh, nhẹ và dễ tùy chỉnh. Với backend, **Spring Boot** là một framework mạnh mẽ của Java, hỗ trợ xử lý tốt các logic nghiệp vụ trong hệ thống. **MySQL** được dùng để lưu trữ thông tin người dùng, tin tuyển dụng và các dữ liệu liên quan. Ngoài ra, tích hợp **n8n.io** để xây dựng workflow AI tự động, giúp việc kết nối và thử nghiệm các API AI đơn giản hơn. Trong quá trình làm, **Git** là một công cụ hiệu quả để quản lý code, **Figma** thiết kế UI/UX và **Google Sheets** để theo dõi tiến độ, tránh mất liên kết nội dung và làm việc có tổ chức.

1.5. Bố cục đề tài

Báo cáo có 4 chương, với nội dung như sau:

Chương 1. Tổng quan đề tài: Giới thiệu tổng quan về đề tài "Hệ thống web tìm việc làm". Bắt đầu bằng việc phân tích bối cảnh và các vấn đề của những nền tảng hiện có để làm rõ lý do chọn đề tài. Nội dung chương xác định các mục tiêu cụ thể cho

ứng viên, nhà tuyển dụng và quản trị viên, đồng thời khoanh vùng phạm vi chức năng chính của hệ thống. Các phương pháp thực hiện, kiến trúc hệ thống và công nghệ sử dụng cũng được trình bày một cách chi tiết. Và cuối cùng, chương sẽ khẳng định ý nghĩa khoa học và giá trị thực tiễn mà dự án hướng tới, đồng thời phác thảo cấu trúc của toàn bộ báo cáo đề án để người đọc có cái nhìn bao quát.

Chương 2. Cơ sở lý thuyết: Chương này đi sâu vào các công nghệ được chọn làm nền tảng để xây dựng hệ thống. Về phía backend, Spring Boot được lựa chọn để phát triển các API nhờ vào hiệu năng, khả năng bảo mật và mở rộng linh hoạt. Về frontend, ReactJS đảm nhiệm việc xây dựng một giao diện người dùng hiện đại, tốc độ và giàu tính tương tác. Đặc biệt, chương sẽ trình bày giải pháp tích hợp n8n để tự động hóa các luồng xử lý cho tính năng AI, cụ thể là "Gợi ý cải thiện CV" và "Luyện tập phỏng vấn", nhằm mang lại giá trị gia tăng độc đáo cho ứng viên.

Chương 3. Phân tích thiết kế hệ thống: Nội dung trọng tâm trong phần này là mô tả chi tiết quá trình từ việc phân tích yêu cầu đến việc thiết kế hệ thống hoàn chỉnh. Các yêu cầu chức năng của hệ thống sẽ được mô tả thông qua các biểu đồ Use Case, trong khi các yêu cầu phi chức năng được trình bày trong phần đặc tả chi tiết. Và chương sẽ định nghĩa rõ vai trò và quyền hạn của ba tác nhân chính: Quản trị viên, Nhà tuyển dụng và Ứng viên. Từ đó, các phần tiếp theo trình bày chi tiết thiết kế hệ thống với kiến trúc, sơ đồ tuần tự và cơ sở dữ liệu, giúp làm rõ cách thức hoạt động của FlexiConnect.

Chương 4. Kết quả đạt được: Đây là chương mang đến cái nhìn khái quát về hệ thống. Các kết quả hiện thực hóa từ giai đoạn thiết kế sẽ được minh họa bằng hình ảnh trực quan. Những chức năng cốt lõi, một số chức năng bổ sung, cùng với các tính năng AI sẽ lần lượt được mô tả, giúp người đọc hình dung rõ ràng cách hệ thống vận hành.

Chương 5. Kết luận và hướng phát triển: Chương này khép lại báo cáo bằng việc tổng kết các kết quả nghiên cứu và phát triển đã đạt được. Đồng thời, một đánh giá toàn diện về mức độ thành công của hệ thống sẽ được trình bày, bao gồm cả những thành tựu nổi bật, lợi ích đến từ hệ thống và các hạn chế cần tiếp tục cải thiện. Trên cơ sở đó, chương đề xuất những định hướng phát triển tiềm năng trong tương lai, như mở rộng quy mô, bổ sung thêm chức năng và tối ưu hóa các thuật toán AI, cuối cùng là tích hợp thêm công nghệ mới nhằm gia tăng giá trị cho người dùng.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Backend Framework - Spring Boot

2.1.1. Giới thiệu Spring Boot

Spring Boot được phát hành vào tháng 4 năm 2014 để giảm bớt một số gánh nặng của việc phát triển một ứng dụng web Java. Nó cho phép các nhà phát triển tập trung nhiều hơn vào logic nghiệp vụ thay vì mã kỹ thuật mẫu và các cấu hình liên quan. Spring Boot hướng đến việc tạo ra các ứng dụng dựa trên Spring, sẵn sàng cho môi trường production, độc lập với ít thay đổi về cấu hình từ phía nhà phát triển ứng dụng. Nó có một quan điểm định sẵn (opinionated view) về Spring Framework, vì vậy các nhà phát triển ứng dụng có thể nhanh chóng bắt đầu với những gì họ cần. Nó cung cấp một lớp bổ sung giữa Spring Framework cho người dùng để đơn giản hóa các khía cạnh cấu hình nhất định. [1]



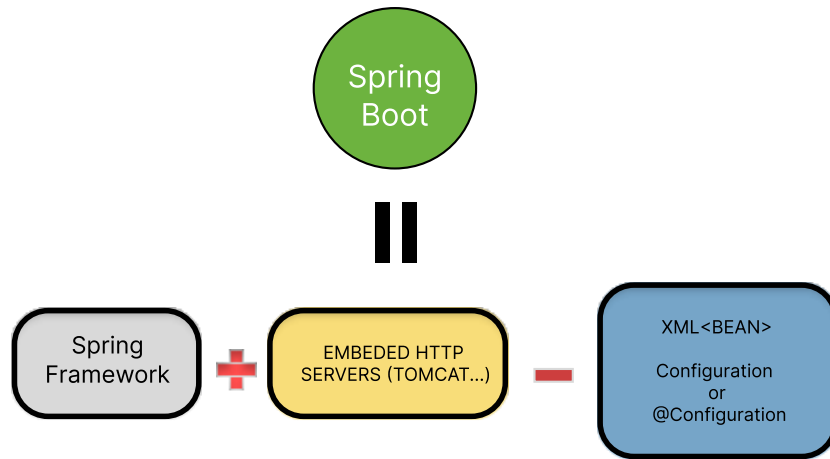
Hình 2.1: Spring Boot

2.1.2. Tại sao lại chọn Spring Boot

2.1.2.1. Spring Boot đơn giản hóa

Spring Boot được lựa chọn vì nó khắc phục được nhược điểm cấu hình phức tạp của Spring truyền thống. Với Spring, lập trình viên phải tự cấu hình nhiều thành phần như DispatcherServlet, ViewResolver, DataSource hay sử dụng nhiều file XML và annotation, khiến cho việc phát triển trở nên chậm và tốn công sức. Spring Boot mang đến giải pháp Auto Configuration, tự động nhận diện các thư viện có trong classpath và thiết lập cấu hình phù hợp. Bên cạnh đó, hệ thống Starter Projects giúp dễ dàng quản lý dependency, trong khi máy chủ nhúng (Tomcat, Jetty) cho phép triển khai ứng

dụng nhanh chóng mà không cần server bên ngoài. Những ưu điểm này giúp rút ngắn thời gian phát triển, đặc biệt phù hợp với xu hướng xây dựng microservices – nơi cần tạo ra nhiều dịch vụ nhỏ, độc lập, triển khai nhanh và dễ mở rộng.



Hình 2.2: So sánh Spring Boot và Spring truyền thống

2.1.2.2. Những tính năng nổi bật của Spring Boot

Spring Boot là một phần mở rộng của Spring Framework, được phát triển với mục tiêu chính là đơn giản hóa quá trình khởi tạo và phát triển ứng dụng. Các tính năng cốt lõi của nó giúp các nhà phát triển tập trung vào logic nghiệp vụ thay vì các công việc cấu hình lặp đi lặp lại.

Khả năng phát triển ứng dụng web độc lập

Điểm nổi bật đầu tiên và dễ thấy nhất của Spring Boot là khả năng tạo ra các ứng dụng web có thể chạy độc lập.

Trước đây với Spring MVC, bạn phải viết code, đóng gói thành file .war rồi triển khai (deploy) lên một máy chủ web cài sẵn như Tomcat. Quá trình này khá nhiều bước và rườm rà. Spring Boot đã thay đổi điều đó bằng cách tích hợp sẵn một máy chủ (như Tomcat hoặc Jetty) ngay bên trong ứng dụng của bạn.

Khi bạn nhấn nút "Run", ứng dụng Java của bạn khởi động và máy chủ web cũng tự khởi động theo. Kết quả là bạn chỉ cần một file .jar duy nhất là có thể chạy ở bất cứ đâu. Cách làm này cực kỳ hữu ích trong thời đại Microservices và Docker, nơi mỗi dịch vụ cần được đóng gói gọn nhẹ và độc lập.

Đơn giản hóa Quá trình khởi tạo và Cấu hình

Spring Boot cung cấp cơ chế Autoconfiguration (tự động cấu hình), giúp loại bỏ hầu hết thao tác cấu hình thủ công. Chỉ cần khai báo các thư viện cần thiết, framework sẽ tự động thiết lập các bean liên quan. Ví dụ: khi thêm **spring-boot-starter-data-jpa** và driver database, hệ thống sẽ tự động cấu hình **DataSource**

Điểm khởi đầu cho tất cả sự kiện này chỉ gói gọn trong một lớp với hàm main() quen thuộc và một dòng lệnh duy nhất:

Java
<pre>@SpringBootApplication public class FlexiConnectApplication { public static void main(String[] args) { SpringApplication.run(FlexiConnectBE.class, args); } }</pre>

Bảng 2.1: Lớp chạy ứng dụng

Nhờ vậy, việc khởi tạo dự án trở nên nhanh chóng, giúp lập trình viên tập trung vào xây dựng tính năng thay vì cấu hình lặp lại.

Quản lý Sự kiện và Listener

Spring Boot mở rộng hệ thống **ApplicationEvent** của Spring Framework, giúp các thành phần trong ứng dụng giao tiếp với nhau mà không cần phụ thuộc trực tiếp. Khi một sự việc xảy ra (ví dụ user đăng ký thành công), hệ thống sẽ phát ra một event, và các listener đã đăng ký sẽ tự động xử lý các tác vụ liên quan.

Điều này giúp tách biệt logic nghiệp vụ rất hiệu quả. Thay vì module đăng ký phải gọi trực tiếp đến module gửi email, bạn chỉ cần phát ra **UserRegisteredEvent** và để các listener riêng biệt lo việc gửi email, ghi log, hay các xử lý khác.

Cấu hình linh hoạt theo môi trường

Spring Boot cho phép tách riêng cấu hình khỏi mã nguồn, giúp cùng một ứng dụng có thể chạy trên nhiều môi trường khác nhau mà không cần build lại.

Hệ thống sẽ đọc cấu hình từ nhiều nguồn theo thứ tự ưu tiên: command-line arguments > environment variables > file properties/YAML. Cấu hình từ nguồn có độ ưu tiên cao sẽ override những cái ở mức thấp hơn.

Tích hợp Spring Security

Khi thêm **spring-boot-starter-security** vào project, Spring Boot tự động bảo vệ tất cả endpoint, yêu cầu xác thực mặc định. Mặc định Spring Boot sẽ khóa các endpoint nên lập trình viên phải tự cấu hình để mở những gì cần công khai.

Nhờ vậy mà hạn chế được rủi ro bảo mật khi lập trình viên quên bảo mật endpoint.

Spring Boot cũng đơn giản hóa việc setup Basic Auth, Form Login, OAuth2 và các cơ chế bảo mật khác.

Quản lý thư viện thông minh (Starter Dependencies)

Spring Boot hỗ trợ các gói **starter**, tức là tập hợp sẵn các dependency cần cho một tính năng, giúp bạn không phải tự khai báo từng thư viện thủ công.

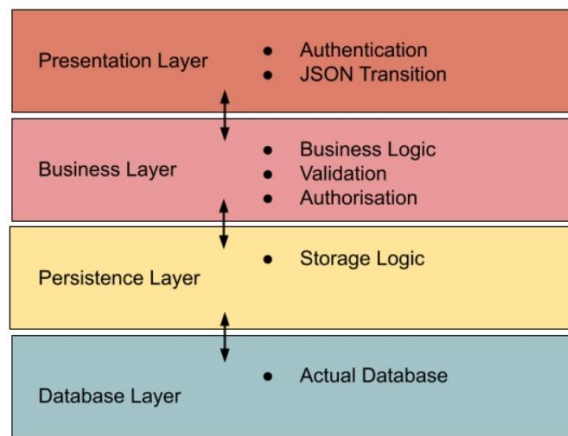
Chỉ cần thêm **spring-boot-starter-web** là bạn đã có đủ Spring MVC, Jackson, Tomcat và các thư viện liên quan với phiên bản đã được test tương thích. Không còn phải lo xung đột giữa các phiên bản hay thiếu thư viện nữa.

2.1.3. Kiến trúc Spring Boot

2.1.3.1. Tổng quan kiến trúc

Về cơ bản, Spring Boot không áp đặt một kiến trúc cứng nhắc nào lên người phát triển. Tuy nhiên, nó khuyến khích và hỗ trợ mạnh mẽ cho việc xây dựng ứng dụng theo kiến trúc phân tầng (**Layered Architecture**). Đây là một mô hình kiến trúc phổ biến và đã được chứng minh hiệu quả trong việc xây dựng các hệ thống lớn, giúp mã nguồn trở nên rõ ràng, dễ bảo trì và dễ phát triển.

Với đồ án này, hệ thống được xây dựng tuân thủ theo kiến trúc phân tầng mặc định của Spring Boot. Kiến trúc này chia ứng dụng thành các tầng (layer) riêng biệt, mỗi tầng có một trách nhiệm và vai trò cụ thể. Các tầng giao tiếp với nhau theo một luồng xác định, giúp tách biệt các mối quan tâm (**separation of concerns**) và giảm sự phụ thuộc lẫn nhau giữa các thành phần. [2]



Hình 2.3: Kiến trúc Spring Boot

2.1.3.2. Chi tiết các tầng kiến trúc

1. Tầng trình bày (Presentation Layer)

Đây là tầng trên cùng của kiến trúc, đóng vai trò là giao diện tương tác của hệ thống với thế giới bên ngoài. Trách nhiệm chính của tầng này là tiếp nhận các yêu cầu HTTP từ phía client, xử lý chúng và trả về các phản hồi HTTP tương ứng (thường ở định dạng JSON).

Trong Spring Boot, tầng này thường được triển khai bằng các lớp Controller, được đánh dấu bằng các annotation như **@RestController**. Các phương thức trong Controller sẽ được ánh xạ tới các URL (endpoint) cụ thể. Tầng này chủ yếu xác thực đầu vào và chuyển tiếp yêu cầu đến tầng nghiệp vụ và điều phối yêu cầu đến tầng nghiệp vụ.

Ví dụ: Một UserController tiếp nhận yêu cầu lấy thông tin người dùng theo ID

```

Java

@RestController
@RequestMapping("/api/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        User user = userService.findUserById(id);
        return ResponseEntity.ok(user);
    }
}
  
```

```

    }
}

```

Bảng 2.2: REST Controller xử lý yêu cầu

2. Tầng nghiệp vụ (Business Layer)

Tầng nghiệp vụ được coi là trung tâm xử lý logic của ứng dụng, chứa đựng toàn bộ các logic, quy tắc và quy trình xử lý chính của hệ thống. Tầng này nhận yêu cầu từ tầng trình bày, thực hiện các tính toán, và phối hợp với tầng truy cập dữ liệu để lấy hoặc lưu thông tin.

Các lớp trong tầng này thường được gọi là các lớp Service và được đánh dấu bằng annotation **@Service**. Việc tách biệt logic nghiệp vụ ra một tầng riêng giúp mã nguồn dễ đọc, dễ kiểm thử và có thể tái sử dụng.

Ví dụ: Một UserService chứa logic tìm kiếm người dùng.

```

Java

@Service
public class UserServiceImpl implements UserService {

    @Autowired
    private UserRepository userRepository;

    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));
    }
}

```

Bảng 2.3: Service xử lý logic nghiệp vụ

3. Tầng Truy cập Dữ liệu (Persistence Layer)

Tầng này có nhiệm vụ giao tiếp trực tiếp với cơ sở dữ liệu, chịu trách nhiệm **CRUD** với cơ sở dữ liệu. Trong hệ sinh thái Spring, tầng này thường được triển khai bằng **Spring Data JPA** với các lớp Repository và được đánh dấu bằng **@Repository**. Tầng này giúp trừu tượng hóa các câu lệnh SQL, cho phép lập trình viên tương tác với cơ sở dữ liệu thông qua các phương thức Java đơn giản.

Ví dụ: UserRepository kế thừa từ JpaRepository để có sẵn các phương thức thao tác với bảng User.

Java

@Repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // Spring Data JPA sẽ tự động cung cấp các phương thức như findById(), save(),...  
    // Hoặc có thể định nghĩa thêm các phương thức truy vấn tùy chỉnh.  
}
```

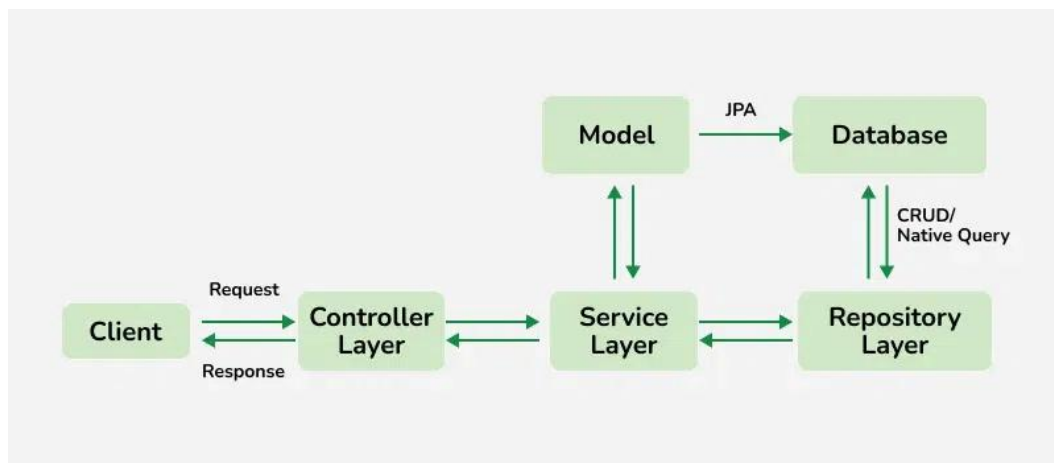
Bảng 2.4: Repository truy xuất dữ liệu

4. Tầng Cơ sở dữ liệu (Database Layer)

Đây là tầng cuối cùng trong kiến trúc, nơi dữ liệu của ứng dụng được lưu trữ và quản lý một cách vật lý. Tầng này có thể là bất kỳ hệ quản trị cơ sở dữ liệu nào như MySQL, PostgreSQL, MongoDB... Ứng dụng sẽ tương tác với tầng này thông qua tầng truy cập dữ liệu.

2.1.3.3. Luồng xử lý của Spring Boot

(1) Khi một client gửi yêu cầu HTTP (ví dụ: GET /api/users/1), Tomcat sẽ tiếp nhận và chuyển yêu cầu đến **DispatcherServlet** của Spring. (2) DispatcherServlet sẽ định tuyến yêu cầu đến phương thức tương ứng trong **UserController**, nơi kiểm tra và điều phối dữ liệu. (3) Controller sau đó gọi các phương thức trong **UserService**, nơi thực thi logic nghiệp vụ và phối hợp với **UserRepository** để truy xuất hoặc cập nhật cơ sở dữ liệu thông qua JPA/Hibernate. (4) Khi **Repository** nhận dữ liệu từ Database, nó ánh xạ thành đối tượng Java và trả lại cho Service. (5) Service chuyển kết quả về cho Controller, và cuối cùng Controller đóng gói thành HTTP response (JSON) gửi trả lại client.



Hình 2.4: Luồng xử lý của Spring Boot

2.1.4. Spring Security

2.1.4.1. Giới thiệu tổng quan và lý do chọn Spring Security

Trong bất kỳ ứng dụng web nào, việc đảm bảo an toàn, bảo mật thông tin và phân quyền truy cập cho người dùng là một yêu cầu tiên quyết. Để giải quyết bài toán này cho hệ thống, bắt đầu tiến hành nghiên cứu và quyết định lựa chọn Spring Security - một framework con của Spring Framework, chuyên cung cấp giải pháp bảo mật toàn diện và linh hoạt.

Lý do chính để lựa chọn Spring Security là khả năng tích hợp liền mạch với hệ sinh thái Spring mà đồ án này đang sử dụng, giúp giảm thiểu thời gian và độ phức tạp trong việc cấu hình. Bên cạnh đó, Spring Security cung cấp sẵn các cơ chế mạnh mẽ để chống lại những hình thức tấn công phổ biến như Giả mạo yêu cầu chéo trang (CSRF), Tấn công chiếm quyền phiên (Session Fixation),... đáp ứng các tiêu chuẩn bảo mật phổ biến. Framework này tập trung giải quyết hai bài toán cốt lõi: Xác thực (Authentication) - xác định danh tính người dùng, và Ủy quyền (Authorization) - quyết định các quyền hạn của người dùng đó trong hệ thống.

2.1.4.2. Giới thiệu Kiến trúc Xác thực trong Spring Security

Để xây dựng một cơ chế bảo mật vững chắc, đồ án này đã áp dụng kiến trúc xác thực của Spring Security. “Tuy nhiên, để phục vụ các API theo kiến trúc stateless, cơ chế xác thực dựa trên session mặc định được thay thế bằng luồng xác thực tùy chỉnh sử dụng JWT. Cốt lõi của kiến trúc này vẫn dựa trên các thành phần gốc của Spring Security, nhưng được bổ sung một filter tùy chỉnh để xử lý JWT.

SecurityContextHolder là nơi lưu trữ thông tin xác thực (Authentication) của người dùng trong phạm vi một request

Authentication là interface đại diện thông tin đăng nhập ban đầu và, sau xác thực, chứa đầy đủ thông tin người dùng cùng quyền hạn.

GrantedAuthority đại diện cho một quyền hạn duy nhất được cấp cho người dùng (ví dụ như ROLE_ADMIN). Các quyền này được mã hóa vào bên trong JWT.

AuthenticationManager là interface chính xác thực hiện xác thực, ủy quyền cho **AuthenticationProvider** xử lý logic và trong đồ án này chỉ được sử dụng tại endpoint đăng nhập ban đầu..

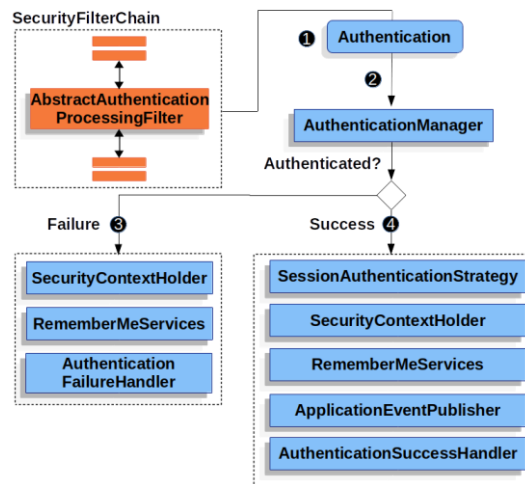
AuthenticationSuccessHandler được gọi khi xác thực thành công. Nhiệm vụ của nó trong đồ án là tạo và trả về JWT cho client.

AuthenticationFailureHandler được gọi khi xác thực thất bại. Nhiệm vụ của nó là trả về lỗi 401 Unauthorized.

SessionAuthenticationStrategy có chức năng quản lý và bảo vệ session, nhưng thành phần này không được sử dụng vì ứng dụng được cấu hình theo kiến trúc stateless.

RememberMeServices cung cấp chức năng "ghi nhớ đăng nhập" qua cookie. Chức năng này cũng không được sử dụng vì đã được thay thế bằng JWT.

AbstractAuthenticationProcessingFilter là cơ sở để xử lý các bộ lọc xác thực trong Spring Security.



Hình 2.4: Sơ đồ chứng thực tài khoản

2.1.4.3. Luồng xử lý Xác thực với JWT

Sau khi đã định nghĩa các thành phần kiến trúc, phần này sẽ trình bày chi tiết về luồng xử lý xác thực phi trạng thái (stateless) bằng JWT được áp dụng trong đồ án. Luồng này bao gồm hai giai đoạn chính: Cấp phát token khi người dùng đăng nhập và Xác thực token trên mỗi yêu cầu API.

Cấp phát Token (Authentication)

(1) Người dùng gửi yêu cầu đăng nhập chứa username và password đến endpoint `/api/auth/login`.

(2) Controller tạo ra một đối tượng `UsernamePasswordAuthenticationToken` (chưa xác thực) với thông tin đăng nhập.

(3) Đối tượng này được chuyển đến AuthenticationManager để xác thực thông tin với cơ sở dữ liệu thông qua DaoAuthenticationProvider.

(4) Nếu xác thực thành công, AuthenticationSuccessHandler sẽ được gọi. Tại đây, đã tùy chỉnh để tạo ra một chuỗi JWT chứa thông tin người dùng và các GrantedAuthority. (5) JWT này được trả về trong body của response, client sẽ lưu trữ token này để sử dụng cho các yêu cầu sau. (6) Nếu xác thực thất bại exception được throw và @ExceptionHandler xử lý trả về lỗi 401 Unauthorized.

Xác thực Token trên mỗi Yêu cầu (Authorization)

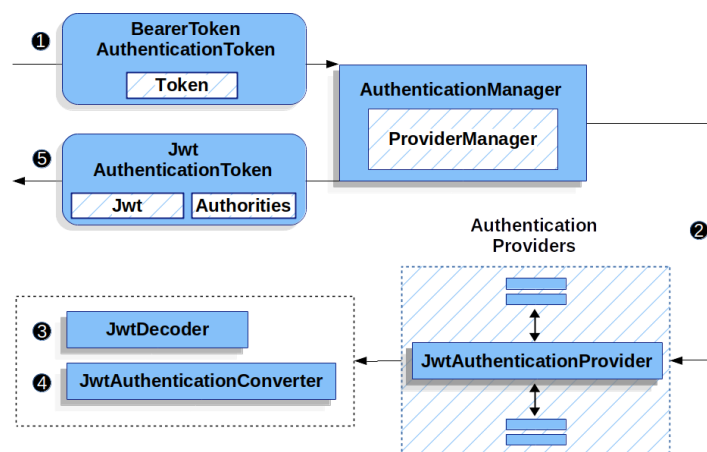
(1) Với mỗi yêu cầu đến tài nguyên được bảo vệ, client đính kèm JWT vào header Authorization theo dạng Bearer <token>. Filter trích xuất token và tạo ra một đối tượng BearerTokenAuthenticationToken.

(2) Token này được chuyển đến AuthenticationManager, sau đó được ủy quyền cho JwtAuthenticationProvider xử lý.

(3) Bên trong JwtAuthenticationProvider, JwtDecoder sẽ giải mã và xác thực chữ ký của token.

(4) JwtAuthenticationConverter trích xuất thông tin người dùng và các quyền (Authorities) từ JWT đã được decode.

(5) Nếu mọi thứ hợp lệ, một đối tượng JwtAuthenticationToken (đã được xác thực đầy đủ) chứa JWT và Authorities được tạo ra và trả về cho AuthenticationManager. (6) Cuối cùng, đối tượng Authentication này được đặt vào SecurityContextHolder, hoàn tất quá trình xác thực cho request.



Hình 2.5: Luồng xác thực JWT trong Spring Security

2.1.4.4. Tích hợp và Cấu hình Spring Security

Để bảo vệ hệ thống, tích hợp Spring Security và cấu hình luồng xác thực tùy chỉnh sử dụng JSON Web Tokens (JWT), phù hợp với kiến trúc frontend ReactJS..

1. Bổ sung Dependency

Đầu tiên, thêm các dependency cần thiết vào file pom.xml của dự án, bao gồm **spring-boot-starter-security** để có các tính năng bảo mật cốt lõi và **nimbus-jose-jwt** để hỗ trợ việc tạo và xác thực token.

2. Đóng gói Logic JWT

Lớp này đóng gói toàn bộ logic xử lý, chịu trách nhiệm cho ba nhiệm vụ chính: tạo token mới khi người dùng đăng nhập, giải mã token để lấy thông tin, và xác thực tính hợp lệ của token.

3. Chặn và Xử lý Request với JwtFilter

Để kiểm tra token trên mỗi yêu cầu API, triển khai bộ lọc tùy chỉnh **JwtFilter**. Bộ lọc này sẽ tự động chặn các request đến, trích xuất JWT từ header Authorization, sau đó sử dụng JwtUtils để xác thực. Nếu token hợp lệ, thông tin người dùng sẽ được thiết lập vào SecurityContextHolder để duy trì trạng thái xác thực trong suốt quá trình xử lý request.

4. Kết nối trong SecurityConfig

Cuối cùng, tất cả các thành phần trên được kết nối và kích hoạt trong lớp cấu hình trung tâm SecurityConfig, nơi định nghĩa SecurityFilterChain để áp dụng các quy tắc bảo mật.

Java
<pre>@Bean public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception { http .csrf(csrf -> csrf.disable()) .sessionManagement(sess -></pre>

```

sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

    .authorizeHttpRequests(auth -> auth

        .requestMatchers("/api/auth/**").permitAll()

        .anyRequest().authenticated()

    )

    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

Bảng 2.5: Đoạn mã cấu hình SecurityFilterChain chính

Việc tích hợp Spring Security đã khép lại phần trình bày kỹ thuật trong chương này. Nội dung đã giải đáp các câu hỏi trọng tâm: Spring Boot là gì, mang lại tiện ích gì cho dự án, và cơ chế hoạt động ra sao. Từ cơ sở lý thuyết đến triển khai thực tế, chương này đã xây dựng một lộ trình kiến thức rõ ràng, tạo nền tảng cho việc áp dụng Spring Boot trong các dự án tiếp theo.

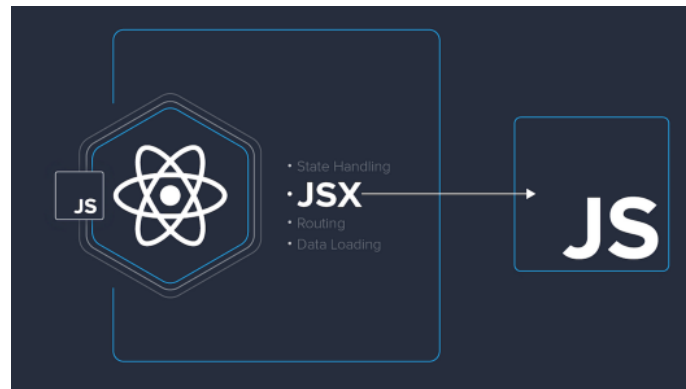
2.2. Frontend ReactJS

2.2.1. Giới thiệu về ReactJS

ReactJS là thư viện **JavaScript** mã nguồn mở được Meta phát triển, chuyên dụng cho việc xây dựng giao diện người dùng. React áp dụng kiến trúc **component-based**, cho phép chia giao diện thành các thành phần độc lập có thể tái sử dụng. Công nghệ này sử dụng **Virtual DOM** để tối ưu hiệu suất render và áp dụng mô hình one-way data binding nhằm đảm bảo luồng dữ liệu đơn hướng, giúp ứng dụng dễ bảo trì và mở rộng.

Trong React, **JSX** là một cú pháp mở rộng, cho phép viết mã có cấu trúc tựa như HTML trực tiếp trong logic JavaScript. Cách viết này giúp việc xây dựng các component giao diện trở nên trực quan và dễ đọc hơn. Để xử lý dữ liệu và các giai đoạn trong vòng đời của component, React giới thiệu một tính năng gọi là **Hooks**. Các hook phổ biến như **useState**, **useEffect**, và **useContext** cung cấp khả năng quản lý trạng thái (state) và các hiệu ứng phụ ngay trong các component dạng hàm, mà không cần đến cấu trúc của component dạng lớp như trước đây. [3]

Việc quản lý trạng thái trong ứng dụng được tiếp cận một cách linh hoạt: **useState** được dùng để quản lý trạng thái cục bộ cho các thành phần đơn lẻ. Khi cần chia sẻ dữ liệu trên quy mô lớn hơn, hệ thống có thể mở rộng bằng **Context API** có sẵn của React hoặc tích hợp các thư viện bên thứ ba như Redux để quản lý trạng thái toàn cục một cách hiệu quả.



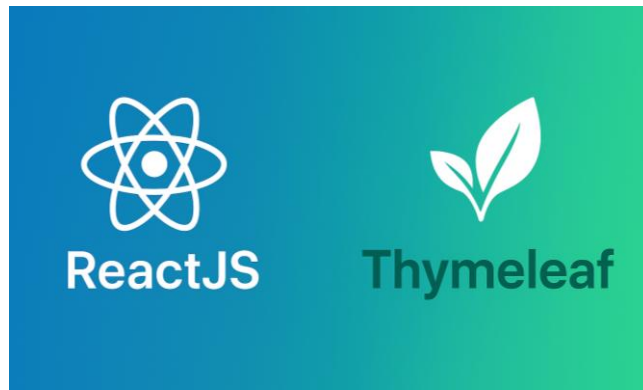
Hình 2.6: Biên dịch JSX trong React.

2.2.2. Lý do lựa chọn ReactJS thay thế Thymeleaf

Với đề tài web tìm việc làm sẽ sử dụng ReactJS làm công nghệ frontend chính thay vì Thymeleaf template engine. ReactJS phù hợp với yêu cầu xây dựng giao diện động với khả năng tương tác cao và dễ dàng mở rộng tính năng. React sở hữu cộng đồng developer mạnh mẽ cùng hệ sinh thái thư viện phong phú, tạo điều kiện thuận lợi cho việc tích hợp các công nghệ bên thứ ba. Điều này đặc biệt quan trọng trong việc phát triển một hệ thống quản lý linh hoạt như FlexiConnect.

Đặc biệt, đối với giao diện quản trị (Admin Dashboard) của hệ thống, việc lựa chọn ReactJS tỏ ra vượt trội hơn hẳn. Giao diện admin có đặc thù là xử lý lượng lớn dữ liệu động, bao gồm các bảng biểu, thống kê, và form nhập liệu phức tạp. Với Thymeleaf, các thao tác tương tác cao như lọc, tìm kiếm, hay sắp xếp dữ liệu theo thời gian thực thường đòi hỏi phải tải lại toàn bộ trang, làm giảm trải nghiệm của người quản trị.

Ngược lại, ReactJS với kiến trúc component và Virtual DOM cho phép xây dựng các thành phần giao diện độc lập có thể tự cập nhật dữ liệu một cách tức thì. Điều này tạo ra một trải nghiệm quản trị mượt mà, nhanh chóng và hiệu quả hơn, vốn là yêu cầu cốt lõi đối với người dùng quản trị hệ thống.



Hình 2.7: ReactJS so sánh với Thymeleaf

2.2.3. Các công nghệ được sử dụng trong ReactJS

Với việc xây dựng giao diện người dùng, dự án đã sử dụng bộ công nghệ ReactJS hiện đại và hiệu quả, kết hợp các thư viện chuyên dụng để tối ưu hóa từ quá trình phát triển đến trải nghiệm người dùng cuối.

Nền tảng của dự án được khởi tạo và quản lý bằng **Vite**, một công cụ xây dựng thể hệ mới giúp tăng tốc đáng kể chu kỳ phát triển nhờ tốc độ đóng gói mã nguồn và khả năng tự động cập nhật giao diện khi lưu. Về phần giao diện, phương pháp utility-first với **TailwindCSS** để tạo ra một hệ thống định dạng linh hoạt, cho phép xây dựng các thiết kế tùy chỉnh một cách nhanh chóng và nhất quán. Thay vì dùng các thư viện thành phần giao diện có sẵn, các component được xây dựng dựa trên nền tảng Radix UI, cung cấp các component "headless" (không định dạng sẵn) với khả năng tiếp cận tốt. Giao diện được hoàn thiện về mặt thẩm mỹ nhờ các bộ thư viện biểu tượng đa dạng từ **Lucide React** và **React Icons**.

Để quản lý việc điều hướng giữa các trang và bảo vệ các đường dẫn được bảo mật, **React Router DOM** đã được tích hợp. Việc giao tiếp với backend Spring Boot được đảm nhiệm bởi **Axios**, một HTTP client mạnh mẽ hỗ trợ các tính năng nâng cao như interceptors, tự động đính kèm JWT token vào mỗi yêu cầu. Token này được quản lý an toàn ở phía client nhờ thư viện React Cookies.

Cuối cùng, để nâng cao trải nghiệm người dùng, các hiệu ứng chuyển động mượt mà đã được thêm vào bằng **Framer Motion**, và hệ thống thông báo được triển khai với **Sonner** để cung cấp phản hồi tức thì. Đối với các trang thống kê, biểu đồ được vẽ bằng sự kết hợp của **Chart.js** và react-chartjs-2. Các tác vụ tiện ích khác như xử lý ngày giờ cũng được tối ưu hóa bằng thư viện gọn nhẹ **Day.js**.



Hình 2.8: Các công nghệ được sử dụng trong ReactJS

2.2.4. Kiến trúc kết nối ReactJS với Backend

Giao diện ReactJS được thiết kế theo mô hình Client-Server, hoạt động độc lập và giao tiếp với backend Spring Boot thông qua một hệ thống RESTful API đã được định nghĩa rõ ràng.

Luồng hoạt động chính như sau: Axios được sử dụng làm HTTP client để thực hiện các request (GET, POST...). Hệ thống xác thực được triển khai bằng JWT token, với react-cookies đảm nhận việc lưu trữ và gửi token kèm mỗi request lên server. Ở phía client, React Router DOM sẽ dựa vào trạng thái đăng nhập (sự tồn tại của token) để bảo vệ các route, đảm bảo chỉ người dùng đã xác thực mới có thể truy cập các trang quản trị của ứng dụng.

2.3. n8n

2.3.1. Giới thiệu về n8n

n8n là một nền tảng tự động hóa luồng công việc (workflow automation) mã nguồn mở. Về cơ bản, n8n được thiết kế để kết nối các ứng dụng, API và dịch vụ riêng lẻ, cho phép chúng tương tác và trao đổi dữ liệu với nhau một cách tự động theo các quy trình được định sẵn.

Nền tảng này hoạt động dựa trên một giao diện trực quan, nơi người dùng có thể xây dựng các luồng xử lý (workflow) bằng cách kết nối các "nút" (nodes). Mỗi nút đại diện cho một hành động hoặc một dịch vụ cụ thể (ví dụ: đọc email, ghi dữ liệu vào Google Sheets, gọi một API, xử lý văn bản...). Bằng cách liên kết các nút này lại với nhau, người dùng có thể tạo ra các kịch bản tự động hóa từ đơn giản đến rất phức tạp mà không cần can thiệp thủ công. [4]



Hình 2.9: n8n

2.3.2. Tại sao lại chọn n8n

Việc tích hợp n8n vào kiến trúc của đồ án được dựa trên những lý do chiến lược sau:

Tính linh hoạt cao giữa No-code và Low-code: n8n cung cấp một môi trường phát triển trực quan (kéo-thả), giúp đẩy nhanh tốc độ xây dựng các luồng logic cơ bản.

Song song đó, nền tảng cho phép can thiệp sâu bằng cách viết mã tùy chỉnh (custom code) thông qua các nút JavaScript hoặc Python. Sự kết hợp này vừa giúp tiết kiệm thời gian, vừa đảm bảo khả năng xử lý các yêu cầu nghiệp vụ đặc thù của đồ án.

Khả năng tự chủ và bảo mật với Mã nguồn mở: Vì n8n là một dự án mã nguồn mở, nó cho phép triển khai trên hạ tầng máy chủ riêng (self-hosting). Điều này mang lại hai lợi ích cốt lõi cho đồ án:

Toàn quyền kiểm soát dữ liệu: Mọi thông tin xử lý qua luồng công việc đều nằm trong tầm kiểm soát, không phụ thuộc vào các dịch vụ đám mây của bên thứ ba, đảm bảo tính bảo mật và riêng tư.

Khả năng mở rộng không giới hạn: Không bị ràng buộc bởi các giới hạn của những gói dịch vụ thương mại, cho phép tùy chỉnh và mở rộng hệ thống một cách tự do.

Đơn giản hóa kiến trúc Backend: Trong khuôn khổ đồ án này, n8n đóng vai trò như một lớp điều phối trung gian (Orchestration Layer), hoạt động tương tự mô hình Backend-for-Frontend (BFF). Thay vì phải xây dựng một hệ thống backend phức tạp để giao tiếp giữa giao diện người dùng (UI) và các dịch vụ AI, n8n đã đảm nhận trách nhiệm này. Nó tiếp nhận yêu cầu từ UI, điều phối chúng đến các API xử lý chuyên dụng, sau đó tổng hợp và trả kết quả về cho người dùng. Cách tiếp cận này giúp giảm

thiếu đáng kể thời gian và công sức phát triển, đồng thời giúp kiến trúc tổng thể của hệ thống trở nên tinh gọn và dễ dàng bảo trì hơn.

2.3.3. Kiến trúc cốt lõi: Node, Workflow, Mô hình Hướng sự kiện

Nền tảng n8n được xây dựng dựa trên một số khái niệm kiến trúc cơ bản, tạo thành một hệ thống logic và mạnh mẽ để điều phối dữ liệu và tác vụ.

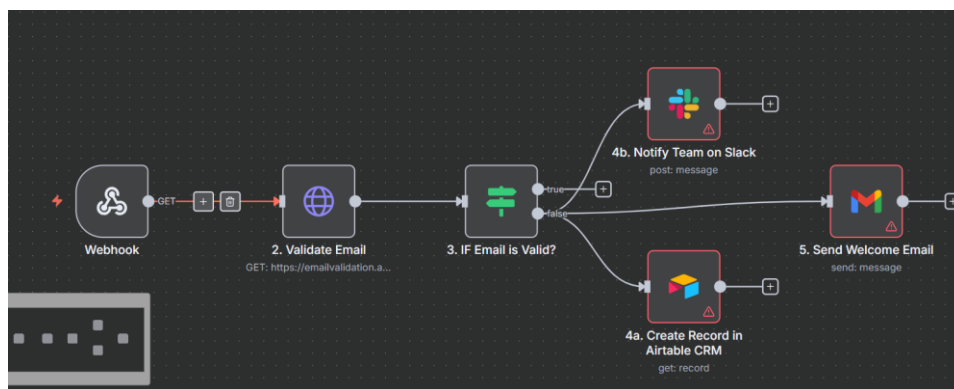
Workflow (Luồng công việc) là đơn vị trung tâm của n8n, đại diện cho một chuỗi các tác vụ được tự động hóa. Nó mô tả một quy trình nghiệp vụ hoàn chỉnh từ điểm bắt đầu đến khi kết thúc. Mỗi workflow được thiết kế để giải quyết một bài toán tự động hóa cụ thể.

Nodes (Nút) là các khối xây dựng cơ bản trong một workflow, mỗi node thực hiện một hành động cụ thể như đọc dữ liệu, gửi email, hoặc tương tác với một API. Các node được kết nối với nhau để tạo thành một chuỗi xử lý logic. Chúng chính là những thành phần cấu tạo nên một luồng công việc.

Triggers (Kích hoạt) là một loại node đặc biệt, đóng vai trò là điểm khởi đầu của một workflow. Chúng "lắng nghe" các sự kiện cụ thể, chẳng hạn như một lịch trình định sẵn (cron job), một yêu cầu HTTP đến (webhook), hoặc một sự kiện từ ứng dụng khác. Khi sự kiện xảy ra, trigger sẽ tự động khởi chạy workflow, tạo nên kiến trúc hướng sự kiện và cho phép tự động hóa theo thời gian thực.

Connections (Kết nối) là các đường nối giữa các node, xác định thứ tự thực thi của các tác vụ. Quan trọng hơn, chúng quy định cách dữ liệu được truyền từ node này sang node tiếp theo, tạo thành một luồng dữ liệu rõ ràng và có thể kiểm soát được.

Ví dụ, một workflow có thể tự động lấy thông tin khách hàng mới từ Google Sheets, sau đó gửi email chào mừng đến họ.



Hình 2.10: Workflow gửi mail chào mừng của n8n

2.3.4. Triển khai Kiến trúc và Tùy chỉnh

Tự lưu trữ qua Docker

Việc tự lưu trữ n8n mang lại nhiều lợi ích chiến lược, không chỉ giúp tiết kiệm chi phí mà còn đảm bảo chủ quyền dữ liệu, nâng cao mức độ bảo mật và cho phép tùy chỉnh linh hoạt. Khi triển khai trên hạ tầng riêng, toàn bộ dữ liệu nhạy cảm như CV hoặc câu trả lời phỏng vấn luôn được giữ trong tầm kiểm soát, loại bỏ rủi ro khi dữ liệu đi qua bên thứ ba. Đồng thời, môi trường này cho phép cài thêm thư viện, chỉnh sửa cấu hình lỗi và mở rộng chức năng, điều vốn không thể thực hiện trên dịch vụ SaaS.

Bảo mật và Mạng

Một phiên bản n8n triển khai trực tiếp ra internet không an toàn và tiềm ẩn nhiều rủi ro. Thực tiễn khuyến nghị là đặt n8n phía sau reverse proxy như NGINX, vừa che giấu cổng nội bộ vừa xử lý SSL/TLS. Giải pháp kết hợp Certbot và Let's Encrypt giúp cấp và gia hạn chứng chỉ miễn phí, bảo đảm toàn bộ lưu lượng được mã hóa HTTPS, đồng thời đáp ứng yêu cầu của nhiều cơ chế xác thực như OAuth2 và cookie bảo mật trên trình duyệt.

Tùy chỉnh và Mở rộng

Bên cạnh vận hành ổn định, dự án có thể mở rộng n8n theo nhu cầu riêng. Một cách phổ biến là xây dựng Dockerfile tùy chỉnh kế thừa từ image gốc, trong đó cài thêm các gói npm hoặc chèn các tệp cấu hình cần thiết. Khi muốn cho phép workflow truy cập trực tiếp vào các gói này trong Code Node, có thể bật biến

`NODE_FUNCTION_ALLOW_EXTERNAL`. Ngoài ra, n8n hỗ trợ phát triển node tùy chỉnh bằng TypeScript để định nghĩa giao diện, logic xử lý và thông tin xác thực, từ đó

biến việc sử dụng n8n không chỉ là ứng dụng tự động hóa mà còn là một nền tảng được mở rộng theo đặc thù của dự án.

2.3.5. Triển khai và Phân tích Luồng công việc Cốt lõi

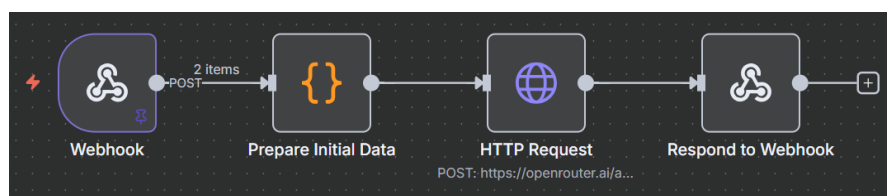
Đây là phần trung tâm của báo cáo về n8n, nơi công việc cụ thể của dự án được phân tích một cách chi tiết. Phần này sẽ giải cấu trúc các tệp JSON của workflow đã được cung cấp, giải thích logic, luồng dữ liệu và mục đích của từng thành phần trong bối cảnh của các tính năng được hỗ trợ bởi AI.

2.3.5.1. Workflow I: Cải thiện hồ sơ người dùng được Hỗ trợ bởi AI

1. Mục tiêu Workflow

Mục tiêu của workflow này là tự động hóa quy trình phân tích và đưa ra đề xuất cải thiện cho hồ sơ của người dùng. Luồng dữ liệu tổng thể như sau:

1. Người dùng gửi dữ liệu hồ sơ của họ thông qua giao diện web.
2. Hệ thống backend của ứng dụng nhận dữ liệu và kích hoạt workflow này trên n8n bằng cách gửi một yêu cầu HTTP.
3. n8n nhận dữ liệu, xử lý sơ bộ và gửi đến một dịch vụ AI bên ngoài (có thể là một API của OpenAI, Gemini hoặc một mô hình tùy chỉnh) để phân tích.
4. Dịch vụ AI trả về các đề xuất cải thiện.
5. n8n nhận các đề xuất này, định dạng lại chúng thành một cấu trúc dữ liệu chuẩn hóa.
6. Cuối cùng, n8n trả về dữ liệu đã được định dạng cho backend, sau đó backend sẽ hiển thị các đề xuất này cho người dùng trên giao diện.



Hình 2.11: Workflow Cải thiện hồ sơ người dùng

2. Giải cấu trúc CV Suggestion.json: Phân tích Từng Node

2.1 Webhook (Trigger Node)

Chức năng: Khởi động workflow khi backend gửi request đến endpoint /cv-suggestion bằng phương thức POST.

Cấu hình: response mode đặt là responseNode, tức là phản hồi sẽ do một node khác quyết định (chính là node cuối).

Ý nghĩa: Đây là điểm vào cho dữ liệu từ hệ thống backend.

Loại Node	Webhook
Mục đích trong Workflow	Tạo endpoint API để nhận dữ liệu từ backend.
Tham số/Biểu thức Chính	HTTP Method: POST, Path: /cv-suggestion, Authentication: Header Auth
Nguồn Dữ liệu Đầu vào	Yêu cầu HTTP từ backend
Cấu trúc Dữ liệu Đầu ra	Dữ liệu thô (JSON)

Bảng 2.6: Phân tích Webhook (Trigger Node)

2.2 Prepare Initial Data (Code Node)

Chức năng: Trích xuất dữ liệu gốc từ body request.

Code chính:

JSON
<pre>const original_input = \$json.body?.original_input ""; return { json: { original_input: original_input } };</pre>

Bảng 2.7: Chuẩn hóa dữ liệu đầu vào

Ý nghĩa: Chuẩn hóa dữ liệu thô thành một trường duy nhất original_input để chuyển sang bước xử lý AI.

Loại Node	Set
Mục đích trong Workflow	Chuẩn hóa dữ liệu , kết hợp các trường, và tạo prompt cho AI.
Tham số/Biểu thức Chính	fullName: {{ \$json.firstName }} {{ \$json.lastName }}, prompt: "Analyze this CV..."

Nguồn Dữ liệu Đầu vào	Output của node Webhook
Cấu trúc Dữ liệu Đầu ra	Đối tượng JSON đã được làm sạch và bổ sung

Bảng 2.8: Phân tích Prepare Initial Data (Code Node)

2.3 HTTP Request (API Communication Node)

Chức năng: Gửi original_input đến API OpenRouter (gpt-3.5-turbo) để phân tích và nâng cấp dữ liệu hồ sơ của người dùng.

Cấu hình
url: https://openrouter.ai/api/v1/chat/completions headers: Authorization (Bearer API Key), Content-Type: application/json body: JSON gồm model, temperature, định dạng output (bắt buộc JSON), cùng với system prompt chi tiết hướng dẫn AI phân loại (EXPERIENCE, SKILLS, INTRODUCTION, UNKNOWN) và nâng cấp nội dung. messages: gồm 2 phần: system: hướng dẫn chi tiết cách AI xử lý input. user: chèn dữ liệu <input_to_process> + \$json.original_input.

Bảng 2.9: Cấu hình HTTP Request

Ý nghĩa: Đây là “trái tim” của workflow — nơi AI thực hiện phân tích và sinh gợi ý cải thiện hồ sơ người dùng theo định dạng JSON chuẩn.

Loại Node	HTTP Request
Mục đích trong Workflow	Gửi original_input đến API OpenRouter (GPT-3.5-turbo) để phân tích và tạo gợi ý.
Tham số/Biểu thức Chính	url: https://openrouter.ai/api/v1/chat/completions, method: POST, headers: Authorization + Content-Type, body: JSON (system prompt + user input)
Nguồn Dữ liệu Đầu vào	Output từ node Prepare Initial Data
Cấu trúc Dữ liệu Đầu ra	Phản hồi từ API (object JSON trong

	choices[0].message.content)
--	-----------------------------

Bảng 2.10: Phân tích HTTP Request (API Communication Node)

2.4 Respond to Webhook (Response Node)

Chức năng: Gửi kết quả cuối cùng về backend.

Cấu hình: Response body

JSON
JSON.parse (\$json.choices[0].message.content)

Bảng 2.11: Parse kết quả trả về từ AI thành JSON chuẩn

Ý nghĩa: Lấy phần nội dung AI trả về, parse thành JSON, và trả trực tiếp cho request ban đầu. Backend nhận được ngay kết quả { "section": "...", "suggestion": "..." }.

Loại Node	Respond to Webhook
Mục đích trong Workflow	Trả kết quả cuối cùng về cho backend qua HTTP response.
Tham số/Biểu thức Chính	js JSON.parse(\$json.choices[0].message.content) >
Nguồn Dữ liệu Đầu vào	Output từ node HTTP Request
Cấu trúc Dữ liệu Đầu ra	{ "section": "...", "suggestion": "..." }

Bảng 2.12: Phân tích Respond to Webhook (Response Node)

2.3.5.2. Workflow II: Mô phỏng Phỏng vấn thử với AI

1. Mục tiêu Workflow

Workflow này phức tạp hơn đáng kể so với workflow cải thiện CV vì nó mang tính chất hội thoại và yêu cầu quản lý trạng thái qua nhiều lượt tương tác. Mục tiêu là mô phỏng một buổi phỏng vấn, trong đó hệ thống đặt câu hỏi, người dùng trả lời, và hệ thống đưa ra phản hồi, lặp lại cho đến khi kết thúc buổi phỏng vấn.

Luồng hỏi–đáp được triển khai theo mô hình “ping-pong”:

1. Backend → n8n (Webhook interview-analysis): Backend gửi câu trả lời ứng viên kèm session_id.

2. Interview Analyst (HTTP Request): n8n gọi API OpenRouter (GPT-3.5-turbo) để phân tích câu trả lời, chấm điểm và tạo feedback.

3. Process AI Response (Code): Parse JSON AI trả về, fallback nếu sai định dạng, đảm bảo đầu ra luôn có trường ai_score, feedback, strengths, weaknesses, suggested_answer.

4. Respond to Webhook: Trả kết quả phân tích về backend.

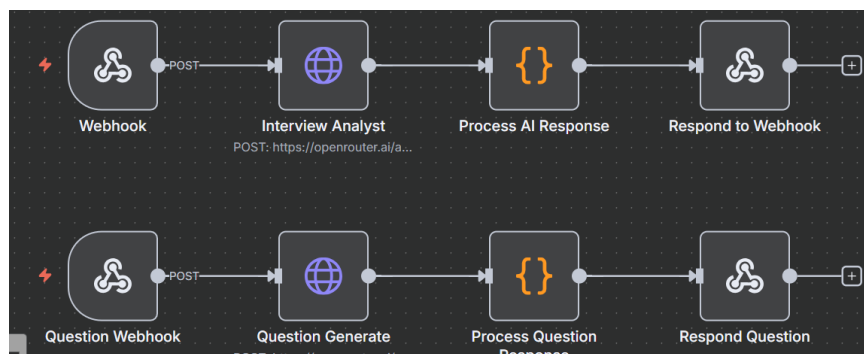
Song song với đó:

5. Backend → n8n (Question Webhook): Backend yêu cầu tạo câu hỏi mới với thông tin jobTitle, jobDescription, category, difficulty...

6. Question Generate (HTTP Request): Gọi AI để sinh câu hỏi tiếng Việt.

7. Process Question Response (Code): Parse kết quả, fallback nếu AI trả về sai.

8. Respond Question: Trả câu hỏi mới cho backend.



Hình 2.12: Workflow Mô phỏng Phỏng vấn thử

Như vậy, backend sẽ nhận cả kết quả đánh giá câu trả lời và câu hỏi tiếp theo, rồi hiển thị lại cho ứng viên. Quá trình này lặp lại cho đến khi backend quyết định kết thúc.

2. Giải cấu trúc Workflow Mock Interview.json

2.1 Webhook (interview-analysis) – Trigger Node

Chức năng: Khởi động workflow phân tích câu trả lời khi backend gửi request đến endpoint /interview-analysis bằng phương thức POST.

Cấu hình: responseMode: responseNode, nghĩa là phản hồi sẽ do node cuối quyết định.

Ý nghĩa: Đây là điểm vào để nhận câu trả lời ứng viên từ hệ thống backend, kèm session context.

Loại Node	Webhook
Mục đích trong Workflow	Nhận câu trả lời ứng viên từ backend qua endpoint /interview-analysis (POST).
Tham số/Biểu thức Chính	httpMethod: POST, path: interview-analysis, responseMode: responseNode
Nguồn Dữ liệu Đầu vào	Yêu cầu HTTP từ backend
Cấu trúc Dữ liệu Đầu ra	Dữ liệu CV/Answer thô (JSON trong request body)

Bảng 2.13: Phân tích Webhook (interview-analysis) – Trigger Node

2.2 Interview Analyst (HTTP Request Node)

Chức năng: Gửi câu hỏi + câu trả lời ứng viên đến API OpenRouter (GPT-3.5-turbo) để phân tích.

Cấu hình
url: https://openrouter.ai/api/v1/chat/completions headers: Authorization (Bearer API Key), Content-Type: application/json body: gồm model, temperature, và prompt (hướng dẫn AI chấm điểm 0–100, feedback ngắn gọn, strengths, weaknesses, suggested_answer). messages: system: chỉ dẫn chi tiết cách chấm điểm. user: nội dung gồm câu hỏi và câu trả lời ứng viên.

Bảng 2.14: Cấu hình Interview Analyst (HTTP Request Node)

Ý nghĩa: Đây là nơi AI thực hiện phân tích câu trả lời của ứng viên và tạo phản hồi dưới dạng JSON.

Loại Node	HTTP Request
Mục đích trong Workflow	Gửi câu hỏi + câu trả lời đến API OpenRouter (GPT-3.5-turbo) để phân tích.
Tham số/Biểu thức Chính	url: https://openrouter.ai/api/v1/chat/completions, headers:

	Authorization + Content-Type, body: JSON gồm system prompt + user input
Nguồn Dữ liệu Đầu vào	Output từ node Webhook
Cấu trúc Dữ liệu Đầu ra	Kết quả AI trả về (JSON trong choices[0].message.content)

Bảng 2.15: Phân tích Interview Analyst (HTTP Request Node)

2.3 Process AI Response (Code Node)

Chức năng: Parse JSON trả về từ AI, đảm bảo đầu ra hợp lệ ngay cả khi AI trả sai định dạng.

JSON
<pre>const content = \$json.choices[0].message.content; try { const parsed = JSON.parse(content); return { ai_score: parsed.ai_score, feedback: parsed.feedback, ... }; } catch { return { ai_score: 50, feedback: "Fallback message" }; }</pre>

Bảng 2.16: Code xử lý phản hồi AI

Ý nghĩa: Đảm bảo output luôn có cấu trúc chuẩn { ai_score, feedback, strengths, weaknesses, suggested_answer }, giúp backend không bị lỗi khi xử lý.

Loại Node	Code
Mục đích trong Workflow	Parse kết quả JSON từ AI, fallback nếu định dạng sai.
Tham số/Biểu thức Chính	JSON.parse(content) với fallback object nếu lỗi
Nguồn Dữ liệu Đầu vào	Output từ node Interview Analyst
Cấu trúc Dữ liệu Đầu ra	{ "ai_score": ..., "feedback": "...", "strengths": "...",

	"weaknesses": "...", "suggested_answer": "..." }
--	--

Bảng 2.17: Phân tích Process AI Response (Code Node)

2.4 Respond to Webhook (Response Node)

Chức năng: Trả kết quả phân tích cuối cùng về backend.

Cấu hình
responseBody = \$json

Bảng 2.18: Cấu hình Respond to Webhook (Response Node)

Ý nghĩa: Gửi toàn bộ object JSON đã được xử lý ở bước trước cho backend, kết thúc một lượt hỏi–đáp.

Loại Node	Respond to Webhook
Mục đích trong Workflow	Trả kết quả phân tích về backend qua HTTP response.
Tham số/Biểu thức Chính	responseBody = \$json
Nguồn Dữ liệu Đầu vào	Output từ node Process AI Response
Cấu trúc Dữ liệu Đầu ra	JSON chứa kết quả phân tích AI

Bảng 2.19: Phân tích Question Webhook (generate-question) – Trigger Node

2.5 Question Webhook (generate-question) – Trigger Node

Chức năng: Nhận yêu cầu từ backend để sinh câu hỏi phỏng vấn mới.

Cấu hình
Endpoint /generate-question
httpMethod: POST
responseMode: responseNode

Bảng 2.20: Cấu hình Question Webhook (generate-question) – Trigger Node

Ý nghĩa: Đây là điểm vào để tạo câu hỏi mới dựa trên thông tin jobTitle, jobDescription, category, difficulty.

Loại Node	Webhook
------------------	---------

Mục đích trong Workflow	Nhận yêu cầu sinh câu hỏi mới từ backend.
Tham số/Biểu thức Chính	httpMethod: POST, path: generate-question, responseMode: responseNode
Nguồn Dữ liệu Đầu vào	Yêu cầu HTTP từ backend (jobTitle, jobDescription, category, difficulty, questionNumber)
Cấu trúc Dữ liệu Đầu ra	Dữ liệu input cho AI sinh câu hỏi

Bảng 2.21: Phân tích Question Generate (HTTP Request Node)

2.6 Question Generate (HTTP Request Node)

Chức năng: Gọi API OpenRouter để sinh câu hỏi phỏng vấn tiếng Việt.

Cấu hình
url: https://openrouter.ai/api/v1/chat/completions headers: Authorization (Bearer API Key), Content-Type: application/json body: prompt yêu cầu AI sinh câu hỏi duy nhất, đúng format JSON với các trường: question, category, difficulty, expected_skills, context.

Bảng 2.22: Question Generate (HTTP Request Node)

Ý nghĩa: Đây là nơi AI tạo câu hỏi mới phù hợp với yêu cầu công việc và không trùng lặp với các câu hỏi trước đó.

Loại Node	HTTP Request
Mục đích trong Workflow	Gọi API OpenRouter để sinh câu hỏi phỏng vấn tiếng Việt.
Tham số/Biểu thức Chính	url: https://openrouter.ai/api/v1/chat/completions, headers: Authorization + Content-Type, body: JSON prompt yêu cầu câu hỏi duy nhất
Nguồn Dữ liệu Đầu vào	Output từ node Question Webhook

Cấu trúc Dữ liệu Đầu ra	Kết quả AI trả về (JSON trong choices[0].message.content)
--------------------------------	---

Bảng 2.23: Phân tích Question Generate (HTTP Request Node)

2.7 Process Question Response (Code Node)

Chức năng: Parse JSON trả về từ AI để lấy câu hỏi mới, fallback nếu sai định dạng.

JSON
<pre>const content = \$json.choices[0].message.content; try { const parsed = JSON.parse(content); return { question: parsed.question, category: parsed.category, ... }; } catch { return { question: "Fallback question", category: "general" }; }</pre>

Bảng 2.24: Code xử lý câu hỏi từ AI sinh ra

Ý nghĩa: Đảm bảo output luôn hợp lệ, dù AI trả về sai định dạng.

Loại Node	Code
Mục đích trong Workflow	Parse JSON câu hỏi do AI sinh ra, fallback nếu định dạng sai.
Tham số/Biểu thức Chính	JSON.parse(content) với fallback { question, category, difficulty, expected_skills, context }
Nguồn Dữ liệu Đầu vào	Output từ node Question Generate
Cấu trúc Dữ liệu Đầu ra	{ "question": "...", "category": "...", "difficulty": "...", "expected_skills": "...", "context": "..." }

Bảng 2.25: Phân tích Process Question Response (Code Node)

2.8 Respond Question (Response Node)

Chức năng: Trả câu hỏi mới đã được xử lý về backend.

Cấu hình
responseBody = \$json

Bảng 2.26: Cấu hình Respond Question (Response Node)

Ý nghĩa: Hoàn tất chu trình tạo câu hỏi, backend có thể hiển thị cho ứng viên.

Loại Node	Respond to Webhook
Mục đích trong Workflow	Trả câu hỏi mới đã được xử lý về backend qua HTTP response.
Tham số/Biểu thức Chính	responseBody = \$json
Nguồn Dữ liệu Đầu vào	Output từ node Process Question Response
Cấu trúc Dữ liệu Đầu ra	JSON chứa câu hỏi phỏng vấn mới

Bảng 2.27: Phân tích Respond Question (Response Node)

Kết luận lại, **n8n** đã đóng một vai trò chuyển đổi trong kiến trúc của dự án này. Nó không chỉ hoạt động như một công cụ tự động hóa tác vụ, mà còn là một bộ điều phối dịch vụ vi mô và API nhẹ, thân thiện với nhà phát triển. Việc áp dụng **n8n** đã cho phép một nhà phát triển cá nhân xây dựng một hệ thống mạnh mẽ, dễ bảo trì và có khả năng mở rộng, điều mà sẽ khó có thể đạt được từ đầu trong khung thời gian của một đề án. Bằng cách xử lý các công việc nặng nhọc về cơ sở hạ tầng tích hợp, **n8n** đã cho phép dự án tập trung vào giá trị cốt lõi của nó: cung cấp các tính năng thông minh, được hỗ trợ bởi AI, cho người dùng cuối. Do đó, **n8n** có thể được coi là một yếu tố xúc tác quan trọng, không chỉ giúp dự án thành công về mặt chức năng mà còn nâng cao chất lượng kiến trúc của nó lên một tầm chuyên nghiệp.

2.4. Các công cụ và thư viện hỗ trợ

2.4.1. Docker

2.4.1.1. Giới thiệu chung về Docker

Docker là một nền tảng mã nguồn mở cho phép đóng gói ứng dụng cùng với tất cả các thư viện, công cụ và môi trường cần thiết vào trong một đơn vị triển khai gọi là container. Container này có thể chạy đồng nhất trên mọi hệ thống, từ máy phát triển cá

nhân cho đến môi trường đám mây, mà không bị ảnh hưởng bởi sự khác biệt về hệ điều hành hay cấu hình.

So với cách triển khai truyền thống, Docker mang lại sự nhẹ nhàng hơn so với máy ảo (virtual machine) vì container chỉ chia sẻ kernel của hệ điều hành, không cần chạy một hệ điều hành riêng biệt. Điều này giúp container khởi động nhanh, tốn ít tài nguyên hơn và dễ dàng mở rộng khi cần. [5]

2.4.1.2. Tại sao nên sử dụng Docker?

Có nhiều lý do khiến Docker trở thành một công cụ quan trọng trong phát triển và triển khai phần mềm hiện nay:

Tính nhất quán môi trường: Docker loại bỏ tình huống “chạy được trên máy tôi nhưng không chạy trên server”, bởi vì container đảm bảo toàn bộ dependency và cấu hình luôn đồng nhất.

Triển khai nhanh chóng: Một ứng dụng có thể được build thành image và triển khai nhiều nơi chỉ với vài dòng lệnh.

Khả năng cách ly: Các container hoạt động độc lập với nhau, giúp giảm thiểu rủi ro xung đột thư viện hoặc phiên bản phần mềm.

Mở rộng linh hoạt: Khi hệ thống cần phục vụ nhiều người dùng hơn, chỉ cần tăng số lượng container chạy song song thay vì cấu hình phức tạp.

Hệ sinh thái phong phú: Docker Hub cung cấp hàng nghìn image có sẵn (ví dụ: MySQL, Redis, PostgreSQL) giúp tiết kiệm thời gian cài đặt.

2.4.1.3. Docker hoạt động như thế nào?

Docker vận hành dựa trên ý tưởng tách biệt ứng dụng ra khỏi môi trường hệ điều hành cơ bản, nhờ đó ứng dụng có thể chạy ở bất kỳ đâu. Quá trình làm việc với Docker có thể hình dung qua ba bước chính:

1. Xây dựng (Build):

Nhà phát triển viết một tệp Dockerfile, trong đó mô tả chi tiết cách ứng dụng cần được đóng gói. Ví dụ: chọn một base image (chẳng hạn openjdk:17 cho backend Java Spring Boot), cài đặt các thư viện phụ thuộc, sao chép mã nguồn vào trong image, và

định nghĩa công mà ứng dụng sẽ lắng nghe. Đây giống như một bản “công thức nấu ăn” cho ứng dụng.

2. Tạo image:

Từ Dockerfile, Docker tiến hành build ra một image. Image này là một gói chứa toàn bộ ứng dụng và môi trường cần thiết (runtime, thư viện, cấu hình). Image có tính bất biến, nghĩa là một khi được build ra thì nó luôn giữ nguyên nội dung, giúp đảm bảo tính nhất quán giữa các lần triển khai.

3. Chạy container:

Từ image, Docker có thể khởi tạo một hoặc nhiều container. Mỗi container là một instance độc lập của ứng dụng, có thể chạy song song với các container khác mà không ảnh hưởng lẫn nhau. Điểm khác biệt lớn so với máy ảo là container không cần hệ điều hành riêng, chúng chỉ chia sẻ kernel của hệ điều hành host nhưng vẫn giữ môi trường tách biệt. Vì vậy, container nhẹ hơn nhiều, khởi động gần như tức thì và tốn ít tài nguyên.

Nhờ cơ chế này, Docker giúp việc triển khai ứng dụng trở nên linh hoạt: một image build ra có thể chạy trên laptop của lập trình viên, server nội bộ hoặc các dịch vụ đám mây như AWS, GCP, Render... mà không cần thay đổi cấu hình. Trong project FlexiConnect, điều này có nghĩa là backend Spring Boot và dịch vụ n8n có thể được build thành image một lần duy nhất rồi triển khai ổn định trên Render, bất kể môi trường ban đầu ra sao.

2.4.1.4. Liên hệ với đề tài

Trong project FlexiConnect, Docker được sử dụng để triển khai backend Spring Boot và dịch vụ n8n lên Render. Nhờ Dockerfile, nhóm phát triển không cần lo lắng việc cài đặt JDK, Maven, Node.js hay cấu hình phức tạp trên máy chủ Render. Thay vào đó, chỉ cần build image một lần và Render sẽ tự động chạy container dựa trên image này.

Điều này giúp backend và n8n hoạt động ổn định, dễ dàng mở rộng khi cần, đồng thời loại bỏ sự khác biệt giữa môi trường local và production. Trong tương lai, Docker cũng có thể được mở rộng để tích hợp thêm Redis, tất cả đều quản lý trong một hệ thống container hóa thống nhất.

2.4.2. Nền tảng Triển khai: Render và Clever Cloud

2.4.2.1. Render

[6]Render là một nền tảng Platform-as-a-Service (PaaS) cho phép triển khai ứng dụng web, dịch vụ nền và static site mà không cần trực tiếp quản lý máy chủ. Nền tảng này hỗ trợ nhiều ngôn ngữ phổ biến như Java, Node.js, Python, Go và có khả năng triển khai trực tiếp từ Dockerfile, giúp các ứng dụng container hóa chạy nhất quán trên mọi môi trường.

Ngoài ra, Render tích hợp sẵn CI/CD, tự động build và deploy mỗi khi có commit mới, đi kèm các dịch vụ tiện ích như cấp phát domain, SSL miễn phí, logging và autoscaling. Trong project FlexiConnect, Render được lựa chọn để triển khai backend Spring Boot, frontend React/Vite và dịch vụ nền, nhờ đó toàn bộ ứng dụng có thể hoạt động ổn định trên cùng một nền tảng.



Hình 2.13: Render

2.4.2.2. Clever Cloud

Clever Cloud là một nền tảng đám mây chuyên cung cấp **DBaaS**, nổi bật với khả năng quản lý và vận hành cơ sở dữ liệu một cách tự động. Nền tảng này hỗ trợ nhiều hệ quản trị như PostgreSQL, MySQL, MongoDB, kèm theo các tính năng quan trọng như giám sát, sao lưu định kỳ, phục hồi dữ liệu và mở rộng tài nguyên linh hoạt.

Điểm mạnh của Clever Cloud nằm ở việc giảm tải hoàn toàn phần vận hành cơ sở dữ liệu: người dùng chỉ cần khởi tạo một instance, hệ thống sẽ lo toàn bộ quá trình cài đặt, bảo mật và cập nhật. Trong FlexiConnect, MySQL được triển khai trên Clever Cloud để đảm bảo dữ liệu ứng viên và công việc luôn ổn định, an toàn và có khả năng mở rộng khi nhu cầu tăng cao.



Hình 2.14: Clever Cloud

2.4.2.3. Tại sao chọn Render cho ứng dụng và Clever Cloud cho database?

Render cho Backend và Frontend:

Render tích hợp sẵn CI/CD, hỗ trợ Dockerfile, có domain và SSL tự động, rất phù hợp để triển khai backend Spring Boot (cần container hóa) và frontend React/Vite (deploy dưới dạng static site). Render giúp quy trình triển khai trở nên đơn giản, nhanh chóng và dễ mở rộng khi số lượng người dùng tăng.

Clever Cloud cho Database MySQL:

Database đòi hỏi khả năng quản lý chuyên sâu (sao lưu, khôi phục, giám sát tài nguyên, bảo mật kết nối). Clever Cloud cung cấp sẵn MySQL như một dịch vụ, lo toàn bộ việc vận hành hạ tầng, cho phép backend và n8n kết nối thông qua credential an toàn. Nhờ đó, dữ liệu của dự án được đảm bảo ổn định và bền vững hơn.

Kết hợp Render và Clever Cloud tạo thành một kiến trúc cân bằng: ứng dụng (frontend, backend, n8n) chạy trên Render, còn cơ sở dữ liệu được vận hành chuyên nghiệp trên Clever Cloud.

Sự phân tách này giúp hạ tầng dễ quản lý hơn: Render tập trung phục vụ ứng dụng và người dùng cuối, còn Clever Cloud đảm bảo dữ liệu được lưu trữ an toàn và có khả năng mở rộng linh hoạt.

Chương 3. HỆ THỐNG WEB TÌM VIỆC LÀM

3.1. Giới thiệu hệ thống web tìm việc làm

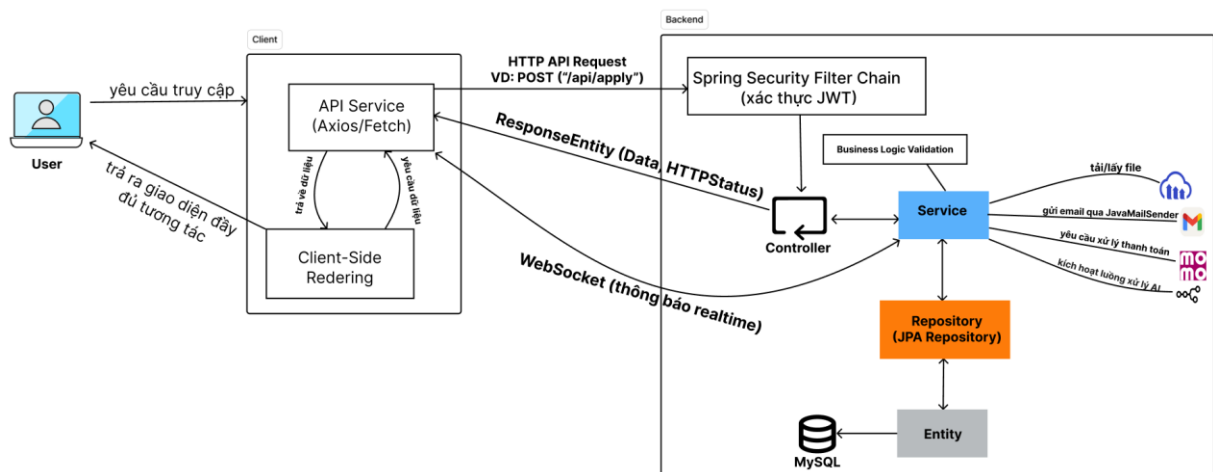
Tuyển dụng nhân sự trong thị trường lao động hiện đại là một quy trình cũng như một bài toán khá phức tạp và đòi hỏi sự kết nối hiệu quả giữa hai đối tượng chính: nhà tuyển dụng và ứng viên. Các doanh nghiệp luôn phải đối mặt với thách thức trong việc thu hút, sàng lọc và lựa chọn những ứng viên phù hợp nhất từ hàng trăm, hàng nghìn hồ sơ. Và về phía ứng viên, họ cũng gặp khó khăn trong việc tạo một hồ sơ nổi bật, tiếp cận đúng nhà tuyển dụng.

Mặc dù hiện nay đã có nhiều nền tảng tuyển dụng, thách thức chính không nằm ở việc thiếu công cụ kết nối mà là ở chất lượng của sự kết nối đó, làm sao để ghép nối đúng người với đúng việc một cách thông minh và hiệu quả. Chính vì vậy, dự án FlexiConnect được đề xuất nhằm giải quyết bài toán về tối ưu hóa sự phù hợp này, đáp ứng nhu cầu thực tiễn của cả ứng viên và nhà tuyển dụng.

Để thực hiện mục tiêu trên, một nền tảng tuyển dụng toàn diện được thiết kế, bao gồm đầy đủ các chức năng cốt lõi. Hệ thống không chỉ là nơi đăng tin và nộp hồ sơ, mà còn cung cấp các công cụ AI để hỗ trợ ứng viên hoàn thiện hồ sơ, rèn luyện kỹ năng phỏng vấn, đồng thời giúp nhà tuyển dụng sàng lọc hiệu quả. Nền tảng này giúp tối ưu hóa quy trình, nâng cao chất lượng kết nối và tăng cường lợi thế cạnh tranh cho cả hai bên.

Phần báo cáo này sẽ đi sâu phân tích hệ thống tìm kiếm việc làm với các chức năng chính như quản lý hồ sơ, quy trình đăng tin và ứng tuyển, cùng các tính năng hỗ trợ bằng AI, thông qua các sơ đồ và mô hình minh họa sự vận hành và tương tác giữa các tác nhân liên quan.

3.2. Kiến trúc hệ thống



Hình 3.1: Luồng xử lý dữ liệu hệ thống web tìm việc làm

1. Khi người dùng thực hiện một thao tác trên giao diện (ví dụ: nhấn nút "Nộp hồ sơ"), **Client (ReactJS)** sẽ được kích hoạt.
2. Thành phần **API Service** (sử dụng Axios/Fetch) bên trong **Client** sẽ tạo và gửi một HTTP API Request (ví dụ: POST `"/api/apply"`) đến hệ thống **Backend**. Yêu cầu này mang theo dữ liệu cần thiết do người dùng cung cấp.
3. Yêu cầu từ Client sẽ được tiếp nhận đầu tiên bởi **Spring Security Filter Chain** của hệ thống **Backend (Spring Boot)**. Tại đây, hệ thống thực hiện xác thực và ủy quyền, điển hình là kiểm tra tính hợp lệ của **JWT** để đảm bảo yêu cầu là hợp lệ và an toàn.
4. Sau khi xác thực thành công, yêu cầu sẽ được chuyển đến lớp **Controller**. **Controller** chịu trách nhiệm điều hướng yêu cầu đến đúng lớp **Service** tương ứng để xử lý logic nghiệp vụ.
5. Lớp **Service** nhận dữ liệu và bắt đầu xử lý logic nghiệp vụ chính. Tại đây, hệ thống sẽ thực hiện **Business Logic Validation** để kiểm tra tính hợp lệ của dữ liệu (ví dụ: kiểm tra xem ứng viên đã ứng tuyển công việc này chưa). Lớp **Service** cũng là nơi điều phối các tương tác với dịch vụ bên ngoài nếu cần thiết, chẳng hạn như: Gửi yêu cầu tải file (CV, avatar) lên **Cloudinary**. Kích hoạt một luồng xử lý AI thông qua **n8n**. Yêu cầu xử lý thanh toán qua cổng **MoMo**. Gửi email thông báo cho người dùng qua **JavaMailSender**.
6. Để tương tác với cơ sở dữ liệu, Service sẽ gọi đến lớp **Repository** (sử dụng **JPA Repository**). Lớp Repository chịu trách nhiệm thực hiện các thao tác truy

vấn, thêm, xóa, sửa dữ liệu với cơ sở dữ liệu **MySQL** thông qua các đối tượng **Entity** đã được định nghĩa.

7. Sau khi lớp **Repository** hoàn tất xử lý và trả về kết quả, **Service** sẽ gửi dữ liệu này ngược lại cho **Controller**. Controller sẽ đóng gói dữ liệu và trạng thái xử lý (thành công hay thất bại) vào một đối tượng **ResponseEntity** rồi gửi phản hồi về cho Client.
8. Client nhận được **ResponseEntity** từ Backend. **API Service** sẽ xử lý dữ liệu này và cập nhật trạng thái của ứng dụng. Quá trình **Client-Side Rendering** sẽ diễn ra, giao diện người dùng được cập nhật một cách linh hoạt với dữ liệu mới mà không cần tải lại toàn bộ trang.
9. Đối với các thông báo thời gian thực (real-time), hệ thống sử dụng **WebSocket**. Khi có một sự kiện quan trọng ở Backend (ví dụ: trạng thái đơn ứng tuyển thay đổi), lớp **Service** có thể chủ động đẩy thông báo trực tiếp xuống Client, giúp cập nhật giao diện ngay lập tức mà không cần người dùng phải gửi yêu cầu mới.

3.3. Phân tích hệ thống

3.3.1. Use case Tổng quan hệ thống

Quy trình tuyển dụng trực tuyến yêu cầu hệ thống phải đảm bảo các bước tương tác giữa ứng viên và nhà tuyển dụng được thực hiện một cách liền mạch và hiệu quả. Use case này mô tả cách các người dùng (bao gồm Ứng viên, Nhà tuyển dụng, và Quản trị viên) tương tác với hệ thống để thực hiện toàn bộ chu trình tuyển dụng, từ việc đăng ký tài khoản, tìm kiếm việc làm, ứng tuyển, đăng tin tuyển dụng, cho đến việc quản lý các đơn ứng tuyển và thanh toán dịch vụ.

Mục tiêu của sơ đồ use case là phân tích các hành động và luồng công việc giữa người dùng và hệ thống, giúp nhận diện các chức năng chính cho từng vai trò, các quy trình cốt lõi như ứng tuyển hay đăng tin, các trường hợp ngoại lệ có thể xảy ra và các điểm tích hợp quan trọng. Sơ đồ cũng hình dung sự tương tác giữa các actor chính (Ứng viên, Nhà tuyển dụng) với các actor phụ là những hệ thống bên ngoài (Momo, Cloudinary, AI, Gmail) để đảm bảo hệ thống hoạt động một cách toàn diện.

	Ứng viên qua email/thông báo hệ thống.
4. Nhà tuyển dụng nhận được thông báo có ứng viên mới ứng tuyển.
Luồng hoạt động (Main Flow)	1. Ứng viên tìm thấy một tin tuyển dụng phù hợp và chọn chức năng "Ứng tuyển". 2. Hệ thống hiển thị giao diện ứng tuyển, có thể yêu cầu điền thêm thông tin như thư xin việc (cover letter). 3. Hệ thống yêu cầu Ứng viên tải lên CV. (Thực thi Use Case <<include>> Tải CV). 4. Ứng viên chọn một tệp CV từ thiết bị của mình. 5. Hệ thống kết nối với dịch vụ Cloudinary để tải tệp CV lên. 6. Sau khi tải CV thành công, Ứng viên nhấn nút "Hoàn tất ứng tuyển". 7. Hệ thống lưu lại hồ sơ ứng tuyển vào cơ sở dữ liệu. 8. Hệ thống kích hoạt chức năng gửi thông báo. (Thực thi Use Case <<include>> Gửi thông báo). 9. Hệ thống gửi email xác nhận cho Ứng viên và thông báo cho Nhà tuyển dụng.
Luồng thay thế (Alternative Flow)	- Tại bước 3, nếu Ứng viên đã có CV được lưu sẵn trong "Quản lý hồ sơ cá nhân", hệ thống có thể cho phép họ chọn sử dụng CV đó mà không cần tải lên tệp mới.
Luồng ngoại lệ (Exception Flow)	- Tại bước 5, nếu quá trình tải CV lên Cloudinary thất bại (do lỗi kết nối, tệp không hợp lệ, dung lượng quá lớn), hệ thống sẽ báo lỗi và yêu cầu Ứng viên thử lại.
- Tại bước 7, nếu hệ thống phát hiện Ứng viên đã ứng tuyển vào tin tuyển dụng này trước đó, hệ thống sẽ hiển thị thông báo "Bạn đã ứng tuyển vị trí này" và không cho phép tạo đơn mới.

Bảng 3.1: Đặc tả use case Tổng quan hệ thống

3.3.3. Use case Ứng viên

Quy trình tìm kiếm việc làm của ứng viên yêu cầu hệ thống cung cấp một bộ công cụ toàn diện và thân thiện, giúp họ tối ưu hóa cơ hội nghề nghiệp. Use case này mô tả chi tiết các tương tác của một **Ứng viên** với hệ thống, bao gồm các bước cơ bản như đăng ký tài khoản, tìm việc, ứng tuyển, cho đến các tính năng nâng cao như hoàn thiện hồ sơ (bằng phương pháp thủ công hoặc cải thiện bằng AI), luyện tập phỏng vấn, và nâng cấp tài khoản để sử dụng các dịch vụ cao cấp.

Mục tiêu của sơ đồ use case này là phân tích sâu vào luồng công việc và các hành động cụ thể từ góc nhìn của Ứng viên. Việc này giúp nhận diện tất cả các tính năng cần thiết để hỗ trợ ứng viên trong suốt quá trình tìm việc, xác định các trường hợp sử dụng đặc thù và các điểm tương tác với hệ thống bên ngoài. Sơ đồ cũng hình dung sự liên kết logic giữa các hành động, ví dụ như việc "Nâng cấp tài khoản" có thể là điều kiện để truy cập "Phòng vấn luyện tập", và sự phụ thuộc vào các dịch vụ như n8n (cho AI), Momo (thanh toán), và Clouddinary (lưu trữ CV).

3.3.4. Đặc tả use case Ứng viên

Tên use case	Cải thiện hồ sơ bằng AI (Improve Profile with AI)
Mô tả	Use case này mô tả quá trình Ứng viên sử dụng trí tuệ nhân tạo (AI) để tự động phân tích và tối ưu hóa nội dung trong hồ sơ cá nhân của mình. Hệ thống sẽ gửi dữ liệu hồ sơ hiện tại đến một dịch vụ AI và hiển thị các gợi ý cải thiện (ví dụ: viết lại kinh nghiệm làm việc, đề xuất kỹ năng liên quan) để ứng viên xem xét và áp dụng.
Actor chính	Ứng viên
Actor phụ	n8n (Dịch vụ AI/Automation)
Pre-conditions (Tiền điều kiện)	1. Ứng viên phải đăng nhập vào hệ thống. 2. Ứng viên phải đang ở trong trang "Hoàn thiện hồ sơ" hoặc trang quản lý hồ sơ cá nhân. 3. Hồ sơ của ứng viên phải có một số dữ liệu ban đầu (ví dụ: đã

	nhập kinh nghiệm làm việc) để AI có thể phân tích.
Post-conditions (Hậu điều kiện)	1. Nội dung trong hồ sơ của ứng viên được cập nhật với các thay đổi đã được chấp thuận từ gợi ý của AI. 2. Dữ liệu hồ sơ mới được lưu thành công vào cơ sở dữ liệu của hệ thống.
Luồng hoạt động (Main Flow)	1. Ứng viên truy cập vào trang quản lý hồ sơ và chọn chức năng "Cải thiện bằng AI". 2. Hệ thống thu thập dữ liệu hiện có từ hồ sơ của ứng viên (ví dụ: các đoạn mô tả công việc, danh sách kỹ năng). >3. Hệ thống thực hiện gọi API đến dịch vụ n8n, gửi kèm dữ liệu đã thu thập. (Thực thi Use Case <<include>> gọi API n8n). 4. Dịch vụ n8n xử lý dữ liệu và trả về một danh sách các gợi ý cải thiện. 5. Hệ thống hiển thị các gợi ý này cho Ứng viên, có thể dưới dạng so sánh "trước" và "sau". 6. Ứng viên xem xét các gợi ý và nhấn nút "Chấp nhận" hoặc "Áp dụng" để cập nhật hồ sơ. 7. Hệ thống cập nhật các trường dữ liệu tương ứng trong hồ sơ của Ứng viên. 8. Hệ thống hiển thị thông báo "Cập nhật hồ sơ thành công".
Luồng thay thế (Alternative Flow)	- Tại bước 6, Ứng viên có thể chọn chỉ áp dụng một vài gợi ý thay vì tất cả. Hệ thống cho phép chọn từng mục để cập nhật. - Ứng viên có thể chỉnh sửa lại nội dung do AI gợi ý trước khi lưu.
Luồng ngoại lệ (Exception Flow)	- Tại bước 3, nếu việc gọi API đến n8n thất bại (do lỗi mạng, hết hạn credit, API không phản hồi), hệ thống sẽ hiển thị thông báo lỗi: "Tính năng AI tạm thời không khả dụng, vui lòng thử lại sau."

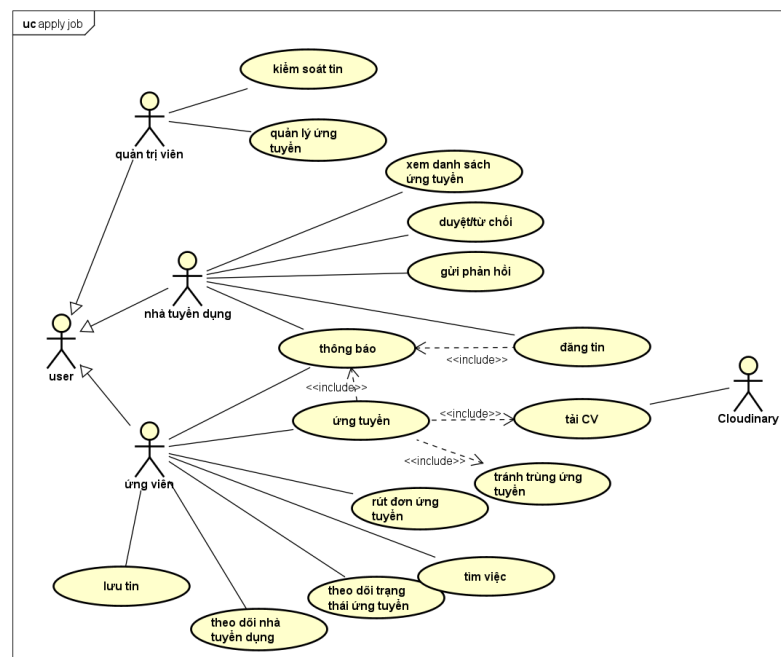
	- Tại bước 4, nếu dịch vụ n8n trả về kết quả không có gợi ý nào hữu ích, hệ thống sẽ thông báo: "Hồ sơ của bạn đã khá tốt, AI không tìm thấy đề xuất nào thêm."
--	---

Bảng 3.2: Đặc tả use case Ứng viên

3.3.5. Use case Tổng quan quy trình tuyển dụng

Quy trình tuyển dụng và ứng tuyển toàn diện yêu cầu hệ thống phải quản lý được một chu trình tương tác phức tạp, đảm bảo luồng thông tin thông suốt từ lúc Nhà tuyển dụng đăng tin cho đến khi Ứng viên nộp đơn và được xét duyệt. Use case này mô tả cách các vai trò người dùng chính tương tác với hệ thống: Ứng viên thực hiện các hành động từ tìm kiếm, lưu tin, ứng tuyển cho đến theo dõi trạng thái; Nhà tuyển dụng đăng tin, quản lý và xét duyệt các hồ sơ ứng tuyển; và Quản trị viên thực hiện các công việc giám sát, kiểm soát chung.

Mục tiêu của sơ đồ use case là phân tích và trực quan hóa toàn bộ các hành động và mối quan hệ trong hệ thống tuyển dụng. Sơ đồ giúp nhận diện các chức năng chính của từng vai trò, làm rõ cấu trúc kế thừa từ một user chung, và định nghĩa các bước phụ bắt buộc (<<include>>) trong các quy trình cốt lõi như Đăng tin và Ứng tuyển. Đồng thời, sơ đồ cũng hình dung sự tương tác giữa hệ thống với các dịch vụ bên ngoài quan trọng như Cloudinary để phục vụ cho các nghiệp vụ cụ thể.



Hình 3.3: Sơ đồ Use case Tổng quan quy trình tuyển dụng

3.3.6. Đặc tả Use case Tổng quan quy trình tuyển dụng

Tên use case	Duyệt/Từ chối ứng viên (Approve/Reject Application)
Mô tả	Use case này mô tả hành động Nhà tuyển dụng (hoặc Quản trị viên) ra quyết định đối với một đơn ứng tuyển cụ thể sau khi đã xem xét. Hành động này sẽ thay đổi trạng thái của đơn ứng tuyển (ví dụ: từ "Mới" sang "Phù hợp" hoặc "Đã từ chối") và có thể kích hoạt việc gửi phản hồi cho ứng viên.
Actor chính	Nhà tuyển dụng
Actor phụ	Ứng viên (người nhận kết quả), Quản trị viên (có thể có quyền thực hiện)
Pre-conditions (Tiền điều kiện)	1. Nhà tuyển dụng phải đăng nhập vào hệ thống. 2. Phải có ít nhất một đơn ứng tuyển đang ở trạng thái chờ xử lý (ví dụ: "Mới", "Đã xem") cho một tin tuyển dụng của họ. 3. Nhà tuyển dụng đang ở trang Xem danh sách ứng tuyển.
Post-conditions (Hậu điều kiện)	1. Trạng thái của đơn ứng tuyển trong cơ sở dữ liệu được cập nhật thành "Phù hợp" (được duyệt) hoặc "Đã từ chối". 2. Một thông báo về việc thay đổi trạng thái được gửi đến Ứng viên. 3. Hệ thống ghi lại (log) hành động duyệt/từ chối này.
Luồng hoạt động (Main Flow)	1. Nhà tuyển dụng chọn một hoặc nhiều ứng viên từ danh sách ứng tuyển. 2. Nhà tuyển dụng nhấn vào nút hành động "Duyệt" hoặc "Từ chối". 3. Hệ thống có thể hiển thị một hộp thoại yêu cầu xác nhận. 4. Sau khi Nhà tuyển dụng xác nhận, hệ thống cập nhật trạng thái cho (các) đơn ứng tuyển đã chọn trong cơ sở dữ liệu. 5. Hệ thống kích hoạt Use Case Gửi phản hồi (hoặc một thông báo tự động) đến (các) ứng viên tương ứng.

	6. Giao diện danh sách ứng viên được cập nhật lại để hiển thị trạng thái mới.
Luồng thay thế (Alternative Flow)	- Nhà tuyển dụng có thể duyệt một ứng viên và hệ thống tự động chuyển ứng viên đó sang một giai đoạn mới trong quy trình tuyển dụng (ví dụ: "Chờ xếp lịch phỏng vấn"). - Nhà tuyển dụng từ chối một ứng viên và hệ thống cung cấp tùy chọn để gửi một email phản hồi theo mẫu có sẵn.
Luồng ngoại lệ (Exception Flow)	- Nếu việc cập nhật trạng thái trong cơ sở dữ liệu thất bại do lỗi hệ thống, một thông báo lỗi sẽ được hiển thị và trạng thái của đơn ứng tuyển không thay đổi. - Nếu đơn ứng tuyển đã được ứng viên Rút đơn trước đó, các nút "Duyệt" và "Từ chối" sẽ bị vô hiệu hóa.

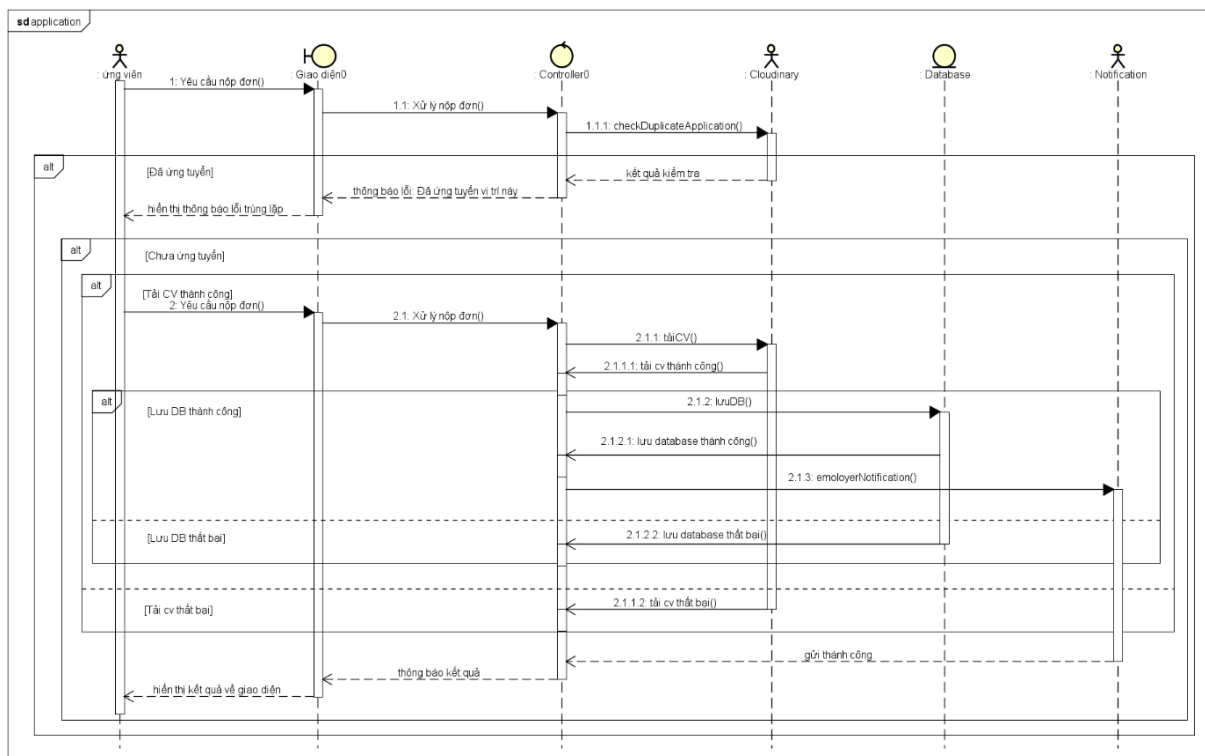
Bảng 3.3: Đặc tả Use case Tổng quan quy trình tuyển dụng

3.4. Thiết kế hệ thống

3.4.1. Sơ đồ tuần tự (Sequence diagram) và Sơ đồ trạng thái (State diagram)

3.4.1.1. Quy trình ứng tuyển

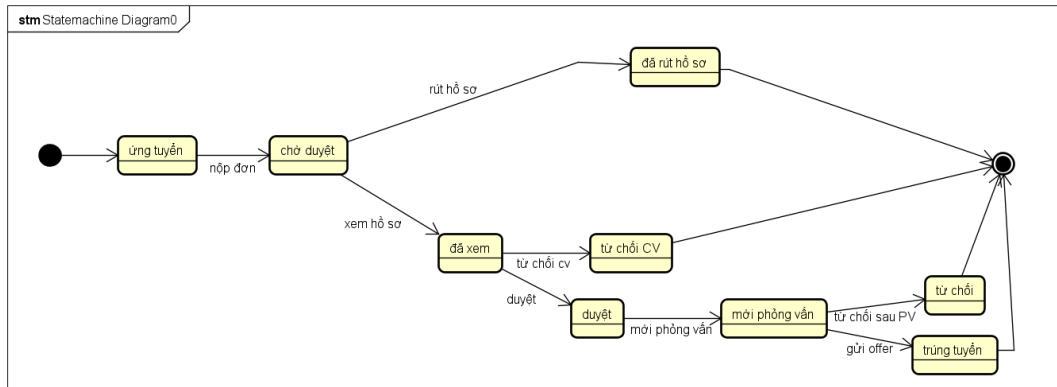
Sơ đồ này mô tả chi tiết quy trình nộp hồ sơ của ứng viên, được xử lý một cách tuần tự và chặt chẽ. Ban đầu, hệ thống thực hiện **kiểm tra trùng lặp** trong Database để xác thực tính hợp lệ. Nếu hợp lệ, quy trình tiếp tục bằng việc **tải file** CV lên Cloudinary, sau đó **lưu toàn bộ thông tin** đơn ứng tuyển vào Database, và cuối cùng là **gửi thông báo** đến nhà tuyển dụng. Sơ đồ cũng thể hiện rõ các luồng xử lý lỗi ở từng giai đoạn quan trọng như tải file hoặc lưu trữ thất bại, đảm bảo mọi kết quả đều được phản hồi về cho người dùng.



Hình 3.4: Sơ đồ quy trình ứng tuyển

Để hiểu được rõ hơn quy trình trên, sơ đồ trạng thái này mô tả vòng đời của một hồ sơ ứng tuyển, bắt đầu từ trạng thái **chờ duyệt** sau khi ứng viên nộp đơn thành công. Từ đây, hồ sơ có thể được nhà tuyển dụng **xem**, và nếu qua được vòng lọc CV, sẽ chuyển sang trạng thái **mời phỏng vấn**. Sau buổi phỏng vấn, hồ sơ sẽ có kết quả cuối cùng là **trúng tuyển** hoặc **từ chối**. Sơ đồ cũng thể hiện các trường hợp kết thúc sớm hơn, như

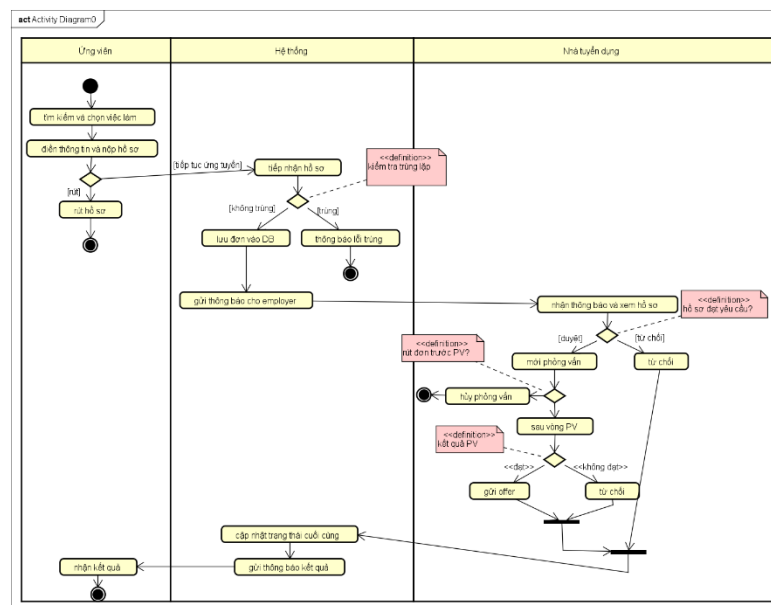
bị **từ chối CV** ngay sau khi xem, hoặc ứng viên chủ động **rút hồ sơ** ở bất kỳ giai đoạn nào trước khi có kết quả cuối cùng.



Hình 3.5: Sơ đồ trạng thái ứng tuyển

3.4.2. Sơ đồ hoạt động (Activity diagram)

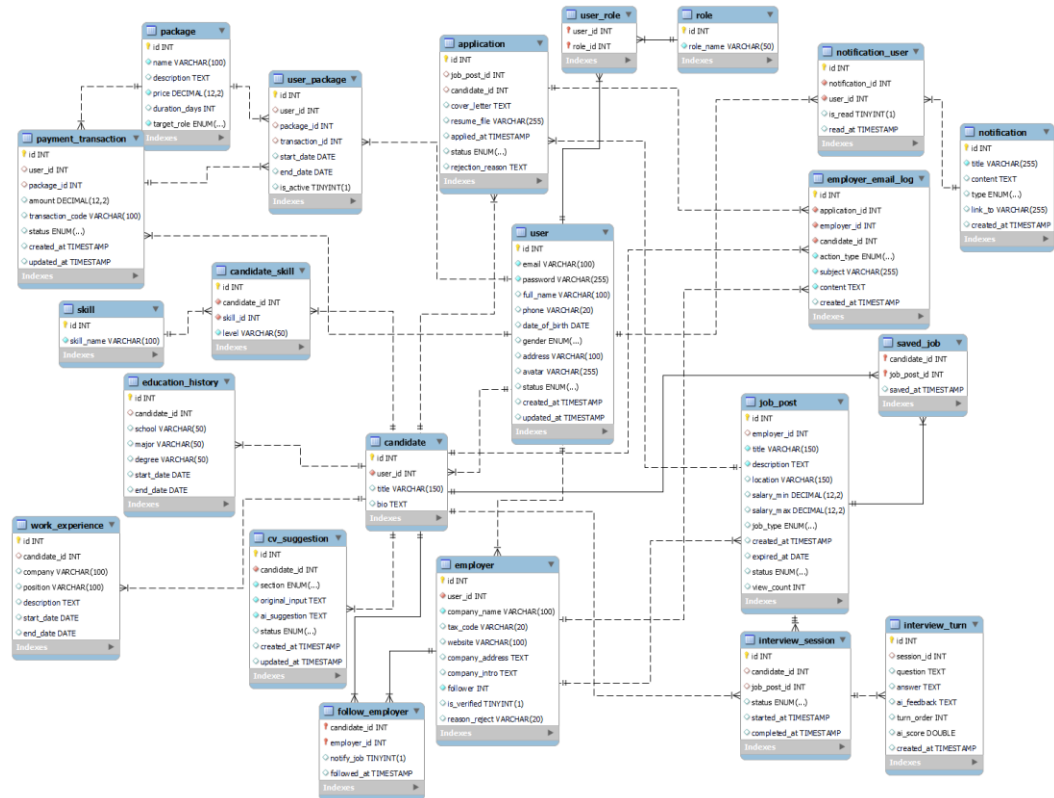
Sơ đồ hoạt động này mô tả chi tiết quy trình ứng tuyển từ đầu đến cuối, phân chia rõ trách nhiệm giữa ba bên: **Ứng viên**, **Hệ thống**, và **Nhà tuyển dụng**. Luồng chính bắt đầu khi Ứng viên nộp hồ sơ, sau đó Hệ thống sẽ kiểm tra tính hợp lệ (trùng lặp), lưu trữ và gửi thông báo cho Nhà tuyển dụng. Nhà tuyển dụng sẽ thực hiện các bước sàng lọc, phỏng vấn và ra quyết định cuối cùng. Sơ đồ cũng thể hiện rõ các luồng phụ quan trọng như ứng viên tự rút hồ sơ, nhà tuyển dụng từ chối ở vòng CV hoặc sau phỏng vấn, đảm bảo mọi trường hợp đều được xử lý trước khi Hệ thống gửi thông báo kết quả cuối cùng cho Ứng viên.



Hình 3.6: Sơ đồ hoạt động quy trình ứng tuyển

3.4.3. Thiết kế cơ sở dữ liệu

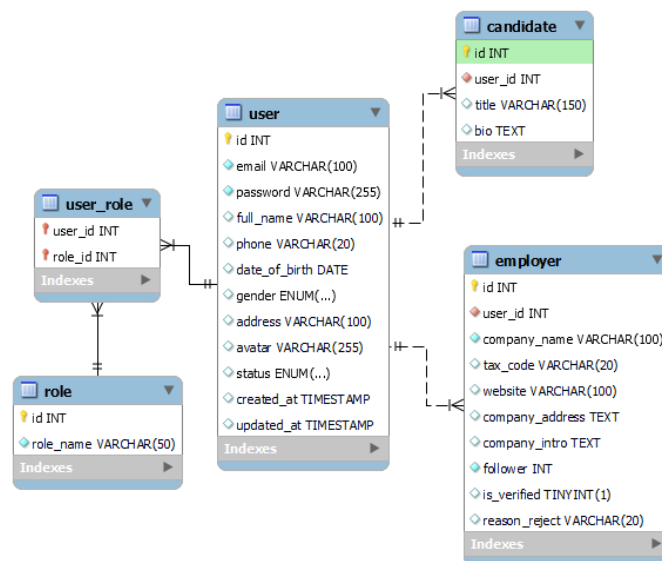
3.4.3.1. Lược đồ cơ sở dữ liệu toàn bộ hệ thống



Hình 3.7: Lược đồ cơ sở dữ liệu toàn bộ hệ thống

3.4.3.2. Vai trò từng bảng

(a) Lược đồ người dùng và hồ sơ người dùng



Hình 3.8: Lược đồ người dùng và hồ sơ người dùng

Role: Lưu các vai trò gốc của hệ thống (ADMIN, EMPLOYER, CANDIDATE). Hoạt động như một danh mục quyền hạn.

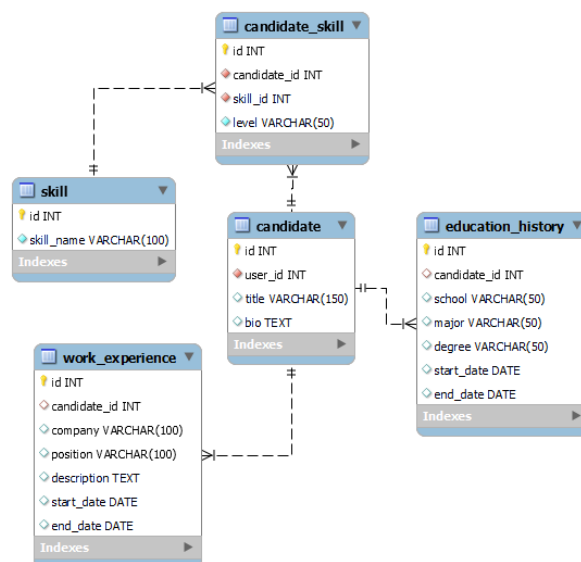
User: Bảng trung tâm chứa thông tin xác thực (email, password) và dữ liệu cá nhân cơ bản (full_name, avatar) cho mọi tài khoản.

UserRole: Bảng trung gian thực thi quan hệ N-N giữa **User** và **Role**, cho phép một người dùng có thể đảm nhiệm nhiều vai trò cùng lúc.

Candidate: Bảng mở rộng của **User**, lưu trữ thông tin chuyên môn chỉ ứng viên mới có như chức danh (title) và tóm tắt bản thân (bio).

Employer: Bảng mở rộng của **User**, lưu trữ thông tin định danh chỉ nhà tuyển dụng mới có như tên công ty (company_name) và mã số thuế (tax_code).

(b) Lược đồ hồ sơ ứng viên



Hình 3.9: Lược đồ hồ sơ ứng viên

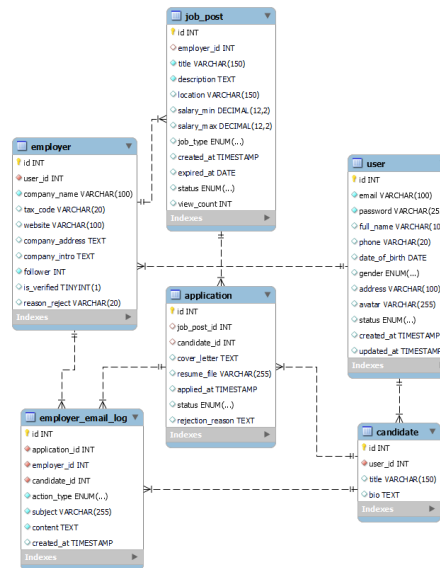
Skill: Bảng danh mục chứa tên tất cả các kỹ năng chuẩn (ví dụ: 'Java', 'Marketing', 'Figma').

CandidateSkill: Bảng trung gian kết nối **Candidate** với **Skill**, bổ sung thêm thuộc tính về cấp độ (level) cho từng kỹ năng của ứng viên.

EducationHistory: Ghi lại lịch sử học vấn của ứng viên, bao gồm trường, ngành học, và bằng cấp.

WorkExperience: Ghi lại lịch sử kinh nghiệm làm việc của ứng viên, bao gồm công ty, vị trí và mô tả công việc.

(c) Lược đồ quy trình ứng tuyển và giao tiếp



Hình 3.10: Lược đồ quy trình ứng tuyển và giao tiếp

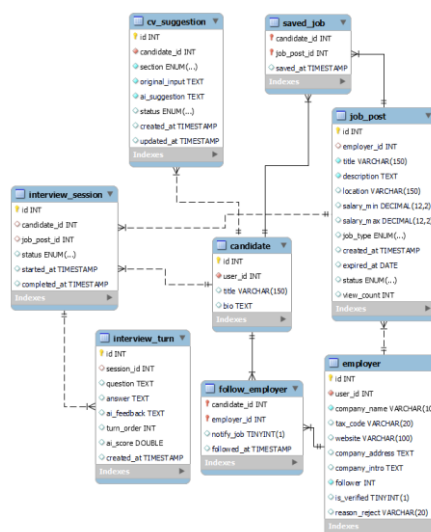
JobPost: Chứa toàn bộ nội dung một tin tuyển dụng do **Employer** đăng tải.

Application: Ghi lại một lượt ứng tuyển, kết nối một **Candidate** với một **JobPost**.

Đây là bảng lỗi của quy trình, lưu CV, thư xin việc và trạng thái hồ sơ.

EmployerEmailLog: Hoạt động như một nhật ký, lưu lại tất cả các email quan trọng mà **Employer** gửi cho **Candidate** liên quan đến một **Application** cụ thể.

(d) Lược đồ tính năng AI và tương tác



Hình 3.11: Lược đồ tính năng AI và tương tác

CvSuggestion: Lưu các gợi ý do AI tạo ra để giúp **Candidate** cải thiện nội dung CV theo từng phần cụ thể.

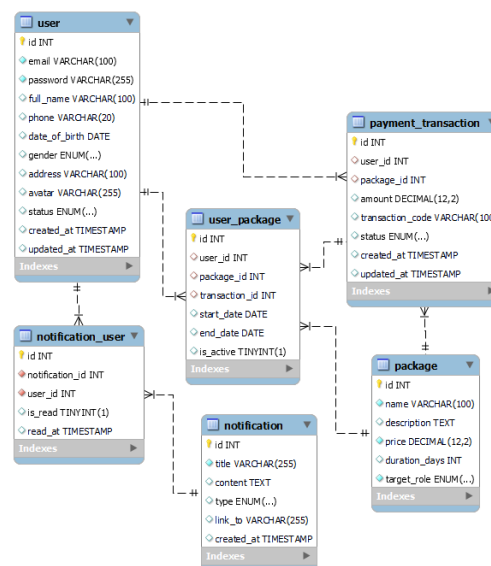
FollowEmployer: Ghi lại hành động một **Candidate** theo dõi một Employer để nhận cập nhật.

SavedJob: Ghi lại hành động một **Candidate** lưu một JobPost để xem lại sau.

InterviewSession: Ghi lại một phiên phỏng vấn thử với AI của **Candidate** cho một **JobPost**.

InterviewTurn: Ghi lại chi tiết từng lượt hỏi và đáp trong một **InterviewSession**, bao gồm câu hỏi, câu trả lời, cuối cùng là nhận phản hồi và điểm số từ AI.

(e) Lược đồ dịch vụ thanh toán và thông báo



Hình 3.12: Lược đồ dịch vụ thanh toán và thông báo

Package: Định nghĩa các gói dịch vụ trả phí (ví dụ: Gói AI Premium) mà hệ thống cung cấp cho người dùng.

PaymentTransaction: Ghi lại lịch sử mỗi giao dịch thanh toán khi người dùng mua một **Package**.

UserPackage: Kích hoạt và quản lý gói dịch vụ mà **User** đã mua, bao gồm thời hạn hiệu lực.

Notification: Chứa nội dung của một thông báo (ví dụ: "Bạn có tin nhắn mới").

NotificationUser: Gửi một **Notification** đến một **User** cụ thể và theo dõi trạng thái đã đọc/chưa đọc.

3.4.3.3. Mối quan hệ giữa các bảng

1. Mối quan hệ giữa các bảng User 1-N UserRole, Role 1-N UserRole:

Trong hệ thống, cần có một cơ chế phân quyền để quản lý các nhóm người dùng khác nhau như Quản trị viên, Nhà tuyển dụng và Ứng viên. Một tài khoản (**User**) có thể đóng nhiều vai trò cùng lúc, ví dụ user có thể vừa tìm việc (**CANDIDATE**) vừa có thể đăng tin tuyển dụng (**EMPLOYER**). Ngược lại, một vai trò (**Role**) sẽ được gán cho nhiều tài khoản người dùng khác nhau. Để giải quyết mối quan hệ **nhiều-nhiều** này, bảng **UserRole** được tạo ra làm bảng trung gian, mỗi dòng trong bảng này sẽ gán một **Role** cụ thể cho một **User** cụ thể.

2. Mối quan hệ giữa bảng User 1-1 Candidate, User 1-1 Employer:

Hệ thống cần một nơi để lưu thông tin đăng nhập chung (**User**), đồng thời phải có những thông tin chuyên biệt chỉ thuộc về Nhà tuyển dụng (**Employer**) hoặc Ứng viên (**Candidate**). Khi một người dùng đăng ký, họ sẽ được lưu vào bảng User. Tùy theo nơi đăng ký, vai trò sẽ được gán vào cơ sở dữ liệu. Mối quan hệ giữa **User** với **Candidate** và **Employer** là **một-một**, vì mỗi tài khoản người dùng chỉ có thể ứng với một hồ sơ chi tiết duy nhất, đảm bảo tính toàn vẹn và không trùng lặp dữ liệu.

3. Mối quan hệ giữa bảng Candidate 1-N Application, JobPost 1-N Application:

Chức năng cốt lõi của hệ thống là cho phép ứng viên nộp hồ sơ vào các tin tuyển dụng và cần có một cấu trúc để ghi nhận chính xác hành động này. Một ứng viên (**Candidate**) có thể gửi hồ sơ đến nhiều tin tuyển dụng (**JobPost**) khác nhau trong quá trình tìm việc. Đồng thời, một tin tuyển dụng cũng sẽ nhận được hồ sơ từ nhiều ứng viên. Để quản lý mối quan hệ **nhiều-nhiều** này, bảng **Application** được tạo ra. Mỗi bản ghi trong **Application** đại diện cho **một lần** nộp hồ sơ duy nhất, lưu trữ các thông tin liên quan (CV, thư xin việc và trạng thái) đồng thời kết nối một **Candidate** cụ thể với một **JobPost** cụ thể.

4. Mối quan hệ giữa bảng Employer 1-N JobPost:

Hệ thống phải cho phép nhà tuyển dụng quản lý các tin đăng của riêng mình. Khi một nhà tuyển dụng (**Employer**) muốn tìm kiếm nhân sự, họ sẽ tạo ra một hoặc nhiều tin

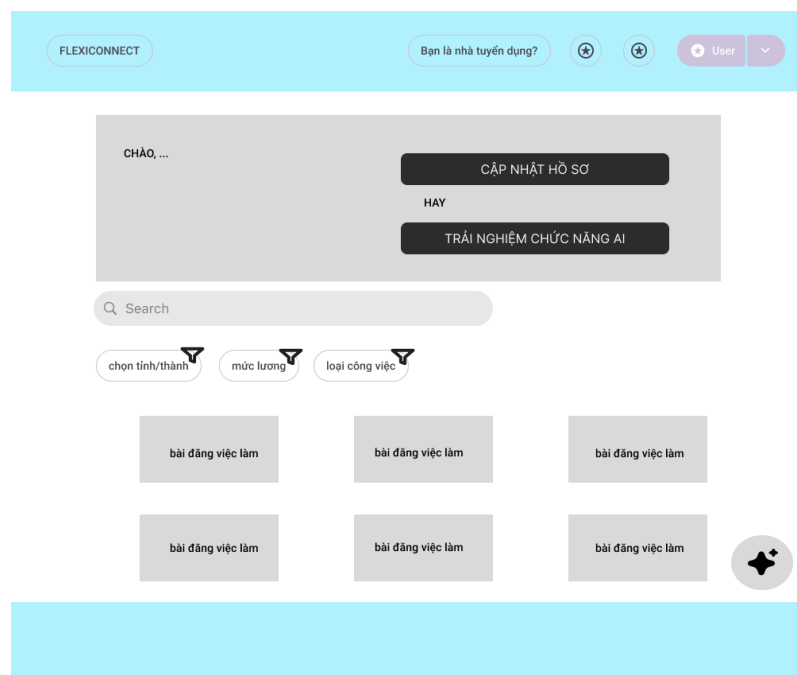
tuyển dụng (**JobPost**) trên hệ thống. Mỗi tin tuyển dụng được tạo ra chỉ có thể thuộc về một nhà tuyển dụng duy nhất để đảm bảo quyền sở hữu và quản lý. Do đó, mối quan hệ giữa **Employer** và **JobPost** là **một-nhiều**.

5. Mối quan hệ giữa các bảng Notification 1-N NotificationUser, User 1-N NotificationUser:

Khi có một sự kiện quan trọng xảy ra (ví dụ: ứng viên nộp đơn, có tin nhắn mới), hệ thống cần thông báo cho những người dùng liên quan. Một thông báo (Notification) có thể cần được gửi đến nhiều người dùng (User). Để quản lý việc này, bảng NotificationUser được tạo ra làm bảng trung gian. Bảng này không chỉ liên kết một thông báo đến một người dùng cụ thể mà còn lưu trữ trạng thái riêng của thông báo đó đối với người dùng đó (ví dụ: đã đọc hay chưa đọc).

3.4.4. Thiết kế giao diện

3.4.4.1. Thiết kế giao diện trang tìm việc



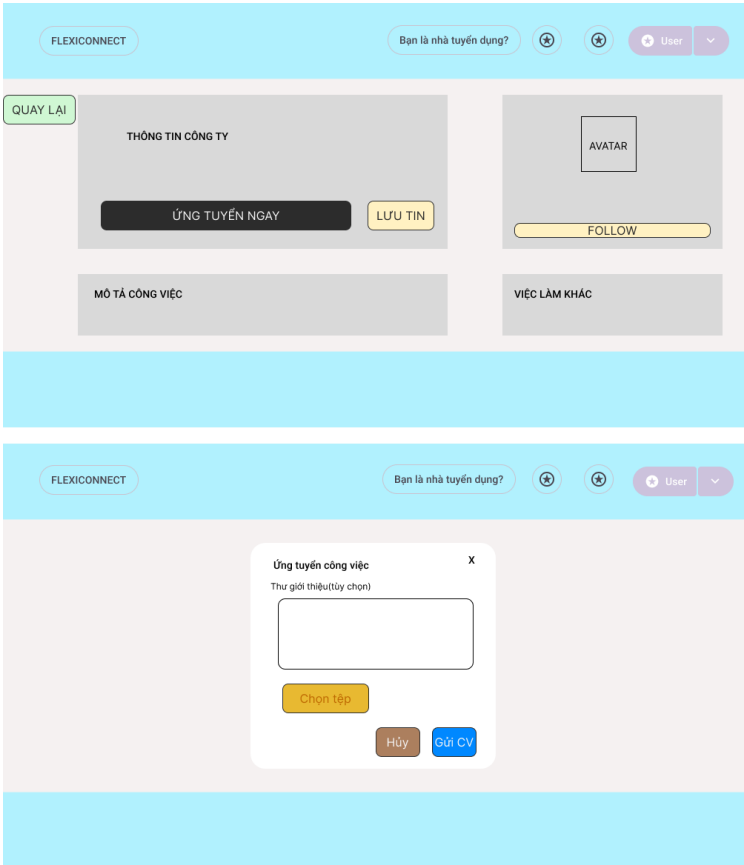
Hình 3.13: Giao diện trang tìm việc

3.4.4.2. Thiết kế xử lý cho trang tìm việc

STT	Tên xử lý	Điều kiện	Ý nghĩa
1	btnDashboard()	Khi người dùng nhấp vào	Chuyển hướng về trang chủ

2	btnRegisterEmployer()	Khi người dùng nhấp vào	Chuyển hướng đến trang đăng ký cho nhà tuyển dụng
3	btnNoti()	Khi người dùng nhấp vào	Mở dialog giao diện thông báo
4	btnSwitchTheme ()	Khi người dùng nhấp vào	Đổi màu chủ đề toàn bộ giao diện
5	btnProfileDropdown()	Khi người dùng nhấp vào	Hiện danh sách tùy chọn chức năng của ứng viên
6	btnUpdateProfile()	Khi người dùng nhấp vào	Chuyển đến trang quản lý hồ sơ cá nhân
7	btnUpgradeAccount()	Khi người dùng nhấp vào	Chuyển đến trang nâng cấp tài khoản
8	searchBar()	Khi người dùng nhấp vào	Điền thông tin tìm kiếm
9	provinceFilters()	Khi người dùng nhấp vào	Hiện thị danh sách chọn lọc tỉnh/thành
10	salaryFilters()	Khi người dùng nhấp vào	Hiện thị danh sách chọn lọc mức lương
11	jobTypeFilters()	Khi người dùng nhấp vào	Hiện thị danh sách chọn lọc loại việc làm
12	jobPostCard()	Khi người dùng nhấp vào	Chuyển đến trang chi tiết của việc làm đó
13	btnAIFunction()	Khi người dùng nhấp vào	Hiện thị lựa chọn chức năng của AI

3.4.4.3. Thiết kế giao diện chức năng ứng tuyển



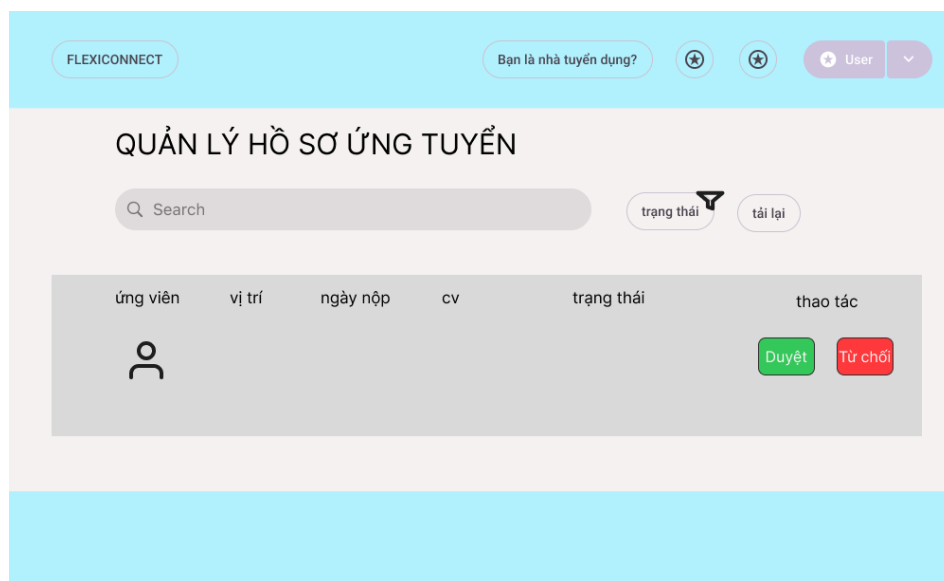
Hình 3.14: Giao diện chức năng ứng tuyển

3.4.4.4. Thiết kế xử lý cho chức năng ứng tuyển

STT	Tên xử lý	Điều kiện	Ý nghĩa
1	btnDashboard()	Khi người dùng nhấp vào	Chuyển hướng về trang chủ
2	btnRegisterEmployer()	Khi người dùng nhấp vào	Chuyển hướng đến trang đăng ký cho nhà tuyển dụng
3	btnNoti()	Khi người dùng nhấp vào	Mở dialog giao diện thông báo
4	btnSwitchTheme ()	Khi người dùng nhấp vào	Đổi màu chủ đề toàn bộ giao diện
5	btnProfileDropdown()	Khi người dùng nhấp vào	Hiện danh sách tùy chọn chức năng của ứng viên
6	btnBack()	Khi người dùng nhấp vào	Quay lại trang trước đó

7	btnApplyDialog()	Khi người dùng nhấp vào	Mở dialog trang gửi CV
8	btnSavedJob()	Khi người dùng nhấp vào	Lưu tin tuyển dụng
9	btnFollow()	Khi người dùng nhấp vào	Theo dõi nhà tuyển dụng
10	inputCoverLetter()	Khi người dùng nhấp vào	Nhập dữ liệu thư xin việc
11	inputFile()	Khi người dùng nhấp vào	Mở cửa sổ hộp thoại chọn tệp của hệ điều hành để chọn CV cần gửi
12	btnCancel()	Khi người dùng nhấp vào	Đóng cửa sổ dialog
13	btnClose()	Khi người dùng nhấp vào	Đóng cửa sổ dialog
14	btnApply()	Khi người dùng nhấp vào	Xác nhận ứng tuyển

3.4.4.5. Thiết kế giao diện trang xét tuyển ứng viên



Hình 3.15: Giao diện trang xét duyệt ứng viên

3.4.4.6. Thiết kế xử lý cho trang xét tuyển ứng viên

STT	Tên xử lý	Điều kiện	Ý nghĩa
1	btnDashboard()	Khi người dùng nhấp vào	Chuyển hướng về trang chủ
2	btnRegisterEmployer()	Khi người dùng nhấp vào	Chuyển hướng đến trang đăng ký cho nhà tuyển dụng
3	btnNoti()	Khi người dùng nhấp vào	Mở dialog giao diện thông báo
4	btnSwitchTheme ()	Khi người dùng nhấp vào	Đổi màu chủ đề toàn bộ giao diện
5	btnProfileDropdown()	Khi người dùng nhấp vào	Hiện danh sách tùy chọn chức năng của ứng viên
6	btnUpdateProifile()	Khi người dùng nhấp vào	Chuyển đến trang quản lý hồ sơ cá nhân
7	btnUpgradeAccount()	Khi người dùng nhấp vào	Chuyển đến trang nâng cấp tài khoản
8	searchBar()	Khi người dùng nhấp vào	Điền thông tin tìm kiếm
9	statusFilters()	Khi người dùng nhấp vào	Hiện thị danh sách chọn lọc trạng thái hồ sơ
10	btnRefresh()	Khi người dùng nhấp vào	Tải lại trang
11	listCandidate ()	Hiện thị	Hiện thị danh sách ứng viên đã ứng tuyển
12	btnAccept()	Khi người dùng nhấp vào	Duyệt ứng viên đã ứng tuyển
13	btnRefuse()	Khi người dùng nhấp vào	Từ chối ứng viên đã ứng tuyển

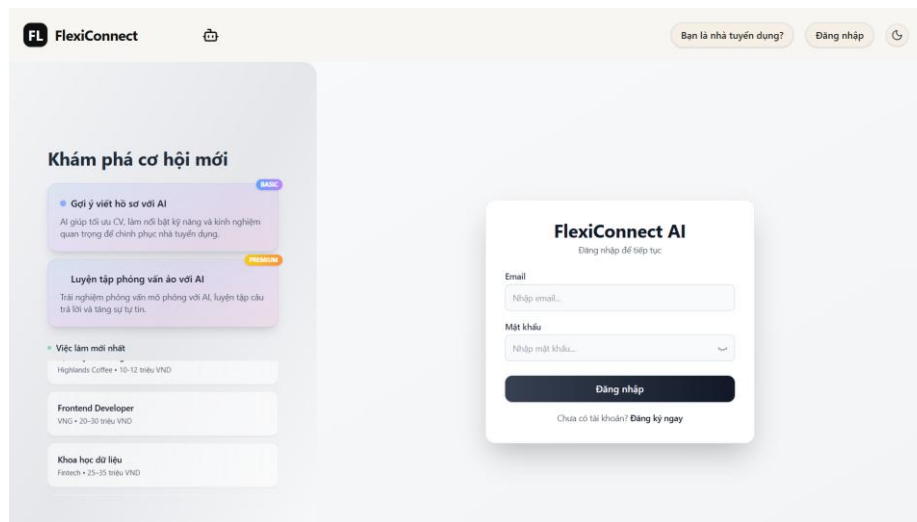
Chương 4. KẾT QUẢ ĐẠT ĐƯỢC

4.1.1. Đăng ký, đăng nhập vào hệ thống

Hệ thống FlexiConnect được thiết kế để phục vụ hai đối tượng người dùng chính là **Ứng viên (Candidate)** và **Nhà tuyển dụng (Employer)**, cùng với vai trò **Quản trị viên (Admin)** để giám sát. Khi người dùng thực hiện đăng nhập, hệ thống sẽ tự động xác thực và dựa trên vai trò đã được phân quyền để điều hướng họ đến trang quản lý chức năng tương ứng.

Hình 4.1: Trang đăng ký dành cho nhà tuyển dụng (PUBLIC)

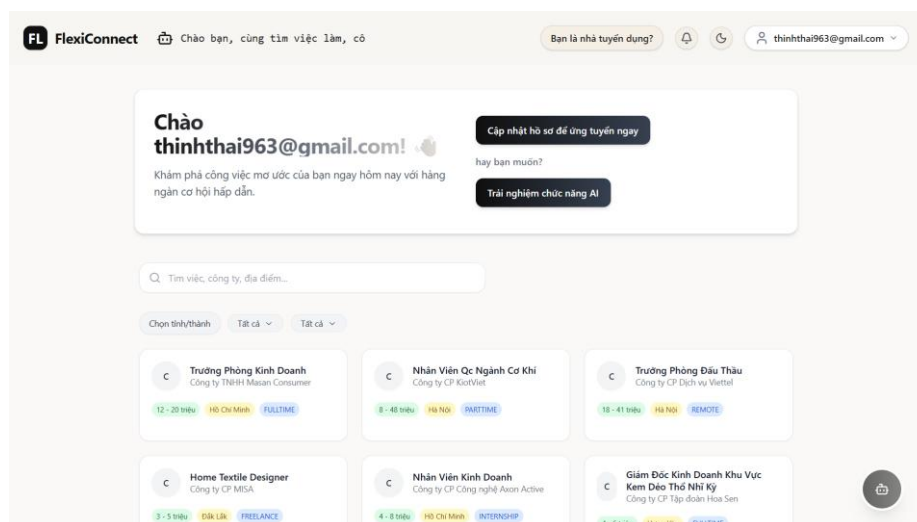
Hình 4.2: Trang đăng ký dành cho ứng viên (PUBLIC)



Hình 4.3: Trang đăng nhập (PUBLIC)

4.1.2. Ứng tuyển việc làm

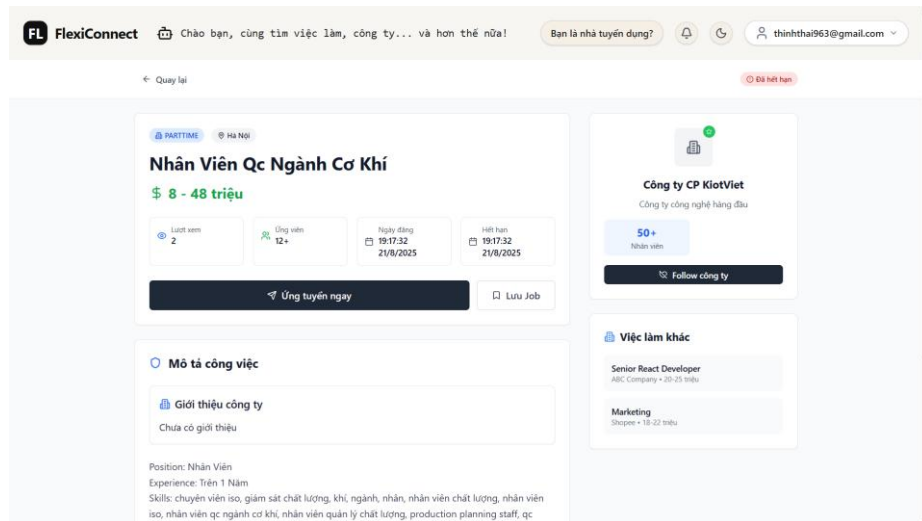
Đối với **Ứng viên**, hệ thống cung cấp một công cụ tìm kiếm cho phép lọc và tìm kiếm việc làm theo nhiều tiêu chí như kỹ năng, địa điểm, và mức lương. Khi tìm được vị trí phù hợp, ứng viên tiếp tục chọn vào từng card để tham khảo chi tiết. Một điểm đặc biệt của hệ thống là tích hợp các công cụ AI để hỗ trợ ứng viên: hệ thống có thể phân tích và gợi ý cải thiện nội dung CV, đồng thời tạo ra các phiên phỏng vấn mô phỏng để giúp họ rèn luyện kỹ năng, qua đó nâng cao cơ hội trúng tuyển.



Hình 4.4: Trang chủ của ứng viên (CANDIDATE)

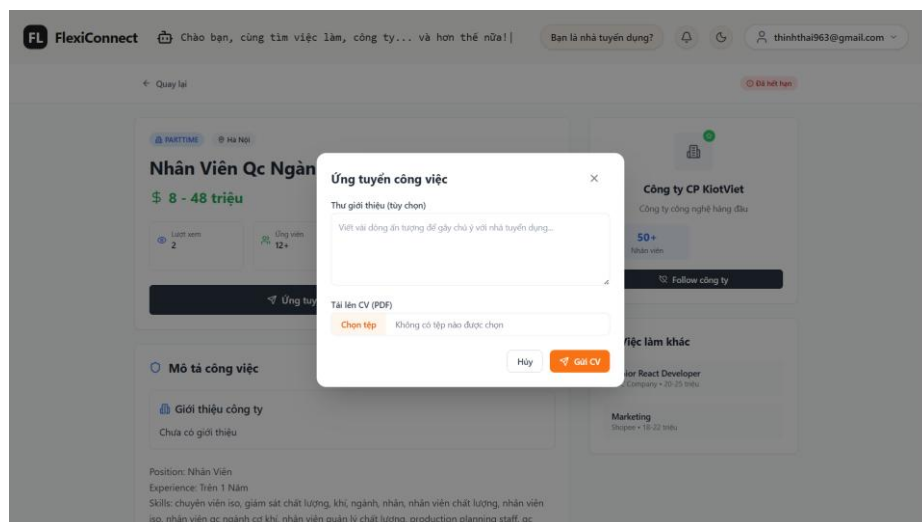
Giao diện này cung cấp thông tin chi tiết về một tin tuyển dụng cụ thể. Khi người dùng chọn một công việc từ trang tìm kiếm, hệ thống sẽ hiển thị đầy đủ các nội dung bao gồm mô tả công việc, yêu cầu, quyền lợi, và thông tin về công ty tuyển dụng. Đây là bước cung cấp dữ liệu để ứng viên đưa ra quyết định ứng tuyển. Từ trang này, hệ

thông cho phép người dùng thực hiện hai hành động chính: "**Ứng tuyển ngay**" để bắt đầu quy trình nộp hồ sơ và nếu cảm thấy công việc phù hợp nhưng chưa muốn ứng tuyển ngay có thể chọn nút "**Lưu Job**" để lưu lại tin tuyển dụng vào danh sách quan tâm. Song song đó là chức năng theo dõi nhà tuyển dụng "**Follow công ty**" để có thể nhận được thông báo việc làm phù hợp với bản thân.



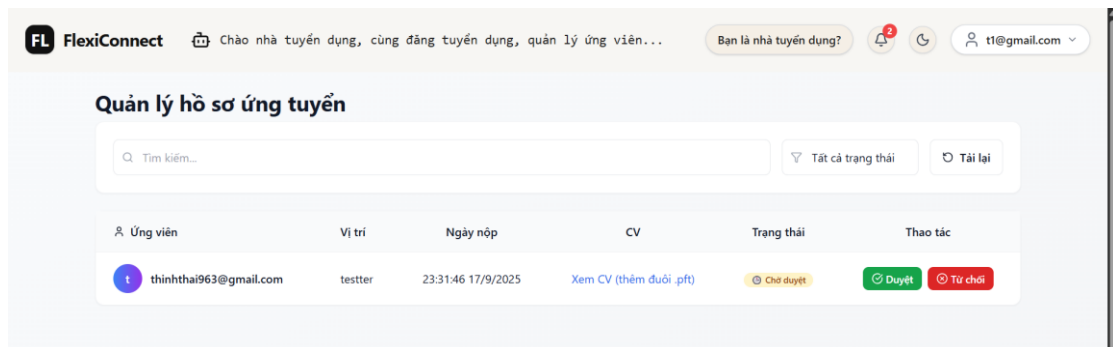
Hình 4.5: Trang chi tiết việc làm (PUBLIC)

Từ giao diện chi tiết việc làm, khi ứng viên quyết định ứng tuyển, hệ thống sẽ hiển thị một biểu mẫu cho phép họ đính kèm các thông tin cần thiết. Chức năng chính của dialog này là thu thập thư giới thiệu (cover letter) và tệp CV của ứng viên. Sau khi người dùng nhấn "**Gửi CV**", hệ thống sẽ tạo một bản ghi ứng tuyển mới, chính thức liên kết hồ sơ của ứng viên với tin tuyển dụng của nhà tuyển dụng và lưu trữ các tài liệu đi kèm.



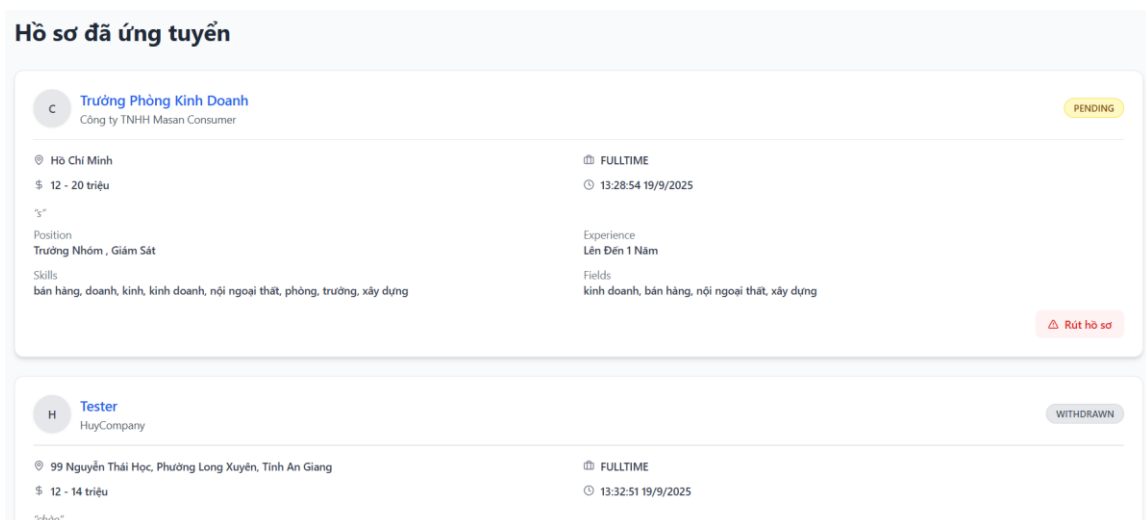
Hình 4.6: Dialog ứng tuyển (CANDIDATE)

Sau khi ứng viên đã nộp đơn ứng tuyển, nhà tuyển dụng sẽ thực hiện chức năng sàng lọc và quản lý luồng ứng viên cho một tin tuyển dụng cụ thể. Hệ thống sẽ tập hợp và hiển thị danh sách tất cả các ứng viên đã nộp hồ sơ vào vị trí này. Tại đây, nhà tuyển dụng có thể thực hiện các thao tác nghiệp vụ quan trọng như xem CV, tìm kiếm, lọc ứng viên theo trạng thái. Quan trọng nhất, hệ thống cho phép nhà tuyển dụng cập nhật trạng thái của từng hồ sơ (ví dụ: Chờ duyệt, Duyệt, Từ chối), qua đó quản lý quy trình tuyển dụng và tự động gửi thông báo tương ứng đến cho ứng viên.



Hình 4.7: Trang quản lý hồ sơ ứng tuyển (EMPLOYER)

Bên cạnh đó, ứng viên cũng có thể **rút đơn ứng tuyển** một cách dễ dàng nếu cảm thấy bản thân không còn phù hợp với việc đã ứng tuyển.



Hình 4.8: Trang theo dõi tin đã ứng tuyển (CANDIDATE)

Quản lý hồ sơ ứng tuyển						
Tìm kiếm...				Tắt cả trạng thái	Tải lại	
Ứng viên	Vị trí	Ngày nộp	CV	Trạng thái	Thao tác	
 Nghia	Tester	13:32:51 19/9/2025	Xem CV (thêm đuôi .pft)	RÚT ĐƠN	Ứng viên đã rút đơn	

Hình 4.9: Trạng thái rút đơn được cập nhật ở trang quản lý ứng tuyển (EMPLOYER)

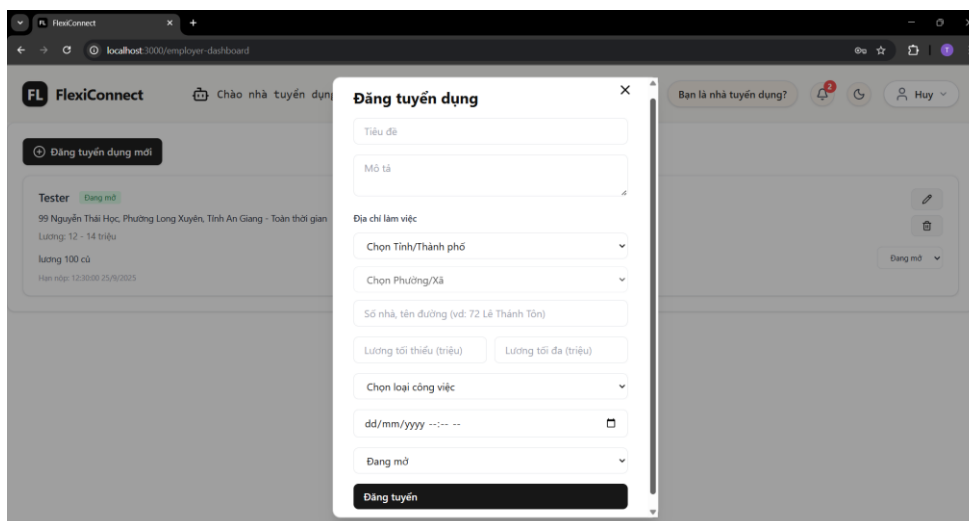
4.1.3. Chức năng đăng tin và quản lý tin

Về phía **Nhà tuyển dụng**, hệ thống cho phép họ dễ dàng tạo và đăng tải các tin tuyển dụng chi tiết. Sau khi tin được đăng, mọi hồ sơ ứng tuyển cho vị trí đó sẽ được tập trung tại một bảng điều khiển duy nhất. Các bài đăng có thể được thêm, xóa, sửa, điều chỉnh sửa trạng thái bài đăng (OPEN, CLOSED, HIDDEN), cho nhà tuyển dụng một giao diện vô cùng cơ bản giúp thao tác và quản lý vô cùng linh hoạt.



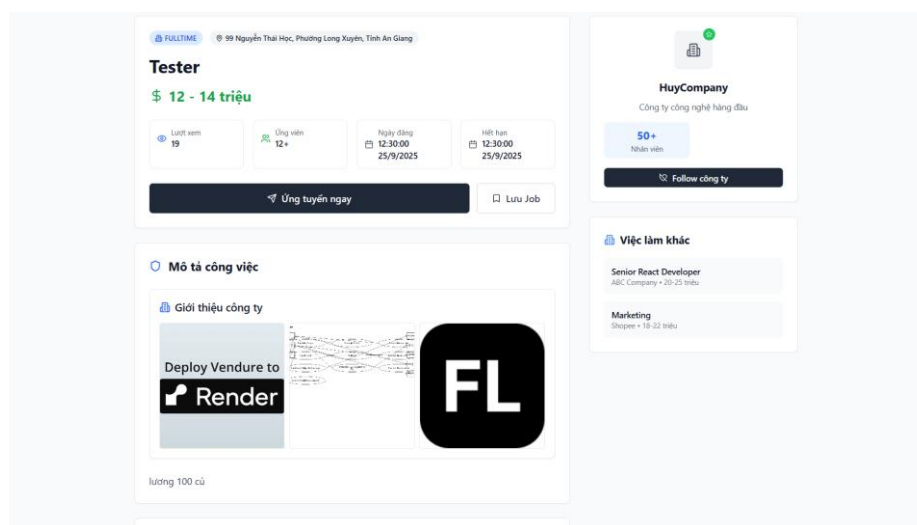
Hình 4.10: Trang chủ đăng tin của nhà tuyển dụng (EMPLOYER)

Khi ấn vào chọn đăng tuyển dụng mới, nhà tuyển dụng nhập tất cả các thông tin cần thiết của một bài đăng tuyển dụng, các thông tin về địa chỉ đã được sàng lọc qua file JSON theo *Nghị quyết 60-NQ/TW* (tháng 4/2025): Thống nhất phương án sáp nhập đơn vị hành chính cấp tỉnh và cấp xã, trong đó có việc xây dựng mô hình chính quyền địa phương 2 cấp.

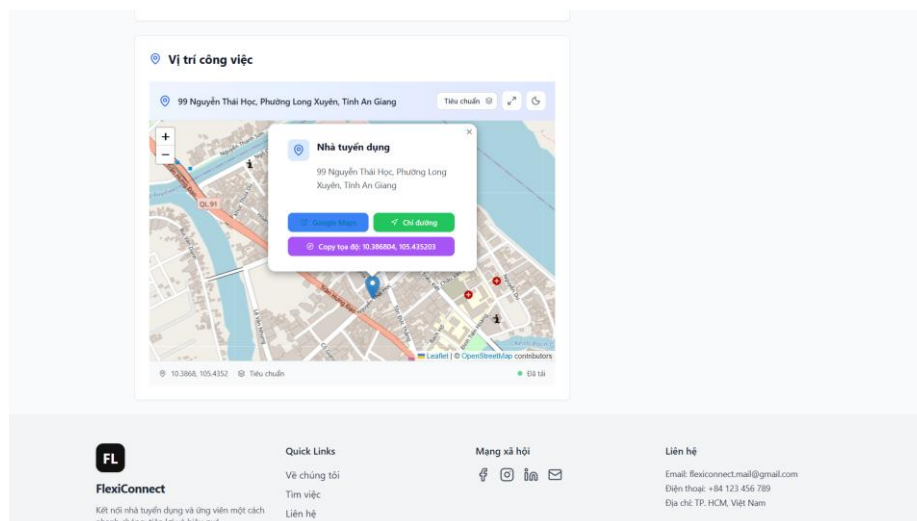


Hình 4.11: Dialog nhập thông tin bài đăng việc làm (EMPLOYER)

Sau khi đăng tin thành công, bài đăng sẽ được hiển thị công khai ở trang chủ, thông tin bài đăng cũng được hiển thị vô cùng chi tiết. Song song đó còn có tích hợp bản đồ qua combo tích maps (**Leaflet + OSM**) giúp cho ứng viên có lựa chọn sáng suốt phù hợp hơn với bản thân



Hình 4.12: Thông tin chi tiết của bài đăng việc làm (1) (PUBLIC)

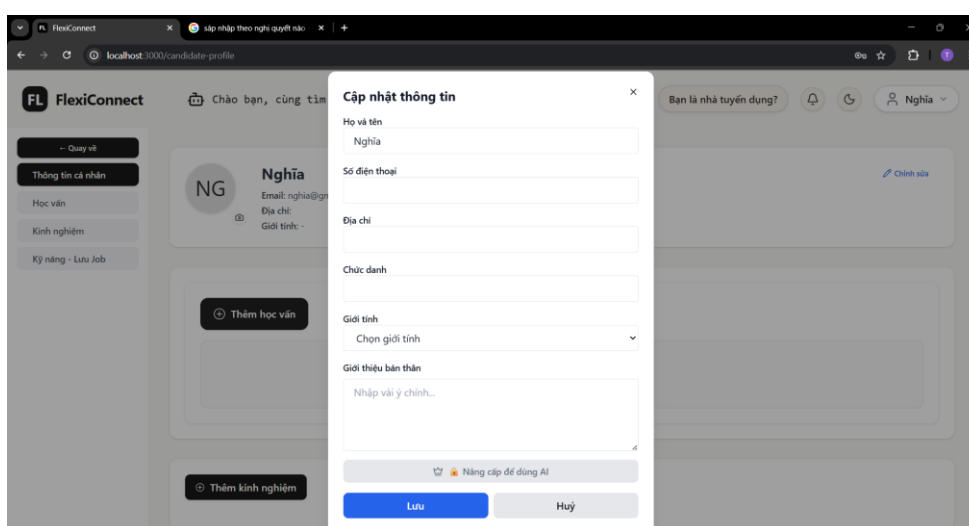


Hình 4.13: Thông tin chi tiết của bài đăng việc làm (2) (PUBLIC)

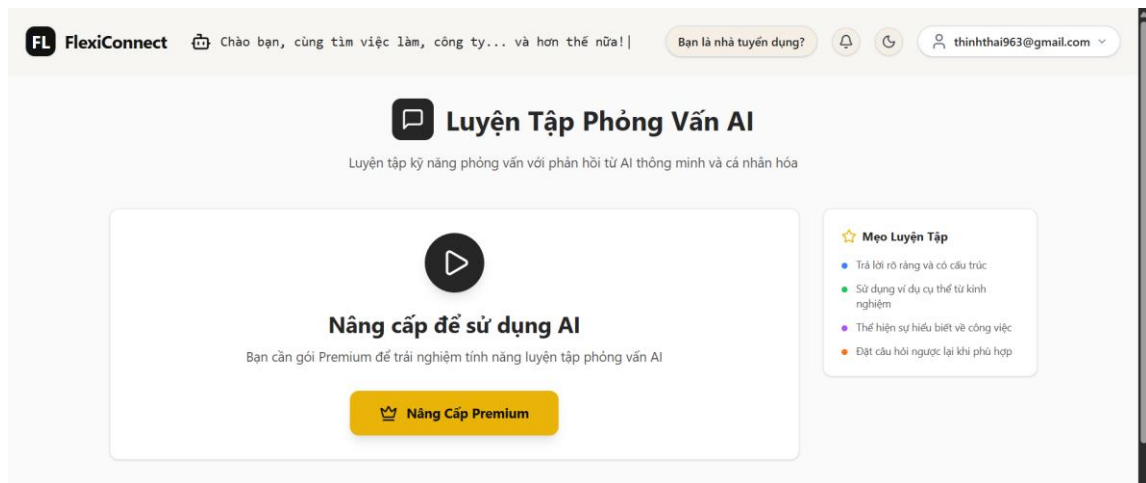
4.1.4. Các chức năng AI

Đây là giao diện vừa để **kiểm soát truy cập**, vừa để **bán gói dịch vụ** (Upsell).

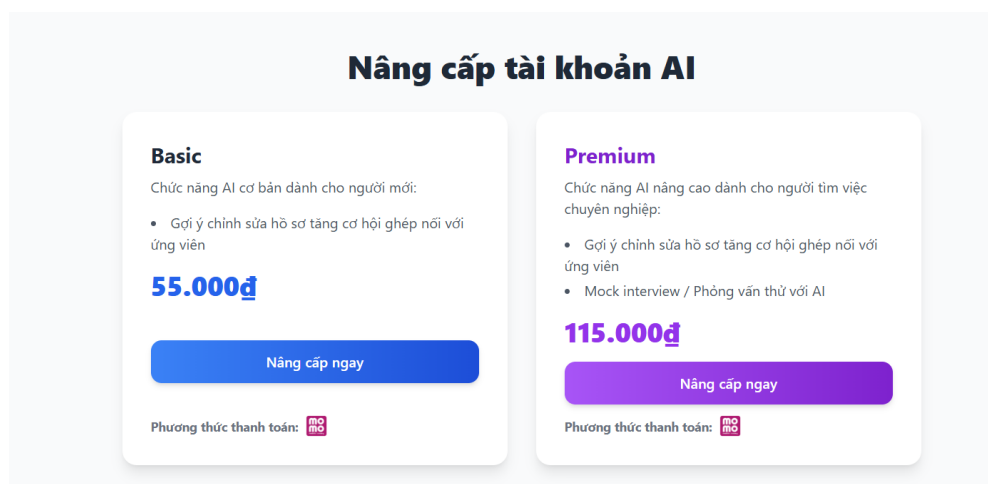
Khi một ứng viên chưa đăng ký gói (Basic, Premium) truy cập vào tính năng phỏng vấn AI, hệ thống sẽ kiểm tra quyền hạn và chặn họ dùng trực tiếp. Thay vào đó, hệ thống hiển thị một trang giới thiệu, nêu rõ lợi ích của tính năng và kèm theo nút "Nâng cấp để dùng AI" nổi bật. Mục tiêu là khuyến khích người dùng miễn phí nâng cấp để trải nghiệm đầy đủ các tính năng nâng cao. Nếu người dùng muốn nâng cấp có thể ấn vào nút nâng cấp để có thể chuyển tới trang nâng cấp tài khoản.



Hình 4.14: Chức năng cải thiện hồ sơ với AI (chưa nâng cấp tài khoản) (CANDIDATE)

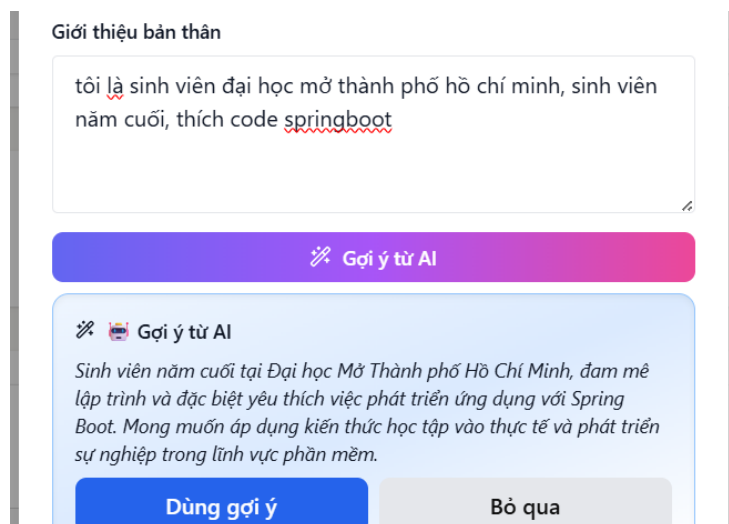


Hình 4.15: Trang luyện tập phỏng vấn AI (chưa nâng cấp tài khoản) (CANDIDATE)
 Khi người dùng muốn cảm thấy các gói nâng cấp đã phù hợp sau khi dialog tùy chọn nâng cấp xuất hiện, ứng viên sẽ tiếp tục chọn nâng cấp theo gói mình muốn thông qua ví điện tử Momo.



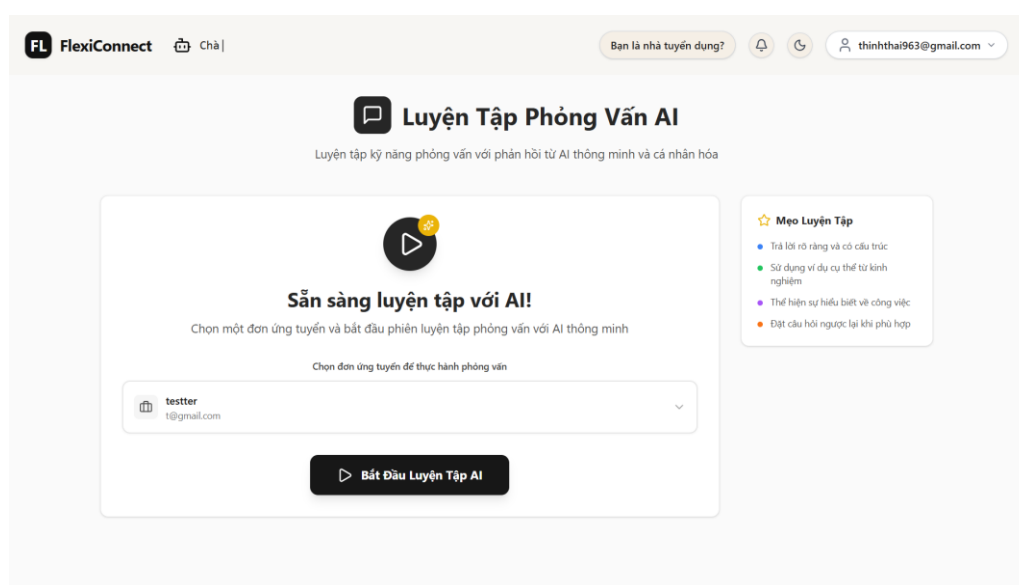
Hình 4.16: Các gói nâng cấp (CANDIDATE)

Với những ứng viên đã có tài khoản **Basic**, giao diện chức năng cải thiện hồ sơ với AI sẽ được mở khóa, người dùng chỉ cần nhập mô tả bản thân hoặc kinh nghiệm việc làm với vài câu ngắn sau đó AI sẽ generate thành một bản nâng cấp hoàn chỉnh và người dùng cũng có thể chọn “dùng gợi ý” hoặc “bỏ qua”.



Hình 4.17: Gợi ý cải thiện hồ sơ từ AI (CANDIDATE)

Với những ứng viên đã có tài khoản **Premium**, giao diện này thực hiện chức năng khởi tạo một phiên phỏng vấn. Trước khi bắt đầu, hệ thống yêu cầu người dùng phải chọn một tin tuyển dụng cụ thể mà họ đã ứng tuyển. Việc lựa chọn này là một bước quan trọng, vì hệ thống sẽ dựa vào thông tin của tin tuyển dụng đó (ví dụ: vị trí, yêu cầu kỹ năng) để AI có thể tạo ra các câu hỏi phỏng vấn phù hợp và sát với thực tế nhất. Sau khi người dùng xác nhận, hệ thống sẽ bắt đầu một phiên luyện tập mới.

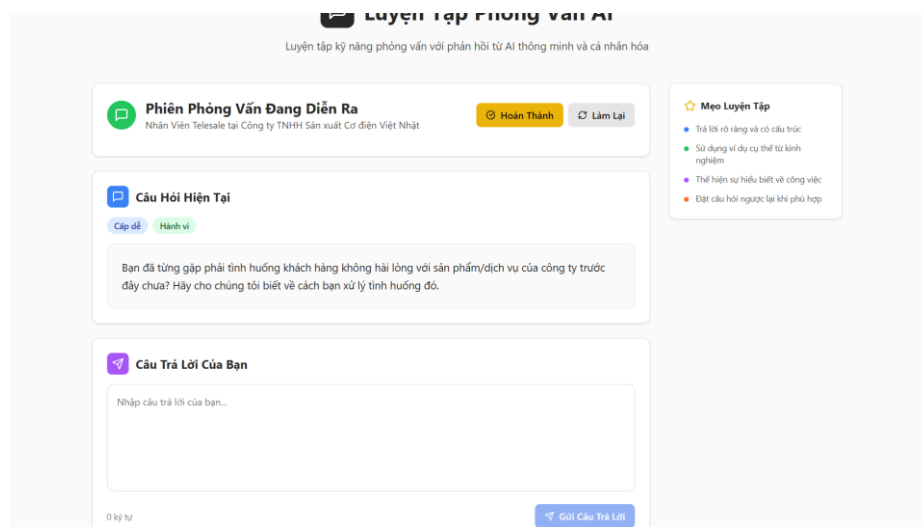


Hình 4.18: Trang luyện tập phỏng vấn AI (CANDIDATE)

Đây là giao diện chính nơi diễn ra quá trình tương tác phỏng vấn mô phỏng. Hệ thống hoạt động theo một quy trình tuần tự:

1. Hệ thống (AI) sẽ đưa ra một câu hỏi phỏng vấn tình huống hoặc chuyên môn.
2. Người dùng sẽ nhập câu trả lời của mình vào ô trả lời và gửi đi.
3. Hệ thống sẽ ghi nhận câu trả lời và (trong các bước tiếp theo) sẽ cung cấp phản hồi, đánh giá từ AI.

Quá trình này lặp lại với nhiều câu hỏi khác nhau, cho phép ứng viên thực hành và làm quen với áp lực cũng như các dạng câu hỏi thường gặp. Giao diện cũng cung cấp các nút để người dùng có thể "Hoàn Thành" phiên hoặc "Làm Lại".

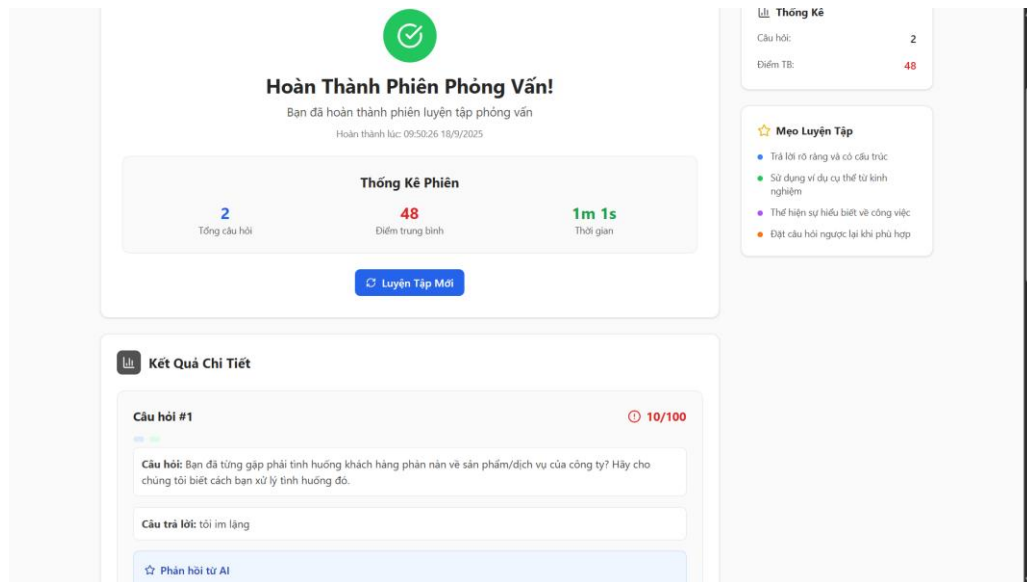


Hình 4.19: Chức năng luyện tập phỏng vấn với AI (CANDIDATE)

Đây là giao diện cuối cùng của luồng luyện tập, thực hiện chức năng tổng kết, phân tích và cung cấp phản hồi cho ứng viên sau khi họ hoàn thành phiên phỏng vấn.

Giao diện Tổng kết: Đầu tiên, hệ thống sẽ xử lý toàn bộ dữ liệu của phiên để đưa ra một bảng thống kê hiệu suất tổng quan. Chức năng này cung cấp các chỉ số như tổng số câu hỏi đã trả lời, điểm trung bình chung, và tổng thời gian thực hiện, giúp người dùng có cái nhìn bao quát về phần thể hiện của mình.

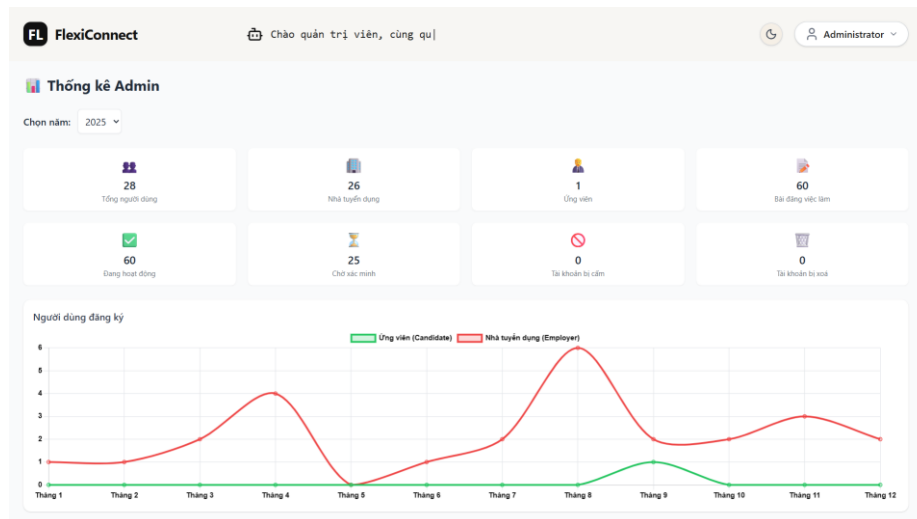
Giao diện Phân tích Chi tiết: Quan trọng hơn, hệ thống thực hiện chức năng phản hồi chi tiết cho từng câu trả lời. Với mỗi câu hỏi trong phiên, giao diện sẽ hiển thị lại câu trả lời của người dùng kèm theo một điểm số cụ thể và phần "Phản hồi từ AI". Chức năng này giúp chỉ ra những điểm mạnh, điểm yếu trong cách trả lời, phân tích cấu trúc câu và đưa ra những gợi ý cải thiện thực tế.



Hình 4.20: Trang kết quả phỏng vấn luyện tập (CANDIDATE)

4.1.5. Chức năng của quản trị viên

Đây là giao diện tổng quan, thực hiện chức năng giám sát và thống kê toàn bộ hoạt động của hệ thống. Trang này đóng vai trò là một bảng điều khiển trung tâm, nơi hệ thống tự động tổng hợp và hiển thị các chỉ số quan trọng theo thời gian thực. Các chỉ số này bao gồm tổng số người dùng, số lượng nhà tuyển dụng và ứng viên, số lượng tin tuyển dụng đang hoạt động, và các tài khoản chờ xác minh. Ngoài ra, hệ thống còn cung cấp biểu đồ trực quan hóa xu hướng đăng ký người dùng, giúp quản trị viên nắm bắt được tình hình tăng trưởng tổng thể của hệ thống.



Hình 4.21: Trang chủ của quản trị viên (ADMIN) (1)



Hình 4.22: Trang chủ của quản trị viên (ADMIN) (2)

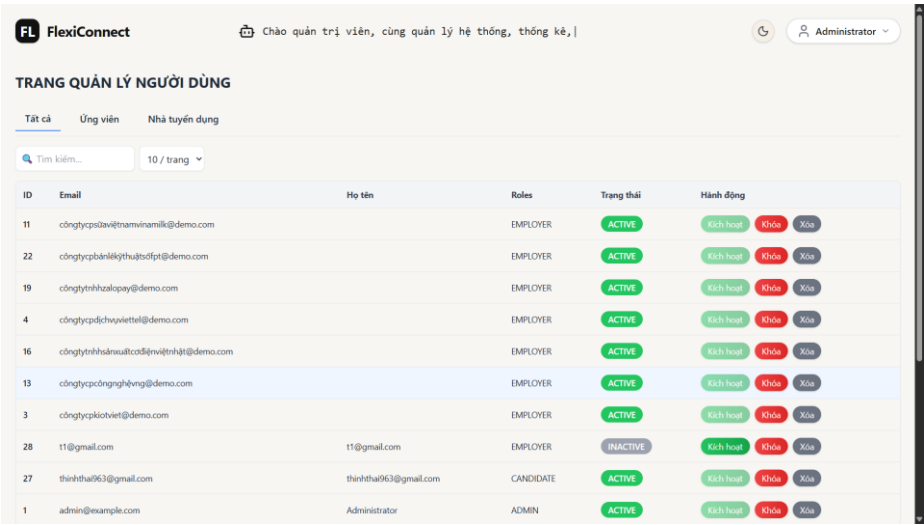
Với chức năng đầu tiên của quản trị viên, giao diện này phục vụ chức năng kiểm duyệt và quản lý chất lượng của nhà tuyển dụng. Hệ thống sẽ tự động tạo một danh sách chờ gồm tất cả các tài khoản nhà tuyển dụng mới đăng ký. Tại đây, quản trị viên thực hiện nghiệp vụ xem xét và xác thực thông tin của các công ty để đảm bảo tính hợp pháp và đáng tin cậy. Hệ thống cung cấp hai hành động cốt lõi: "Verify" (Duyệt) để cấp quyền đăng tin cho nhà tuyển dụng, và "Reject" (Từ chối). Chức năng này là một bước bảo mật quan trọng nhằm bảo vệ ứng viên và duy trì uy tín của nền tảng.

Tên công ty	Mã số thuế	Website	Địa chỉ	Chức năng
Công ty TNHH Masan Consumer				Verify Reject
Công ty CP KiotViet				Verify Reject
Công ty CP Dịch vụ Viettel				Verify Reject
Công ty CP MISA				Verify Reject
Công ty CP Công nghệ Aeon Active				Verify Reject
Công ty CP Tập đoàn Hoa Sen				Verify Reject
Công ty TNHH Thực phẩm Acecook Việt Nam				Verify Reject
Công ty CP Thế Giới Di Động				Verify Reject
Công ty CP Bảo hiểm Bảo Việt				Verify Reject
Công ty CP Sữa Việt Nam Vinamilk				Verify Reject
Công ty TNHH Shopee Việt Nam				Verify Reject

Hình 4.23: Trang duyệt nhà tuyển dụng (ADMIN)

Chức năng tiếp theo, đây là công cụ quản trị người dùng tập trung, cho phép quản trị viên thực thi các chính sách và quản lý tất cả tài khoản trên hệ thống. Giao diện hiển thị một danh sách toàn bộ người dùng không phân biệt vai trò. Chức năng chính của

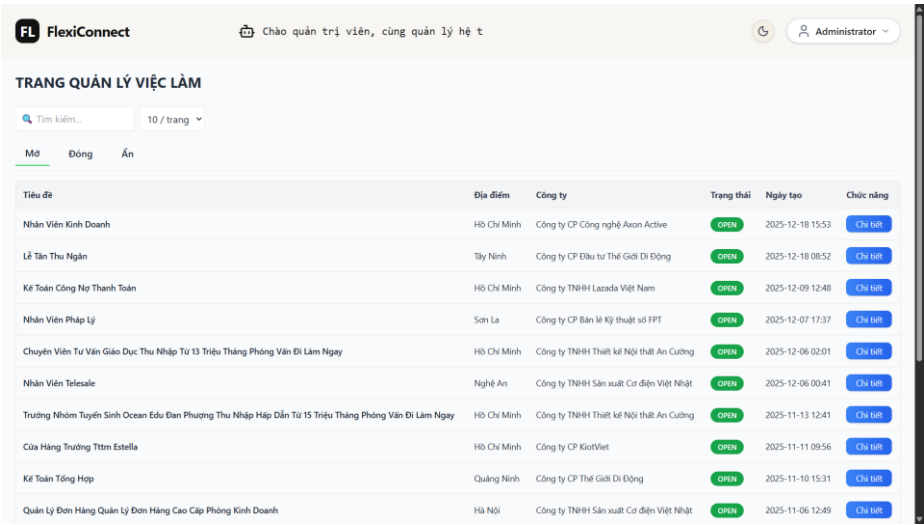
trang này là cung cấp cho quản trị viên các quyền hạn cao nhất đối với tài khoản người dùng, bao gồm: tìm kiếm, xem thông tin chi tiết, và thực hiện các hành động quản trị như Kích hoạt (Activate), Khóa (Deactivate) hoặc Xóa (Delete) một tài khoản.



ID	Email	Họ tên	Roles	Trạng thái	Hành động
11	côngtycpsdavi@namnamilk@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
22	côngtycbanlekythuat@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
19	côngtynhazalopay@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
4	côngtycdchuyenviet@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
16	côngtynhhsuats@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
13	côngtyccongnghe@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
3	côngtycpviet@demo.com		EMPLOYER	ACTIVE	Kích hoạt Khóa Xóa
28	t1@gmail.com	t1@gmail.com	EMPLOYER	INACTIVE	Kích hoạt Khóa Xóa
27	thinhthai963@gmail.com	thinhthai963@gmail.com	CANDIDATE	ACTIVE	Kích hoạt Khóa Xóa
1	admin@example.com	Administrator	ADMIN	ACTIVE	Kích hoạt Khóa Xóa

Hình 4.24: Trang quản lý người dùng (ADMIN)

Cuối cùng, chức năng này cho phép quản trị viên kiểm soát toàn bộ nội dung tin tuyển dụng trên toàn hệ thống. Khác với nhà tuyển dụng (chỉ thấy tin của mình), quản trị viên có thể xem, tìm kiếm và lọc tất cả tin đăng từ mọi công ty. Chức năng chính của trang này là kiểm duyệt nội dung. Khi phát hiện một tin đăng vi phạm chính sách hoặc có báo cáo xấu, quản trị viên có thể sử dụng các công cụ được cung cấp để ẩn hoặc xóa tin đăng đó, nhằm đảm bảo chất lượng và sự phù hợp của các cơ hội việc làm được hiển thị cho ứng viên.

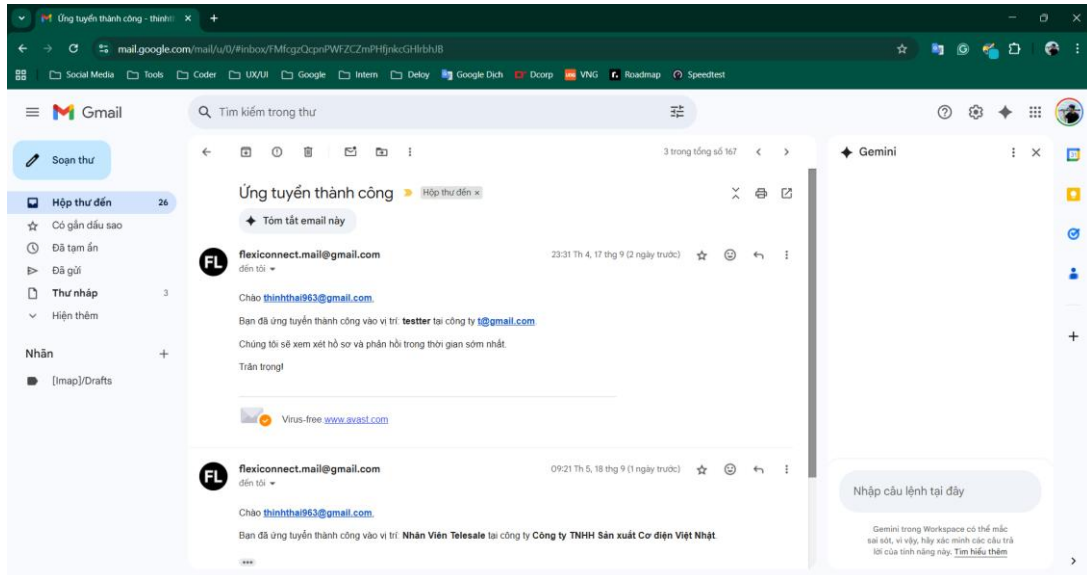


Tiêu đề	Địa điểm	Công ty	Trạng thái	Ngày tạo	Chức năng
Nhân Viên Kinh Doanh	Hồ Chí Minh	Công ty CP Công nghệ Aeon Active	OPEN	2025-12-18 15:53	Chi tiết
Lễ Tân Thu Ngân	Tây Ninh	Công ty CP Đầu tư Thế Giới Di Động	OPEN	2025-12-18 08:52	Chi tiết
Kế Toán Công Nợ Thanh Toán	Hồ Chí Minh	Công ty TNHH Lazada Việt Nam	OPEN	2025-12-09 12:48	Chi tiết
Nhân Viên Pháp Lý	Sơn La	Công ty CP Bán lẻ Kỹ thuật số FPT	OPEN	2025-12-07 17:37	Chi tiết
Chuyên Viên Tư Vấn Giáo Dục Thu Nhập Từ 13 Triệu Tháng Phỏng Vấn Di Lâm Ngay	Hồ Chí Minh	Công ty TNHH Thiết kế Nội thất An Cường	OPEN	2025-12-06 02:01	Chi tiết
Nhân Viên Telesale	Nghệ An	Công ty TNHH Sản xuất Cơ điện Việt Nhật	OPEN	2025-12-06 00:41	Chi tiết
Trưởng Nhóm Tuyển Sinh Ocean Edu Đan Phương Thu Nhập Hấp Dẫn Từ 15 Triệu Tháng Phỏng Vấn Di Lâm Ngay	Hồ Chí Minh	Công ty TNHH Thiết kế Nội thất An Cường	OPEN	2025-11-13 12:41	Chi tiết
Cửa Hàng Trưởng Tittm Estella	Hồ Chí Minh	Công ty CP KiotViet	OPEN	2025-11-11 09:56	Chi tiết
Kế Toán Tổng Hợp	Quảng Ninh	Công ty CP Thế Giới Di Động	OPEN	2025-11-10 15:31	Chi tiết
Quản Lý Đơn Hàng Quản Lý Đơn Hàng Cao Cấp Phòng Kinh Doanh	Hà Nội	Công ty TNHH Sản xuất Cơ điện Việt Nhật	OPEN	2025-11-06 12:49	Chi tiết

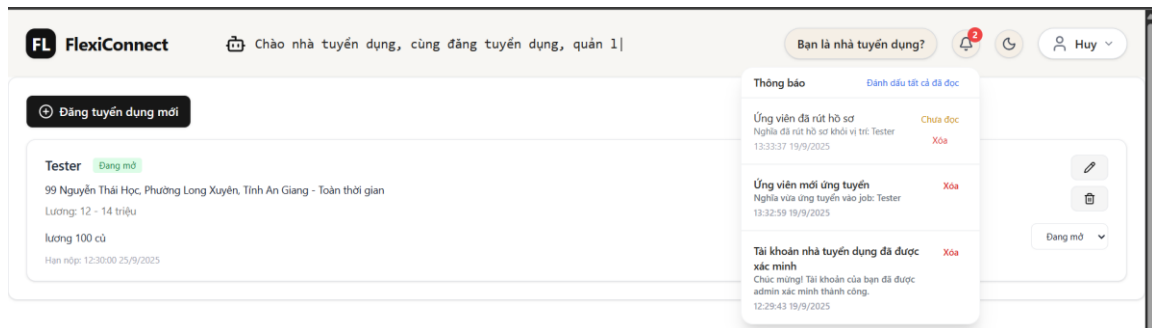
Hình 4.25: Trang quản lý tin tuyển dụng (ADMIN)

4.1.6. Các chức năng phụ

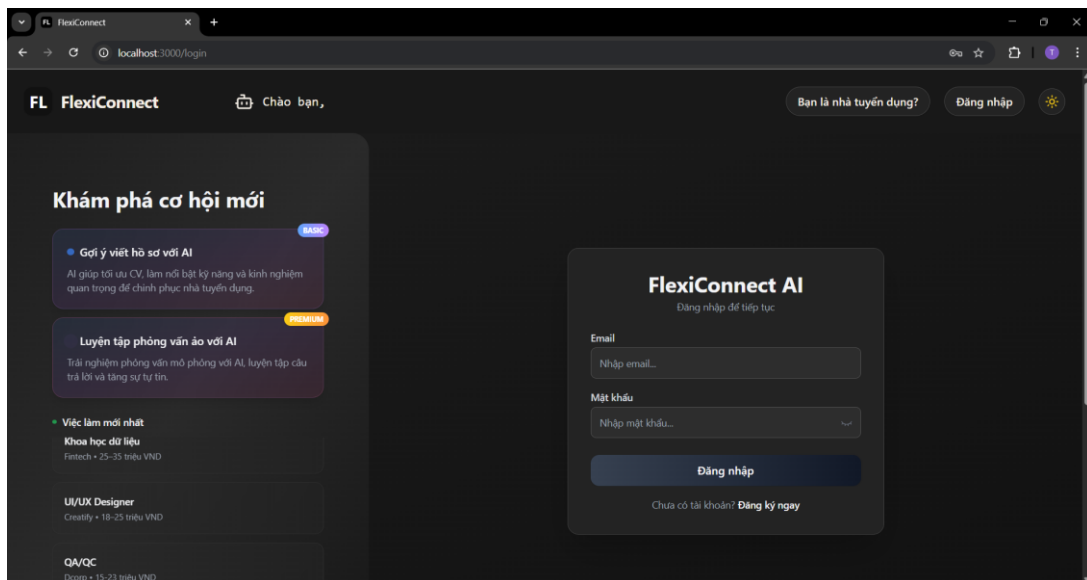
Cuối cùng, hệ thống còn cung cấp một số chức năng tuy mang tính “phụ” nhưng lại cực kỳ hữu ích cho người dùng. Những chức năng này không chỉ tăng tính đa dụng và linh hoạt mà còn cải thiện trải nghiệm giao diện, giúp hệ thống dễ nhìn và thân thiện với đa số đối tượng người dùng hiện nay.



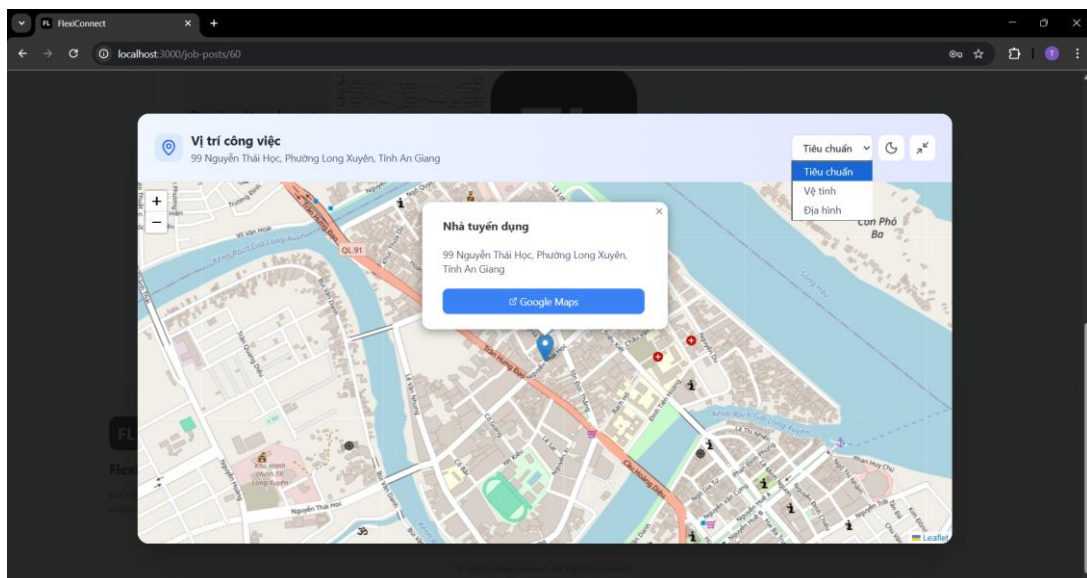
Hình 4.26: Nhận thông báo Gmail từ hệ thống



Hình 4.27: Chức năng thông báo



Hình 4.28: Đổi chủ đề cho toàn hệ thống (LightMode/DarkMode)



Hình 4.29: Bản đồ địa chỉ công ty bài đăng việc làm

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong suốt quá trình thực hiện đồ án, mục tiêu trọng tâm được đặt ra là xây dựng một nền tảng tuyển dụng thông minh, không chỉ kết nối nhà tuyển dụng và ứng viên mà còn tích hợp các công nghệ tiên tiến để nâng cao giá trị cho người dùng. Sau thời gian nghiên cứu và phát triển, dự án "FlexiConnect" đã được hoàn thiện ở phiên bản đầu tiên, đáp ứng đầy đủ các mục tiêu đề ra. Về mặt công nghệ, hệ thống có phần backend được xây dựng vững chắc trên nền tảng Spring Boot và frontend sử dụng ReactJS, với cơ sở dữ liệu MySQL để đảm bảo khả năng lưu trữ và quản lý dữ liệu hiệu quả. Đặc biệt, dự án đã tích hợp thành công hỗ trợ bằng Trí tuệ nhân tạo (AI) qua workflow n8n đạt được các chức năng độc đáo như Gợi ý cải thiện CV và Luyện tập phỏng vấn mô phỏng.

Hệ thống đã giải quyết được những bài toán thực tế trong quy trình tuyển dụng. Đối với ứng viên, nền tảng không chỉ là nơi tìm việc mà còn là một công cụ phát triển sự nghiệp, giúp giải quyết vấn đề hồ sơ chưa đủ nổi bật và thiếu kinh nghiệm phỏng vấn thực tế. Đồng thời, đối với nhà tuyển dụng, FlexiConnect cung cấp một quy trình quản lý tin đăng và hồ sơ ứng tuyển tập trung, cùng với cơ chế xác thực tài khoản để xây dựng một môi trường tuyển dụng an toàn, đáng tin cậy.

Những ưu điểm nổi bật của hệ thống nằm ở việc tích hợp AI để tạo ra lợi thế cạnh tranh, cấu trúc phân quyền vai trò rõ ràng và quy trình nghiệp vụ khép kín. Tuy nhiên, dự án vẫn còn một số hạn chế có thể được cải thiện trong tương lai. Cụ thể, mô hình AI còn phụ thuộc vào dịch vụ bên ngoài và hệ thống chưa có tính năng tự động gợi ý việc làm (Recommendation System). Việc phát triển các tính năng này cùng với các chức năng cộng đồng như đánh giá công ty sẽ là hướng phát triển tiềm năng cho dự án.

Thông qua dự án này, tôi đã học hỏi và áp dụng thành công nhiều công nghệ mới, đồng thời củng cố vững chắc kỹ năng phân tích và xây dựng một hệ thống phần mềm hoàn chỉnh.

5.2. Hướng phát triển

Để nâng cao giá trị và đảm bảo khả năng cạnh tranh lâu dài cho hệ thống "FlexiConnect", các hướng phát triển trong tương lai sẽ tập trung vào ba mảng chính:

cá nhân hóa trải nghiệm người dùng, xây dựng cộng đồng và tối ưu kiến trúc hệ thống.

1. Gợi ý việc làm (Recommendation System)

Hạn chế hiện tại của hệ thống là việc khám phá cơ hội vẫn phụ thuộc nhiều vào thao tác tìm kiếm thủ công, đòi hỏi nỗ lực và thời gian từ người dùng. **Để khắc phục** nó, một hướng phát triển chiến lược là xây dựng hệ thống gợi ý việc làm, có khả năng tự động đề xuất các cơ hội phù hợp. **Giải pháp** này sẽ phân tích dữ liệu từ hồ sơ, kỹ năng, và lịch sử ứng tuyển của ứng viên để tự động kết nối họ với những tin tuyển dụng tương thích nhất. Lợi ích mang lại là rất lớn khi ứng viên sẽ tiết kiệm thời gian và không bỏ lỡ cơ hội tiềm năng, trong khi nhà tuyển dụng sẽ tiếp cận được nguồn ứng viên chất lượng và phù hợp hơn. **Về mặt kinh doanh**, đây là bước tiến hóa giúp "FlexiConnect" từ một công cụ tìm kiếm đơn thuần trở thành một cố vấn sự nghiệp cá nhân hóa, tạo ra lợi thế cạnh tranh và giá trị độc đáo cho người dùng.

2. Mở rộng Dịch vụ Thanh toán & Gói Dịch vụ Cao cấp

Hiện tại, hệ thống đã có nền tảng cho việc thương mại hóa với việc tích hợp cổng thanh toán MoMo. Tuy nhiên, phương thức thanh toán và mô hình này còn khá đơn giản và cần được mở rộng để tối ưu hóa tiềm năng hệ thống. Vì vậy, hướng phát triển sẽ tập trung vào việc **đa dạng hóa phương thức thanh toán và làm mở rộng thêm đa dạng các gói dịch vụ cao cấp**. **Giải pháp** bao gồm việc tích hợp thêm các cổng thanh toán phổ biến trong nước (VNPay, ZaloPay) và quốc tế (Stripe, PayPal) để tăng sự tiện lợi cho khách hàng. Đồng thời, sẽ phát triển thêm các quyền lợi giá trị cao cho nhà tuyển dụng, đặc biệt là **mở khóa các tính năng AI cho nhà tuyển dụng** để hỗ trợ sàng lọc và đánh giá ứng viên, giúp họ tiết kiệm thời gian và tăng độ chính xác trong tuyển dụng. **Về mặt kỹ thuật**, điều này đòi hỏi việc nâng cấp kiến trúc của **PaymentService** hiện tại để trở nên linh hoạt hơn, và **về mặt kinh doanh**, nó sẽ nâng cao mạnh mẽ giá trị mà FlexiConnect cung cấp cho khách hàng doanh nghiệp.

3. Xây dựng Các tính năng Tương tác Cộng đồng

Sự tin tưởng và thông tin được phổ rộng là yếu tố then chốt trong thị trường tuyển dụng. Tuy nhiên, nền tảng hiện tại chủ yếu phục vụ các giao dịch một chiều, chưa có cơ chế để xây dựng lòng tin và sự gắn kết sâu sắc. Do đó, một hướng mở rộng quan

trọng là phát triển các tính năng cộng đồng nhằm thúc đẩy một hệ sinh thái tuyển dụng minh bạch và bền vững. **Giải pháp** bao gồm việc cho phép ứng viên đánh giá công ty, tạo ra một diễn đàn trao đổi kiến thức ngành nghề, và cung cấp chức năng theo dõi nhà tuyển dụng để cập nhật tin tức. Những tính năng này không chỉ gia tăng sự gắn bó của người dùng mà còn tạo ra nguồn nội dung phong phú, giúp ứng viên có cái nhìn toàn diện hơn trước khi quyết định ứng tuyển. **Về mặt kỹ thuật**, việc triển khai đòi hỏi mở rộng cơ sở dữ liệu và xây dựng một hệ thống thông báo real-time đủ mạnh để phục vụ các tương tác này.

4. Tối ưu và Mở rộng Kiến trúc Hệ thống

Kiến trúc nguyên khối (**monolithic**) hiện tại tuy phù hợp cho giai đoạn đầu nhưng sẽ khó đáp ứng khi quy mô hệ thống và số lượng người dùng gia tăng. Để giải quyết, hướng phát triển tiếp theo là chuyển đổi dần sang kiến trúc **microservices** nhằm nâng cao khả năng bảo trì, mở rộng và triển khai. Đồng thời, có thể tích hợp công cụ tìm kiếm chuyên dụng như **Elasticsearch** để cải thiện hiệu năng tìm kiếm và mở rộng hệ thống hỗ trợ đa ngôn ngữ nhằm phục vụ nhu cầu quốc tế hóa. Những cải tiến này sẽ giúp FlexiConnect duy trì tính linh hoạt, đáp ứng tốt hơn nhu cầu của người dùng và hỗ trợ phát triển lâu dài.

5.3. Thử thách và khó khăn

Trong quá trình định hướng và triển khai các hướng phát triển cho hệ thống "FlexiConnect", bên cạnh những giá trị và lợi ích to lớn, cùng theo đó sẽ phải đối mặt với không ít thử thách và khó khăn, đặc biệt toàn bộ dự án được thực hiện bởi cá nhân, từ khâu phân tích, thiết kế cho đến phát triển và triển khai.

1. Lợi ích có được:

Nâng cao trải nghiệm người dùng và cá nhân hóa: Việc tích hợp hệ thống gợi ý sẽ giúp ứng viên tiết kiệm đáng kể thời gian tìm kiếm, chủ động kết nối họ với các cơ hội phù hợp nhất. Ứng viên sẽ bớt phải tự mò mẫm tìm việc, mà hệ thống chủ động gợi ý, giúp tiết kiệm thời gian và tạo cảm giác thoải mái hơn.

Mở ra cơ hội kinh doanh và tăng trưởng bền vững: Việc tích hợp các công thanh toán và triển khai gói dịch vụ cao cấp sẽ tạo ra một mô hình kinh doanh rõ ràng, mang

lại doanh thu để tái đầu tư và phát triển. Hỗ trợ đa ngôn ngữ cũng là nền tảng để mở rộng thị trường ra quốc tế.

Tăng cường sự tin cậy và minh bạch: Các tính năng cộng đồng như đánh giá công ty và diễn đàn thảo luận sẽ cung cấp cho ứng viên những góc nhìn đa chiều, thực tế về môi trường làm việc. Điều này giúp xây dựng một thị trường tuyển dụng minh bạch hơn, nơi ứng viên có đủ thông tin để đưa ra quyết định đúng đắn.

Đảm bảo hiệu năng và khả năng phát triển lâu dài: Việc chuyển đổi sang kiến trúc Microservices và tích hợp Elasticsearch sẽ giúp hệ thống hoạt động nhanh và ổn định hơn khi quy mô người dùng tăng lên. Kiến trúc linh hoạt cũng cho phép việc phát triển và triển khai các tính năng mới trong tương lai trở nên nhanh chóng và độc lập hơn.

2. Thách thức phải đối mặt:

Đảm bảo chất lượng của hệ thống gợi ý AI: Hệ thống gợi ý việc làm đòi hỏi một lượng lớn dữ liệu chất lượng để hoạt động hiệu quả. Khó khăn lớn là làm sao gợi ý tốt cho người dùng mới khi chưa có dữ liệu và đảm bảo các gợi ý đưa ra thực sự phù hợp, tránh làm người dùng mất lòng tin vì các kết quả không liên quan.

Trách nhiệm bảo mật trong giao dịch thanh toán: Việc tích hợp nhiều cổng thanh toán đặt ra một trách nhiệm lớn về bảo mật thông tin nhạy cảm của khách hàng. Việc đảm bảo tuân thủ các tiêu chuẩn an toàn, xử lý lỗi và các vấn đề phát sinh một cách chuyên nghiệp là một bài toán phức tạp và áp lực đối với một nhà phát triển cá nhân.

Quản lý và kiểm duyệt nội dung cộng đồng: Với nguồn lực của một người, việc xây dựng một hệ thống kiểm duyệt hiệu quả (kết hợp tự động và thủ công) để xử lý các nội dung tiêu cực hoặc sai sự thật là một thách thức lớn về mặt vận hành, đòi hỏi phải có quy tắc cộng đồng rõ ràng và công cụ hỗ trợ mạnh mẽ. (chẳng hạn, xử lý bình luận tiêu cực hoặc tin đăng sai sự thật cần một cơ chế kiểm duyệt hiệu quả)

Sự phức tạp trong việc vận hành kiến trúc Microservices: Việc chuyển đổi từ kiến trúc nguyên khối sang Microservices sẽ làm tăng đáng kể sự phức tạp trong quản lý. Điều này đòi hỏi phải cần có thêm nhiều kiến thức về DevOps và các công cụ quản lý hệ thống phân tán, nâng cao một loạt kỹ năng chuyên sâu và các công cụ quản lý để có thể vận hành trơn tru hệ thống.

Tổng kết:

Đồ án với đề tài "Xây dựng Hệ thống web tìm việc làm tích hợp AI - FlexiConnect" đã hoàn thành mục tiêu xây dựng một nền tảng tuyển dụng toàn diện. Hệ thống không chỉ đóng vai trò là cầu nối giữa nhà tuyển dụng và ứng viên mà còn là công cụ hỗ trợ thiết thực, giúp nâng cao năng lực cạnh tranh của người dùng trước những thách thức của thị trường lao động hiện đại. Được phát triển trên nền tảng Spring Boot và ReactJS, dự án đã tích hợp thành công các tính năng AI độc đáo như gợi ý cải thiện CV và luyện tập phỏng vấn.

Quá trình này là cơ hội quý báu để tôi áp dụng kiến thức chuyên ngành vào việc giải quyết một bài toán thực tế, qua đó củng cố và hoàn thiện kỹ năng phát triển phần mềm full-stack. Mặc dù đã đáp ứng các yêu cầu cốt lõi, dự án vẫn còn nhiều tiềm năng để mở rộng trong tương lai và là minh chứng rõ nét cho quá trình học hỏi, nỗ lực của bản thân trong suốt thời gian học tập.

TÀI LIỆU THAM KHẢO

- [1] S. Musib, Spring Boot in Practice, NY: Manning Publications Co., 1964.
- [2] GeeksforGeeks, "Spring Boot - Architecture," [Online]. Available: <https://www.geeksforgeeks.org/springboot/spring-boot-architecture/>. [Accessed 11 Sep 2025].
- [3] T. Point, "ReactJS Tutorial," [Online]. Available: <https://www.tutorialspoint.com/reactjs/index.htm>. [Accessed 11 9 2025].
- [4] n8n, "n8n Docs," [Online]. Available: <https://docs.n8n.io/>. [Accessed 13 9 2025].
- [5] D. Docs, "Get Docker," [Online]. Available: <https://docs.docker.com/get-started/get-docker/>. [Accessed 13 9 2025].
- [6] D. Community, "free website hosting platforms," [Online]. Available: <https://dev.to/vishnusatheesh/5-static-website-hosting-platforms-for-your-next-project-2ahf>. [Accessed 17 9 2025].

PHỤ LỤC

Số hiệu bảng	Tên bảng	Các trường	Mô tả	Khóa chính	Khóa ngoại
A.1	role	id: INT (PK), role_name: VARCHAR(50)	Quản lý các vai trò trong hệ thống.	id	Không có
A.2	user	id: INT (PK), email: VARCHAR(100), password: VARCHAR(255)	Lưu trữ thông tin tài khoản cơ bản.	id	Không có
A.3	user_role	user_id: INT (PK), role_id: INT (PK)	Gán vai trò cho người dùng.	user_id, role_id	user_id tham chiếu user(id), role_id tham chiếu role(id)
A.4	candidate	id: INT (PK), user_id: INT,	Lưu trữ thông tin hồ sơ của ứng viên.	id	user_id tham chiếu user(id)
A.5	employer	id: INT (PK), user_id: INT	Lưu thông tin chi tiết của nhà tuyển dụng.	id	user_id tham chiếu user(id)
A.6	job_post	id: INT (PK), employer_id: INT	Lưu thông tin các tin tuyển dụng.	id	employer_id tham chiếu employer(id)
A.7	application	id: INT (PK),	Lưu thông	id	job_post_id tham

		job_post_id: INT, candidate_id: INT	tin một lượt ứng tuyển.		chiều job_post(id), candidate_id tham chiều candidate(id)
A.8	employer_ email_log	id: INT (PK), application_id: INT, employer_id: INT, candidate_id: INT	Ghi lại lịch sử email nhà tuyển dụng gửi.	id	application_id tham chiều application(id), employer_id tham chiều employer(id), candidate_id tham chiều candidate(id)
A.9	skill	id: INT (PK)	Danh mục các kỹ năng.	id	Không có
A.10	candidate_ skill	id: INT (PK), candidate_id: INT, skill_id: INT	Lưu các kỹ năng của ứng viên.	id	candidate_id tham chiều candidate(id), skill_id tham chiều skill(id)
A.11	education_ history	id: INT (PK), candidate_id: INT	Lịch sử học vấn của ứng viên.	id	candidate_id tham chiều candidate(id)
A.12	work_expe rience	id: INT (PK), candidate_id: INT	Kinh nghiệm làm việc của ứng viên.	id	candidate_id tham chiều candidate(id)
A.13	cv_suggest ion	id: INT (PK), candidate_id: INT	Lưu các gợi ý của AI để	id	candidate_id tham chiều

			cải thiện CV.		candidate(id)
A.14	follow_employer	candidate_id: INT (PK), employer_id: INT (PK)	Lưu việc ứng viên theo dõi nhà tuyển dụng.	candidate_id, employer_id	candidate_id tham chiếu candidate(id), employer_id tham chiếu employer(id)
A.15	saved_job	candidate_id: INT (PK), job_post_id: INT (PK)	Lưu các tin tuyển dụng ứng viên đã lưu.	candidate_id, job_post_id	candidate_id tham chiếu candidate(id), job_post_id tham chiếu job_post(id)
A.16	interview_session	id: INT (PK), candidate_id: INT, job_post_id: INT	Lưu thông tin một phiên phỏng vấn AI.	id	candidate_id tham chiếu candidate(id), job_post_id tham chiếu job_post(id)
A.17	interview_turn	id: INT (PK), session_id: INT	Lưu từng lượt hỏi-đáp trong phiên phỏng vấn.	id	session_id tham chiếu interview_session(id)
A.18	package	id: INT (PK)	Lưu thông tin các gói dịch vụ.	id	Không có
A.19	payment_transaction	id: INT (PK), user_id: INT, package_id: INT	Ghi lại lịch sử giao dịch thanh toán.	id	user_id tham chiếu user(id), package_id tham chiếu package(id)

A.20	user_package	id: INT (PK), user_id: INT, package_id: INT, transaction_id: INT	Quản lý gói dịch vụ người dùng đã mua.	id	user_id tham chiếu user(id), package_id tham chiếu package(id), transaction_id tham chiếu payment_transaction(id)
A.21	notification	id: INT (PK)	Lưu nội dung các thông báo.	id	Không có
A.22	notification_user	id: INT (PK)	Gán thông báo đến người dùng.	id	notification_id tham chiếu notification(id), user_id tham chiếu user(id)

Bảng 5.1: Mô tả chi tiết các bảng cơ sở dữ liệu